



LBISTArchitect™ Process Guide

Software Version 2015.4

December 2015

**© 2004-2015 Mentor Graphics Corporation
All rights reserved.**

This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in whole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

U.S. GOVERNMENT LICENSE RIGHTS: The software and documentation were developed entirely at private expense and are commercial computer software and commercial computer software documentation within the meaning of the applicable acquisition regulations. Accordingly, pursuant to FAR 48 CFR 12.212 and DFARS 48 CFR 227.7202, use, duplication and disclosure by or for the U.S. Government or a U.S. Government subcontractor is subject solely to the terms and conditions set forth in the license agreement provided with the software, except for provisions which are contrary to applicable mandatory federal laws.

TRADEMARKS: The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the owner of the Mark, as applicable. The use herein of a third-party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at: www.mentor.com/trademarks.

The registered trademark Linux[®] is used pursuant to a sublicense from LMI, the exclusive licensee of Linus Torvalds, owner of the mark on a world-wide basis.

Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: www.mentor.com
SupportNet: supportnet.mentor.com/

Send Feedback on Documentation: supportnet.mentor.com/doc_feedback_form

Table of Contents

Chapter 1

Implementing Logic BIST	13
Generating Logic BIST with LBISTArchitect.	13
MTPI and Fault Coverage.	13
Basic Process Flow	13
Architecture Overview of BIST Circuitry	15
Basic LBISTArchitect Tool Phases	15
BIST-Ready Phase	16
BIST Controller Generation Phase	16
Fault Simulation and Signature Generation Phase	16
Optional LBISTArchitect Capabilities	17
Boundary Scan Insertion.	17
BIST Verification	17

Chapter 2

Preparing the Design for Logic BIST	19
BIST-Ready Process Flow	19
Making the Design Ready for Logic BIST	21
Inputs and Outputs of First Run of BIST-Ready Phase	21
Flow of First Run of BIST-Ready Phase	22
Inputs and Outputs of Second Run of BIST-Ready Phase	23
Flow of Second Run of BIST-Ready Phase	24
BIST-Ready Phase Tool Concepts.	27
Identifying System Clocks	28
Identifying Pin Constraints	28
Checking Your Design for Rule Violations	29
Setting Up LBISTArchitect to Insert Scan Chains	30
Controlling Inputs and Correcting Internal X-Generators	31
Inserting Test Points Using MTPI	32
Observation Points for Primary Outputs.	36

Chapter 3

Generating the BIST Controller	39
Understanding the BIST Controller.	39
Examining the BIST Controller Module.	40
Sub-Module Summary	40
PRPG and Phase Shifter	41
MISR and Space Compactor.	41
Pattern Counter	42
Shift Counter	42
BIST-Controller Phase Inputs.	42
BIST-Controller Phase Outputs	42

Examining BIST Controller Generation	45
Invoking and Using LBISTArchitect	46
Examining an Example BIST Controller Generation Run	47
Performing BIST Controller Generation	50
What Commands Should You Use?	51
Setting Up the BIST Controller	52
Connecting Internal Scan Chains	54
Capturing Window Waveforms for At-speed Testing	55
Running the BIST Controller Generation	57
Saving and Writing the Output Files	57
Using On-Chip PLL Clocks for BIST Capture	58
On-Chip Registers	58
Internal and External Views	59
Limitations For Using PLL Clocks	59
The BIST Controller PLL Interface Setup Process	60
BIST Controller PLL Interface Generation Examples	64
 Chapter 4	
At-speed Testing of Designs with High Speed Clocks	71
Generating Clock Signals	71
High-level Architecture	72
lc_lbist_control Block	72
Clock Control Low-level Architecture	73
Generating Clock Signals During Pattern Loading	74
Generating the slow_bist_clk Signal	74
Glitch Recovery in sg_slow_clock_gen Circuit	76
Clock Path and Waveforms When Loading Patterns at bist_clk Frequency	77
Clock Path and Waveforms When Loading Patterns at 1/n bist_clk Frequency	78
Generating Scan Enable Signals	80
Chain Test Patterns	80
Clock Signals During Scan Enable Switching	80
Generating Clock Signals During Capture Window	80
Stuck-at Testing	81
At-speed Testing (Transition Faults)	82
Timing Closure	88
LBIST Scan Chain Shifting	88
LBIST At-speed Capture	89
LBIST Scan Enable Switching	90
LBIST Diagnostics Scan Chain Unload	90
 Chapter 5	
Performing Fault Simulation	91
Understanding Fault Simulation	91
Fault Simulation Types	92
Test and Fault Coverage	92
PRPG Output and MISR Signature	94
Supplemental Fault Simulation Information	95
The Fault Simulation Process	95

Table of Contents

Fault Simulation Process Inputs	96
Fault Simulation Process Dependencies	98
Fault Simulation Process Outputs	98
Performing Design Fault Simulation	98
Running Design Fault Simulation	98
Concluding Design Fault Simulation	99
Performing Core Fault Simulation	99
Running Core Fault Simulation	100
Concluding Core Fault Simulation	100
Examining Fault Simulation Results	100
Interpreting the Transcript	101
 Chapter 6	
Performing Manufacturing Diagnosis	103
Understanding the Diagnostic Flow	104
Single-Step Simulation and MISR Signature Access	104
TAP Controller and Interconnects	104
Basic Diagnostic Flow	104
Before You Begin Diagnosis	106
The Diagnostic Process	106
Diagnosis Process Inputs	108
Diagnosis Process Outputs	108
Performing Diagnosis	109
Using a TAP Controller and Interconnects	109
Step 1 — Generate Basic BIST Test Patterns and Test on the ATE	109
Create the Patterns	109
Step 2 — Generate Failure Investigation Patterns	110
Step 3 — Run Investigation Patterns on the ATE	111
Step 4 — Format the ATE Failure File	111
Step 5 — Map Failures and Create Unload Patterns	112
Step 6 — Run Unload Patterns on the ATE	114
Step 7 — Format the Final ATE Failure File	114
Step 8 — Map the Failing Patterns and Creating Diagnostic File	115
Running Manufacturing Diagnosis	118
Invoking Tessent Diagnosis	118
Manual Verses Automatic Invocation	118
 Appendix A	
Getting Help	121
Documentation	121
Mentor Graphics Support	122
 Appendix B	
Logic BIST Controller	
Data Sheet	123
Architectural Overview	123
Sub-Module Detail	124
General Operation	125

Multi-Phase Test Point Insertion (MTPI)	126
Lockup Latch Timing During Shifting	127
During BIST Mode	128
During Scan Chain Concatenation	128
BIST Controller Interface Description	130
Overview of Dual-Register Interface	130
Decoding the Contents of the Data Register	133
Using Delay Cycles for Scan Enable Transition	135
Layout of Control Register Bits within Data Register	137
Layout of Pattern Counter Start and Stop in Data Register	137
Layout of PRPG within Data Register	137
Layout of MISR within Data Register	138
Layout of the Scan Enable Control Register	138
Layout of the Shift Speed Register	139
Control Signals to the Core	140
ATPG Access for the Scan Chains	141
Asynchronous Logic BIST Test Signals	141
Miscellaneous BIST-Related Signals	142
Timing Between ATE and BIST Controller	143
Loading the dselect Register	143
Loading the Data Register	144
Typical Logic BIST Control Sequence	144
Timing Between BIST Controller and Core	145
Loading and Applying a Single Pattern Into the Core	145
Driving the MTPI Signals Into the Core	146
Brief Description of Sub-blocks	146
Linear Feedback Shift Register	148
sd_scan_driver	148
sm_scan_monitor	149
pc_pattern_count	150
se_scan_enable	150
sc_shift_count	151
co_clock_control	151
cm_clock_muxes	151
eg_enable_generator	151
cc_concat_chain	152
lc_lbist_control	152
li_lbist_interface	152
ts_tristate	152

Appendix C

Example Fault Simulation Dofile	153
Deconstruction of Fault Simulation Dofile	153
Set Design Rule Checks	158
Define the Scan Groups	158
Define the Scan Chains	158
Define I/O Pin Constraints	158
Define the PRPGs	159

Table of Contents

Define the MISR.....	159
Add Mask	159
Define the MTPI Controller	159
Define Capture Waveforms	159
Define Clocks	160
Run the Fault Simulation	160

Index

Third-Party Information

End-User License Agreement

List of Examples

Example 3-1. Interactive LBISTArchitect BIST Controller Run Example.....	48
Example 3-2. Dofile LBISTArchitect BIST Controller Run Example	49
Example 3-3. BIST Controller Generation Run Example	51
Example 5-1. Design Fault Simulation Example Invocation.....	99
Example 5-2. Core Fault Simulation Example Invocation	100
Example 6-1. Step 1 — Basic BIST Test Creation Example Invocation	110
Example 6-2. Step 2 — Failure Investigation Pattern Creation Example Invocation.....	111
Example 6-3. Time-Based Failure File Format Example	111
Example 6-4. Cycle-Based Failure File Format Example	112
Example 6-5. Step 5 — Map Failures Example Invocation	113
Example 6-6. Step 5 — Pattern Unload Example Invocation	114
Example 6-7. Time-Based Unload Format Example.....	115
Example 6-8. Cycle-Based Unload Format Example	115
Example 6-9. Diagnostic Failure Map Example	116
Example 6-10. Failure File Mapping and Diagnostic File Creation Invocation Example. . .	117

List of Figures

Figure 1-1. Basic LBISTArchitect Process Flow	14
Figure 1-2. BIST Controller RTL	15
Figure 2-1. BIST-Ready Steps within Basic LBISTArchitect Flow	20
Figure 2-2. Inputs and Outputs of 1st Invocation of BIST-Ready Phase	22
Figure 2-3. Inputs and Outputs of 2nd Invocation of BIST-Ready Phase	24
Figure 2-4. Flow Chart for Bounding and MTPI	25
Figure 2-5. Pin Constraint Hardware Added by LBISTArchitect:	29
Figure 2-6. Input Bounding with Muxes	31
Figure 2-7. Controlling X Sources with Muxes	32
Figure 2-8. MTPI Overview	34
Figure 2-9. Test Point Control	35
Figure 2-10. MTPI Control Point Logic	36
Figure 2-11. Output Bounding	37
Figure 3-1. BIST Controller and Core Design Relationship	39
Figure 3-2. Logic BIST Controller Sub-Modules	40
Figure 3-3. BIST Controller Generation Phase	46
Figure 3-4. Multicycle Path Example Circuit	56
Figure 3-5. LBISTArchitect PLL Generic Interface	58
Figure 3-6. Example BIST Controller PLL Interface with no End Capture	64
Figure 3-7. Example Waveform for Capture Window cap1	67
Figure 3-8. Example BIST Controller PLL Interface with End Capture	68
Figure 4-1. Load and Capture Cycle	71
Figure 4-2. Clock Control Drive Signals	72
Figure 4-3. Clock Control Logic	73
Figure 4-4. Contents of the sg_slow_clock_gen Block	75
Figure 4-5. Clock Generation When Shifting At Full Speed	78
Figure 4-6. Clock Waveforms When Shifting At Full Speed	78
Figure 4-7. Clock Generation When Shifting At Slow Speed	79
Figure 4-8. Slow Shift Cycles	79
Figure 4-9. Clock Waveforms When Shifting At 1/3	79
Figure 4-10. Clocking with Asynchronous Set and Reset Signals	81
Figure 4-11. Transition Fault Testing	83
Figure 4-12. Hardware for Pulse Mode	84
Figure 4-13. Clock Waveforms During Pulse Mode	84
Figure 4-14. Hardware for Hold Mode	85
Figure 4-15. Waveforms during Hold Mode	85
Figure 4-16. Hardware for Stretch Mode	86
Figure 4-17. Waveforms During Stretch Mode	86
Figure 4-18. Multiple Capture Windows	88
Figure 4-19. Unloading Scan Chain Contents During Diagnostics Testing	90

Figure 5-1. Typical Fault Simulation Statistics Report	94
Figure 5-2. PRPG Output and MISR Signature.....	95
Figure 5-3. Interpreting the Fault Simulation Transcript Results	101
Figure 6-1. Mapped Failures	113
Figure 6-2. Sample Diagnosis Dofile.....	119
Figure B-1. Logic BIST Controller to Core Data Paths.....	124
Figure B-2. Logic BIST Controller	125
Figure B-3. Control Point Gating	127
Figure B-4. Clock Phase Relationships	128
Figure B-5. Out-of-Phase Clock Wiring	129
Figure B-6. Shadow Registers	129
Figure B-7. Shadow Registers and Controllability	129
Figure B-8. Contents of SRCELL Block	131
Figure B-9. Layout of dselect Register	131
Figure B-10. Layout of Data Register	132
Figure B-11. Synchronization Timing	133
Figure B-12. Data_Register to Internal Register Mapping	134
Figure B-13. Example Scan Enable Transition Switching	136
Figure B-14. dselect Register Load	143
Figure B-15. data register load	144
Figure B-16. Scan Chain Pattern Load Timing	145
Figure B-17. Phase Control Signal Timing	146
Figure B-18. BIST Controller Sub-blocks.....	147
Figure B-19. 4-Bit PRPG	149
Figure B-20. 4-Bit MISR	149

List of Tables

Table 3-1. Logic BIST Controller Module Summary	40
Table 3-2. BIST-Controller Phase Inputs	42
Table 3-3. BIST-Controller Phase Outputs	42
Table 3-4. BIST Controller Generation Phase Controller-Related Commands	52
Table 3-5. BIST Controller Generation Phase Internal Scan Chain Commands	54
Table 3-6. BIST Controller Phase Capture Window-Related Commands	55
Table 3-7. BIST Controller Generation Save Commands	57
Table 3-8. BIST Controller PLL Interface Setup Process	60
Table 3-9. BIST Controller PLL Interface Setup Commands	60
Table 4-1. Clock Control Modules	72
Table 4-2. Decoding the shift_speed register	75
Table 5-1. Fault Class Codes and Names	93
Table 5-2. Supplemental Fault Simulation References	95
Table 5-3. Fault Simulation Process	96
Table 5-4. Inputs for the Design Fault Simulation Process	97
Table 5-5. Inputs for the Core Fault Simulation Process	97
Table 5-6. Simulation Process Outputs	98
Table 5-7. Explanation of Transcript Results	102
Table 6-1. Diagnostic Process	107
Table 6-2. Inputs for the Diagnosis Process	108
Table B-1. Basic Logic BIST Controller Interface	130
Table B-2. Decoding of dselect Register	135
Table B-3. Control Register Bits	137
Table B-4. Layout of the Scan Enable Control Register ON Bit Positions	138
Table B-5. Layout of the Scan Enable Control Register OFF Bit Positions	139
Table B-6. Layout of Shift Speed Register	139
Table B-7. Core Interface	140
Table B-8. ATPG interface	141
Table B-9. Parallel Interface Test Signals	142
Table B-10. On-Chip Comparator Test Signals	142
Table B-11. Miscellaneous BIST signals	142

Chapter 1

Implementing Logic BIST

Generating Logic BIST with LBISTArchitect

LBISTArchitect™ is a logic BIST (Built-in-Self-Test) fault analysis and scan insertion application, which embeds much of the tester functionality into the device itself. Using LBISTArchitect, you insert a BIST controller into the design; subsequently, this BIST controller generates and applies a series of scan patterns, the results of which you capture on-chip. This methodology supports testing in environments where the device has no connection to the ATE.

i **Tip:** The *Scan and ATPG User's Manual* details foundational Design-for-Test principles including fault modeling, scan chains, and ATPG.

LBISTArchitect generates Verilog you can synthesize with any standard synthesis tool. LBISTArchitect automatically identifies areas of the design that are untestable with pseudorandom patterns or areas that will result in poor fault coverage, and then provides recommended design enhancements.

MTPI and Fault Coverage

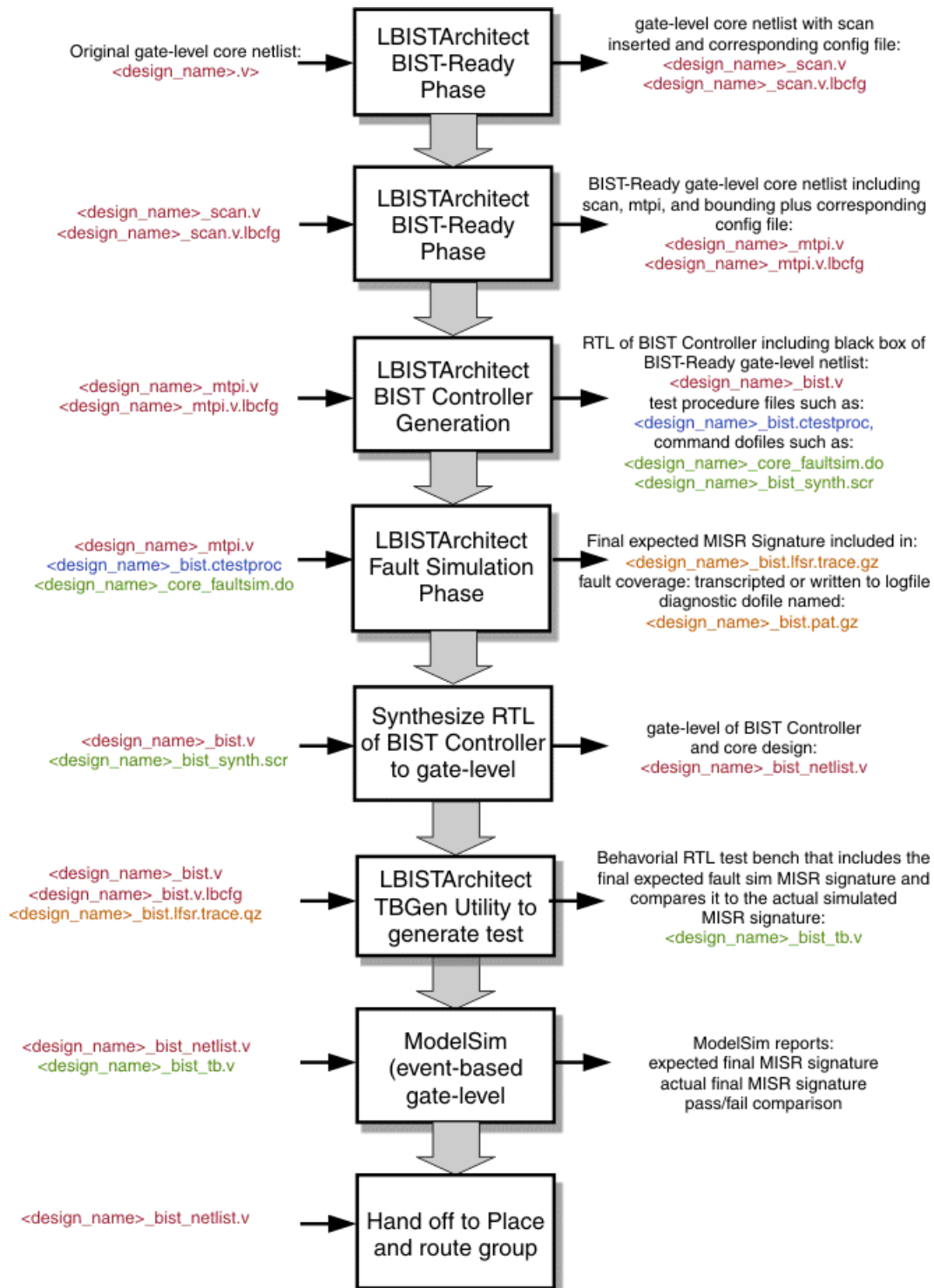
LBISTArchitect uses a unique Multi-Phase Test Point Insertion (MTPI) technology for maximizing a fault coverage of your design while minimizing the impact on the design's area, routing, and critical timing. For areas of a design where testing can be difficult using pseudorandom patterns, MTPI adds control and observation points, making these areas testable. The MTPI methodology uses only one-tenth of the area overhead of traditional BIST methods and ensures that test points are not added to timing-critical areas or paths within the design.

LBISTArchitect supports both the stuck-at and transition fault model types. The tool also supports at-speed testing by generating a BIST controller that handles multiple-clock frequencies, and X-state tolerant logic. LBISTArchitect also automatically creates command dofiles to work with BSDArchitect™ (for boundary scan insertion), and FlexTest™ (sequential fault simulation).

Basic Process Flow

Figure 1-1 shows the most basic process flow for LBISTArchitect.

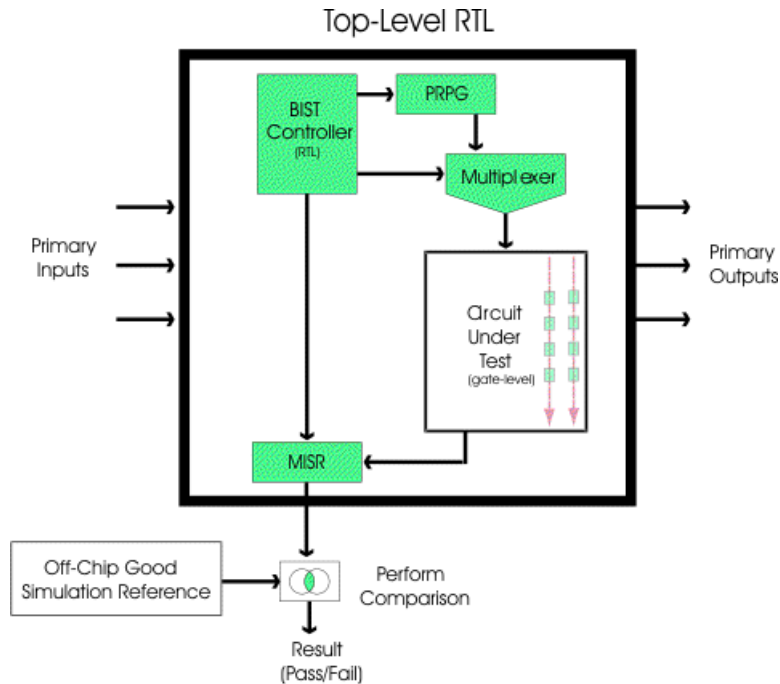
Figure 1-1. Basic LBISTArchitect Process Flow



Architecture Overview of BIST Circuitry

Figure 1-2 shows the main sections of the LBISTArchitect logic BIST controller in green.

Figure 1-2. BIST Controller RTL



In Figure 1-2, the pseudorandom patterns generated by the PRPG are under control of the BIST controller. In test mode, multiplexers pass the test patterns to the device. The MISR collects and compacts the outputs from the device. The resulting signature is then compared with the reference. The comparison is not usually carried out on-chip. The MISR signature is serially scanned out of the device and compared against the off-chip reference.

Basic LBISTArchitect Tool Phases

The LBISTArchitect flow consists of the following three primary phases:

- [BIST-Ready Phase](#)
- [BIST Controller Generation Phase](#)
- [Fault Simulation and Signature Generation Phase](#)

The other phases are optional (such as boundary scan insertion and sequential fault simulation) depending on your design requirements and your company’s overall design process flow—see [“Optional LBISTArchitect Capabilities”](#).

BIST-Ready Phase

You perform this phase first. This ensures you have a design with high-fault coverage that will work in a logic BIST environment. The first invocation of the BIST-Ready phase analyzes the design for testability issues and automatically corrects BIST-related problems. The tool performs Design Rule Checks (DRCs), analyzing the design for clocking issues and scanability problems. You can graphically display the DRC violations to help identify the cause and associated remedy of the problem. During this invocation, the tool also inserts internal scan circuitry into the design.

In a second invocation of the BIST-Ready phase, the tool performs X-bounding, tri-state bus contention correction, and MTPI analysis and insertion to increase the controllability and observability of the design.

LBISTArchitect automatically writes information such as the number of scan chains in the core, scan enable signal names, MTPI control points, and clock names to a binary configuration file each time you exit the BIST-Ready phase. The BIST Controller Generation phase uses this configuration data generated by the BIST-Ready phase. Before moving to the BIST Controller Generation phase, you must first perform the BIST-Ready phase to ensure the core design is testable, and generate the configuration files and new netlists that contain the inserted scan circuitry and MTPI data.

See [“Preparing the Design for Logic BIST”](#) for complete usage and implementation specifics.

BIST Controller Generation Phase

After successfully completing the BIST-Ready phase, you can invoke the BIST Controller Generation phase of LBISTArchitect to generate a synthesizable RTL (Verilog or VHDL) description of the logic BIST circuitry. LBISTArchitect generates the logic BIST controller circuitry, together with its connection to the core design, as an RTL object. The tool generates driver dofiles to aid the downstream processes of logic synthesis, boundary scan implementation, fault simulation, and signature verification.

When you invoke the BIST Controller Generation phase, you use the output netlist from the BIST-Ready phase, and the tool also reads the associated binary configuration data file to set the parameters for generation of the BIST controller.

See [“Generating the BIST Controller”](#) for complete usage and implementation specifics.

Fault Simulation and Signature Generation Phase

The final required phase of the LBISTArchitect flow determines the fault coverage of the BIST patterns when applied to the design, and the final expected MISR signature calculation for the BIST-enhanced design. The fault simulation phase also provides mechanisms to debug the design’s fault coverage. In order to improve fault coverage, the fault analysis (which includes a

graphical display of results) analyzes undetected faults and identifies why they were not detected.

See “[Performing Fault Simulation](#)” for complete usage and implementation specifics.

Optional LBISTArchitect Capabilities

Boundary Scan Insertion	17
BIST Verification	17

Boundary Scan Insertion

Once you insert BIST logic into the core design, you can insert boundary scan circuitry using BSDArchitect, although note that BSDArchitect requires a separate license to run. BSDArchitect produces JTAG logic that is compliant to the IEEE 1149.1 standard. Many designs use IEEE 1149.1 to initiate the logic BIST process. LBISTArchitect generates a driver file for BSDArchitect that generates an IEEE 1149.1 JTAG controller and wires it to the logic BIST controller so that you can initiate the logic BIST process through the IEEE 1149.1 interface.

BSDArchitect can generate a test bench, or a set of test vectors, that use these registers to initialize the internal logic BIST hardware (such as the PRPG in the scan driver, the MISR in the scan monitor, and the pattern counter), initiate the BIST test, and finally scan out the results (MISR value at the end of the test).

Sequential Fault Simulation

After the Fault Simulation phase, you can invoke FlexTest to perform a sequential fault simulation to determine the fault coverage of the logic BIST controller hardware, as opposed to the logic in the core design. To determine the approximate fault coverage of the BIST controller hardware, you must use a sequential fault simulator as the BIST controller portion of the design is non-scan.

In running the sequential fault simulator to compute the overall fault coverage of the BIST hardware, the simulator uses a small subset of the logic BIST patterns to determine the fault coverage of the controller hardware. During this run, you cannot enhance the pattern set, but you can get an estimate of the number of faults that your pattern set detects inside the BIST hardware.

BIST Verification

This task involves generating a test bench for running in an event-based simulator (such as ModelSim®) to verify the basic operation of the LBISTArchitect circuitry prior to sending it to be manufactured. In the BIST verification process, you use the test bench generator utility

(*tbgen*) to create a test bench. The input into the simulator includes a test bench generated by the *tbgen* utility, which provides the simulator with the information to analyze possible timing issues.

The output of the event-based simulation run is a test bench log file that contains either a passing or failing result. If your test bench simulation fails then you can choose to move onto the diagnostic phase of LBISTArchitect for debugging the circuit. To generate a test bench that you need to run in your event-based simulator for the BIST verification task, use the LBISTArchitect -Tbgen invocation switch.

Chapter 2

Preparing the Design for Logic BIST

The set of tasks associated with preparing the design for logic BIST are performed in the BIST-Ready phase. If you are using the LBISTArchitect tool to insert your scan chains, then the BIST-Ready phase requires two separate invocations of the LBISTArchitect -Bist_ready command. If, however, your design already has scan chains inserted, you only need to invoke the BIST-Ready phase once to perform the bounding and test point insertion processes.

The processes within this chapter assume that you are using the LBISTArchitect tool to insert the scan, perform bounding, and insert test points.

Part I: Common Flow in the BIST-Ready Phase of LBISTArchitect:

Making the Design Ready for Logic BIST	21
Inputs and Outputs of First Run of BIST-Ready Phase	21
Flow of First Run of BIST-Ready Phase	22
Performing DRCs and Inserting Scan Circuitry	22
Inputs and Outputs of Second Run of BIST-Ready Phase	23
Flow of Second Run of BIST-Ready Phase	24
Performing Input-Bounding, X-Bounding, and MTPI	25

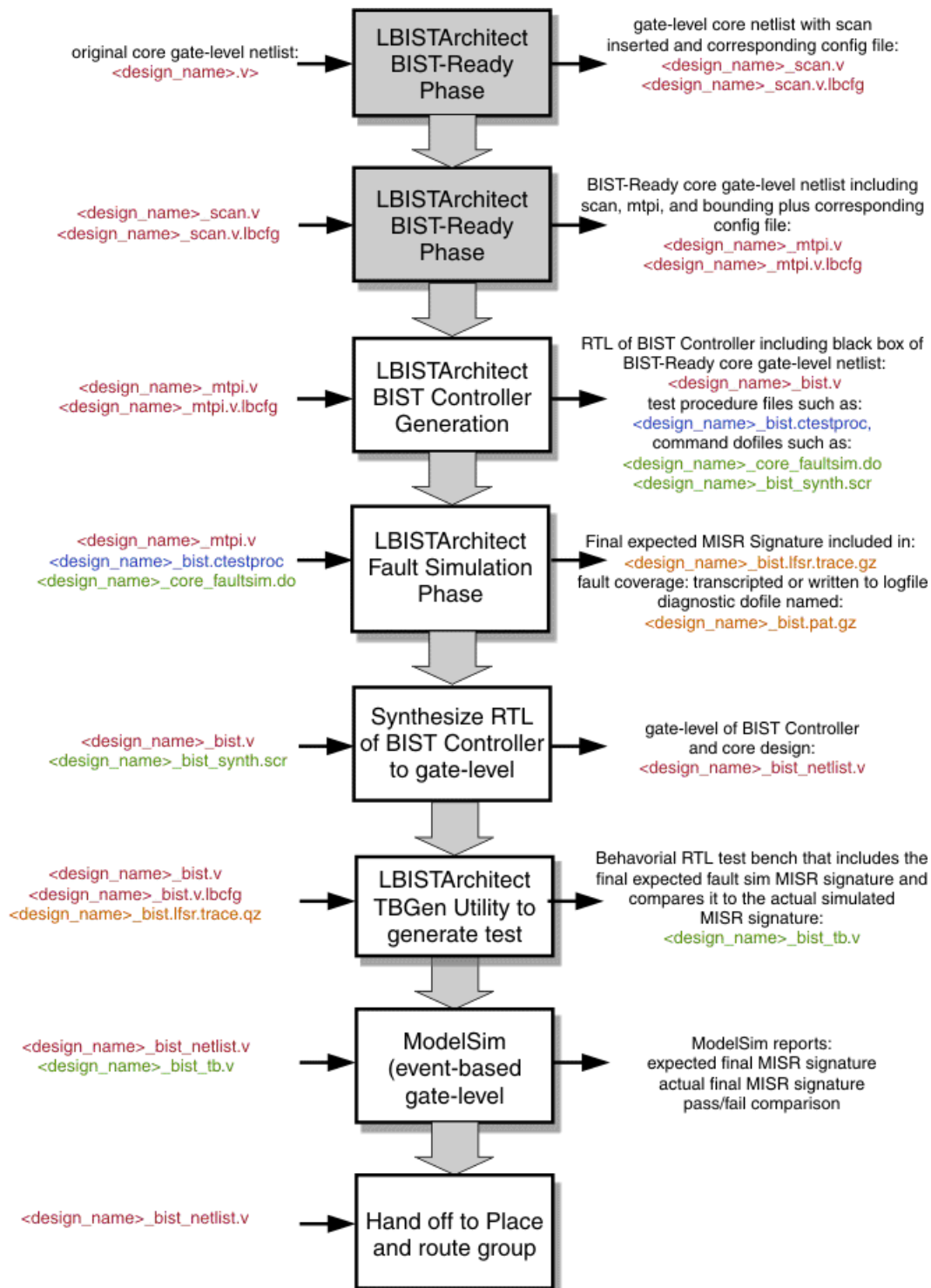
Part II: Concept Details within the BIST-Ready Phase:

BIST-Ready Phase Tool Concepts	27
Identifying System Clocks	28
Identifying Pin Constraints	28
Checking Your Design for Rule Violations	29
Setting Up LBISTArchitect to Insert Scan Chains	30
Controlling Inputs and Correcting Internal X-Generators	31
Inserting Test Points Using MTPI	32
Observation Points for Primary Outputs	36

BIST-Ready Process Flow

Figure 2-1 shows the most basic process flow for the LBISTArchitect tool. The grey boxes show the focus steps of this chapter.

Figure 2-1. BIST-Ready Steps within Basic LBISTArchitect Flow



Making the Design Ready for Logic BIST

The LBISTArchitect tool flow is split into several different phases with the initial tasks requiring you to use the BIST-Ready phase. The goal of the BIST-Ready phase is to ensure that your core design works correctly in a BIST environment (hence the name BIST-Ready). Note that if you are using the tool to insert scan, perform bounding, and insert test points, you must separately invoke the BIST-Ready phase twice, once each time to perform a series of interrelated tasks.

The first invocation of the BIST-Ready phase is to check for design rules violations and then to insert the scan circuitry. The second invocation of the BIST-Ready phase is to perform signal bounding and to then insert the necessary test points using MTPI.

Note



If your design already has scan inserted, you can skip the first run of the LBISTArchitect tool, but because you won't have the configuration output file from the first run, you need to ensure that you correctly define the clocks, scan chains, pin constraints, and have a Test Procedure File. For more information on the stil2mgc utility, refer to “[stil2mgc](#)” section of the *Scan Insertion, ATPG, and Diagnosis Reference Manual*.

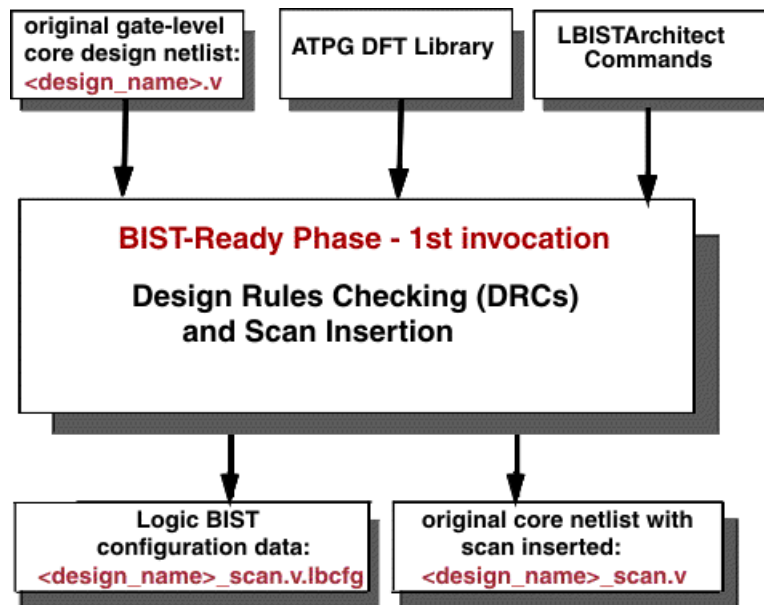
Inputs and Outputs of First Run of BIST-Ready Phase

In the first invocation of the BIST-Ready phase, the LBISTArchitect tool reads in the original gate-level netlist along with the model library. At this point, you are now ready to issue the commands to perform the DRCs followed by the scan insertion. Once you have completed these two required tasks, you need to write out the modified netlist, and the tool then automatically writes out the associated configuration file.

For more information on scan chains and scan insertion, refer to the “[Scan and ATPG Basics](#)” section of the *Scan and ATPG User's Manual*.

[Figure 2-2](#) is a diagram showing all the possible inputs and outputs of the first BIST-Ready run.

Figure 2-2. Inputs and Outputs of 1st Invocation of BIST-Ready Phase



Flow of First Run of BIST-Ready Phase

The following procedure shows flow of the required main tasks that are associated with first invocation of the BIST-Ready phase (which involves the setup and execution of the DRCs and scan insertion).

Performing DRCs and Inserting Scan Circuitry

Because the steps shown here use example data, they do not provide complete command syntax and options, and you should use this example simply as a guide to the process.

To check your design for rule violations and to insert internal scan circuitry for a Verilog design named, *mydesign.v*, using a DFT library named, *mylib.lib*, perform the following procedure:

1. Invoke the initial run of the BIST-Ready phase on the design.

```
shell> <Tessent_Tree_Path>/bin/lbistarchitect -bist_ready  
mydesign.v -verilog -lib mylib.lib
```

Note that if you get a warning about the initialization file not being found, you can ignore the message on the first BIST ready run. For example:

```
// Warning: LBISTArchitect configuration file 'mydesign.v.lbcfg' not  
found.
```

2. Identify and correctly configure the appropriate design clocking information.

```
SETUP> analyze control signals -auto
```

For more information on this task, refer to [Identifying System Clocks](#).

3. Run the rules checking process by setting the system mode to DFT.

```
SETUP> set system mode dft
```

For more information on rules checking, refer to [Checking Your Design for Rule Violations](#).

4. Specify the scan enable line.

```
SETUP> setup scan insertion -sen scan_en
```

5. Treat all sequential instances as scan instances by entering.

```
SETUP> setup scan identification full_scan
```

For more information on rules checking, refer to [Setting Up LBISTArchitect to Insert Scan Chains](#).

6. Identify the scan cell candidates.

```
DFT> run
```

7. Insert the scan cells.

```
DFT> insert test logic -scan on -test_point off -max_length 40
```

8. Display a list of the newly added scan chains.

```
DFT> report scan chains
```

9. Write out the intermediate scan results.

```
DFT> write netlist mydesign.v -verilog
```

```
//Note: Updated LBISTArchitect configuration file  
'mydesign_scan.v.lbcfg'.
```

```
DFT> exit -force
```

Inputs and Outputs of Second Run of BIST-Ready Phase

In the second invocation of the BIST-Ready phase, the LBISTArchitect tool reads in the modified gate-level netlist with scan (*<design_name>_scan.v*) along with the model library, and the configuration file (*<design_name>_scan.v.lbcfg*). At this point, you are now ready to issue the necessary commands to set up the x-generator bounding and MTPI. Once you have completed these two required tasks, you need to write out the modified netlist, and the tool then automatically writes out the associated configuration file.

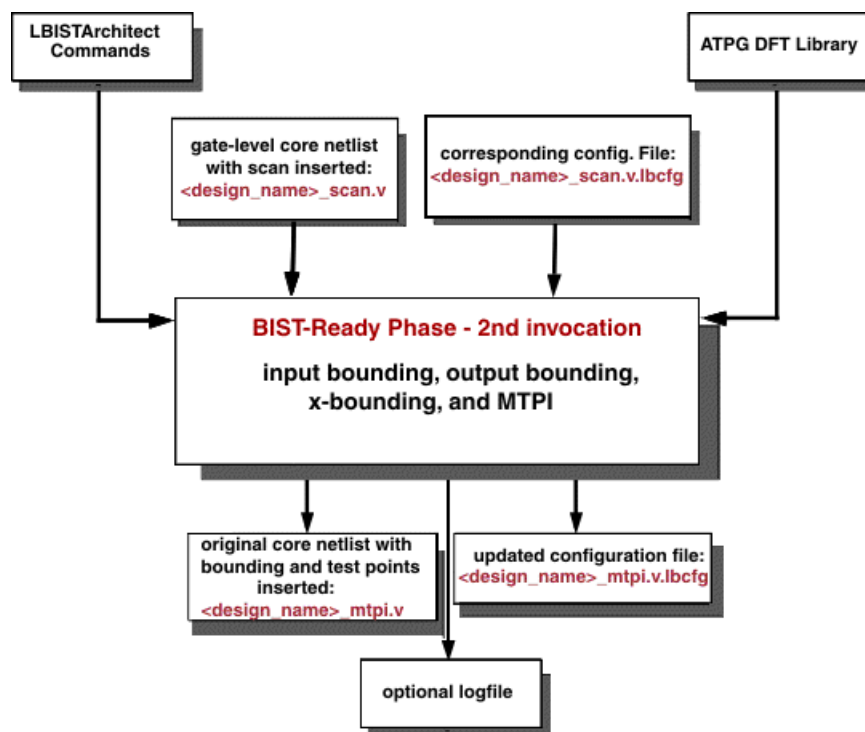
Note



If your design already has scan inserted and you skipped the first run of the BIST-Ready phase, you won't have the configuration file that contains all the tool setup information. Consequently, you need to ensure that you have a correctly formatted Test Procedure File, the clocks and scan chains defined, and constrain the pins.

Figure 2-3 is a diagram showing all the possible inputs and outputs of the first BIST-Ready run assuming that you used the LBISTArchitect tool to insert the scan circuitry. All the examples in this chapter are based on using a Verilog netlist.

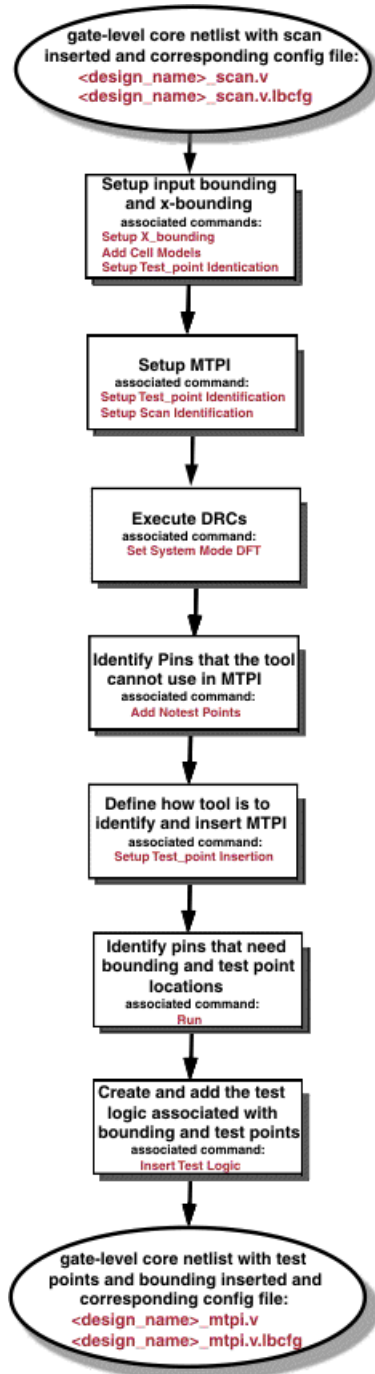
Figure 2-3. Inputs and Outputs of 2nd Invocation of BIST-Ready Phase



Flow of Second Run of BIST-Ready Phase

The following flow chart shows the main tasks that are associated with the second invocation of the BIST-Ready phase (which involves the setting up, identifying, and inserting of logic to both control the x-generators and to create the test points (MTPI)):

Figure 2-4. Flow Chart for Bounding and MTPI



Performing Input-Bounding, X-Bounding, and MTPI

The following procedure assumes that you have already completed the steps in the previous “[Performing DRCs and Inserting Scan Circuitry](#)” subsection, that you have exited the tool, and you are now ready to control your x-generators and insert your test points.

1. Invoke the second run of the BIST-Ready phase on the design.

```
shell> <Tessent_Tree_Path>/bin/lbistarchitect -bist_ready mydesign_scan.v \
      -lib mylib.lib
```

```
// Initializing from LBISTArchitect configuration file
'mydesign_scan.v.lbcfg'.
```

2. Define the input-bounding and x-bounding parameters (the command defaults are explicitly shown here).

```
SETUP> setup x_bounding -connect_to_existing_scan_cell -scan_drive_limit 4
      -scan_cell_proximity 3 -lbist_enable lbist_en
```

For more information on bounding, refer to the subsection titled [Controlling Inputs and Correcting Internal X-Generators](#).

3. Define cells that the tool is to use for the control points and the associated control logic. Note that the following 2-input cells must either be explicitly added by you with the Add Cell Models command, or implicitly defined by having the `cell_type` attribute present in the ATPG library: AND, INV, OR, and XOR (for the XOR tree that merges the new observe points).

```
SETUP> add cell models my_and -type and
SETUP> add cell models my_inv -type inv
SETUP> add cell models my_or -type or
SETUP> add cell models my_xor -type xor
```

If the ATPG library models (for example in *mylib.lib* file) include the definitions for the `cell_type` attribute, then you are not required to use the Add Cell Models command.

4. Define the multi-phase test point insertion parameters.

```
SETUP> setup test_point identification -control 50 -observe 200
      -patterns 32000 -verbose -phases 4
```

For more information on test points, refer to the subsection titled [Inserting Test Points Using MTPI](#).

5. Inform the LBISTArchitect tool that you are going to be inserting test points and not scan cells (which are already in your netlist from the first invocation of the tool).

```
SETUP> setup scan identification none
```

6. Change system mode to DFT.

```
SETUP> set system mode dft
```

7. Issue the, [Add Notest Points](#), Report Notest Point, and Delete Notest Point commands in DFT mode with multi-phase test point insertion to control the generation of test points as you desire.

```
DFT> add notest point clock_generator1
```

8. Set up how you want the tool to identify and insert the observe test points.

DFT> **setup test_point insertion -observe test_cntl -existing_scan_cell -oshare 8**

This command only tells the tool how you want the test points configured. The tool identifies the test point areas during the Run command, and the tool actually modifies the netlist, and adds the circuitry during the [Insert Test Logic](#) command.

Note that by default the inputs from the control test points come from the BIST controller.

9. Perform the automatic test point identification analysis, where the tool identifies where test points are required.

DFT> **run**

For more information on MTPI, refer to the subsection titled “[Inserting Test Points Using MTPI](#)”.

10. Display the results of the MTPI identification run to the transcript window so that you can verify what the tool is doing before you modify the netlist with the test point circuitry in the next step.

DFT> **report test points**

11. Specify for the tool to add the identified test points.

DFT> **insert test logic -test_point on -scan off**

This command modifies the netlist and adds the test logic and its associated circuitry, which is required for the test points that the tool identified during the Run command.

12. Write out the netlist of the design.

DFT> **write netlist mydesign_mtpi.v -verilog**

```
// Note: Updated LBISTArchitect configuration file  
'mydesign_mtpi.v.lbcfg'.
```

BIST-Ready Phase Tool Concepts

Within the BIST-Ready flow, you need to perform these tasks during the *first invocation run*:

1. Identify the design’s clocks.
2. Create any pin constraints that you may need.
3. Run the design rules checker.
4. Insert the scan chains.

During the second invocation, you must perform these tasks:

1. Control the inputs and internal X-generators.
2. Setup, identify the locations, and insert the test points.

3. If needed, add primary outputs to observation point list.

Identifying System Clocks

The LBISTArchitect tool supports designs with multiple clocks. Although the tool pulses each clock domain at one specified frequency (this is the same methodology as for ATPG patterns applied from a tester), there is still the strong possibility of clock skew between each domain. Therefore, to ensure that the clock skew does not produce race conditions (that could result in inconsistent signatures through capturing unexpected data), the LBISTArchitect tool controls the clocks.

By default, the LBISTArchitect tool achieves skew-free capture by ensuring that the BIST controller only captures data into one clock domain for each pattern. All clocks shift scan data into the scan chains, but then, in the capture cycle, it is important that only one clock is pulsed. The only exception to this condition is when the LBISTArchitect tool determines that some clocks never interact with each other, and hence can capture simultaneously without the chance of clock skew producing a race condition.

The LBISTArchitect tool determines which clock domains are captured on each pattern, and ensures that the number of patterns on which each clock captures data is compatible with the number of scan cells driven by each clock (this ensures maximum fault coverage).

The LBISTArchitect tool also identifies and controls asynchronous set and reset signals. These signals are treated as capture clocks for a small number of patterns, ensuring that the logic dependent on these signals can also be fully tested. Set and reset signals are controlled by the BIST controller so that they are always inactive during the shift procedure. Next, for a few patterns only, they are pulsed during the launch-capture cycle. For all other patterns these signals are held inactive.

It is recommended that you use the [Analyze Control Signals -Auto](#) command first, then issue a [Report Clocks](#) command to determine which clocks that command did not automatically identify, so that you can manually specify them with the [Add Clocks](#) command.

Identifying Pin Constraints

The Add Pin Constraints command specifies the LBISTArchitect tool to hold the input pin at a constant state during all simulations, including the design rules checking process. There are times that you may want to hold a pin at a constant state during the rules checking process. For example, you may want to hold a test enable pin at the active state to keep the chip in test mode.

You can set a default pin constraint value for all input pins using the [Setup Pin Constraints](#) command. The pin constraints set by the Setup Pin Constraints command can be overridden by the values set with the [Add Pin Constraints](#) command.

The LBISTArchitect tool has two general methods for determining how to construct the constraints that you specify with the Add Pin Constraints command. These methods are described in the following two sections: “[Simulating Pin Constraints](#)” and “[Synthesizing Pin Constraints in the Hardware](#)”.

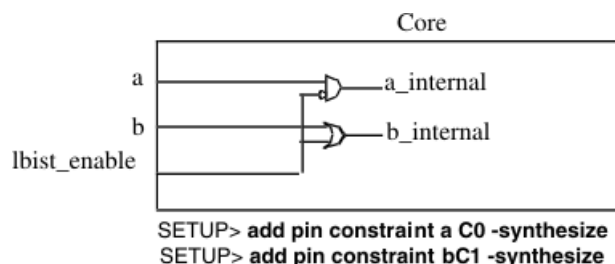
Simulating Pin Constraints

By default, the LBISTArchitect tool uses the simulation method, which specifies for the LBISTArchitect tool to constrain the pins through the test environment (such as the test bench and test vectors). These signals must be driven with the appropriate values any time a logic BIST test is active. If you use the -Simulate command switch, a tester is required to apply these pin constraints. So, the test may not be usable at the board or system level.

Synthesizing Pin Constraints in the Hardware

A second pin constraining method specifies for the LBISTArchitect tool to synthesize the pin constraints in the hardware. That is, the LBISTArchitect tool creates extra gates on the input paths that are controlled by the *lbist_enable* signal as shown in [Figure 2-5](#). The tool adds the necessary hardware when you issue the Insert Test Logic command. However, you should note that you cannot use the -Synthesize switch to constrain a pin to an unknown (CX) or high impedance (CZ) value.

Figure 2-5. Pin Constraint Hardware Added by LBISTArchitect:



Checking Your Design for Rule Violations

Design rule checking ensures that the design is testable from a BIST perspective. There are more than 150 design rules that can be applied using the LBISTArchitect tool. These rules are common to all of the Mentor Graphics' DFT tools; including Tessent FastScan™, Tessent TestKompress®, and DFTAdvisor™, although the emphasis is placed on different sets of rules depending on the target test methodology. The design rules checker executes when you switch from SETUP to DFT mode with the Set System Mode command.

The BIST-specific rules check to see whether the design has any bus contention issues and also whether the design can produce X states that could propagate to scan chains and thereby corrupt

the MISR signature. The following rules are specifically applied to check for potential X propagation and BIST-related checking:

G6 - Scan rules are part of the general rules (G rules) that occur first during rules checking. The purpose of general rules is to check for inconsistencies in the scan information that is defined.

B1 - B12 - BIST rules checks that are applied to ensure proper application of BIST patterns to the circuit when using the Linear Feedback Shift Registers (LFSRs).

B13 - Checks for X state propagation to the observe points. This rule identifies all possible X sources that could propagate to a MISR.

B14 - Checks that wire gates are not driving opposite values. This rule identifies a subset of those identified by the E5 rule. These gates can potentially drive opposite and contending values, which can result in an X state.

B15 - Checks for bus contention mutual exclusivity. This rule identifies a subset of those identified by the E5 rule. This rule checks for tri-state or bidirectional buses and identifies those buses that can produce a contention. All tri-state and bidirectional buses must have one, and only one, driver active in order to prevent possible contention. The E10 rule identifies those that do not obey this rule.

B16 - Checks for the ability for bus gates to drive Z states. This rule identifies a subset of those identified by the E5 rule. This rule checks for tri-state or bidirectional buses and identifies those buses that can produce a high-impedance state (Z state). All tri-state and bidirectional buses must have one driver active in order to prevent possible contention. The E11 rule identifies those that do not obey this rule. A Z state is equivalent to an X state in that it is unknown and can corrupt the MISR.

Setting Up LBISTArchitect to Insert Scan Chains

The LBISTArchitect tool supports a large number of short scan chains. With a BIST controller you are not constrained by the number of available chip-level pins or ATE scan channels. Therefore a configuration of 64, 128 or 256 scan chains is allowed and encouraged. The benefit of having many short scan chains (rather than a few long ones), is that this configuration minimizes the test application time. The test time equals the length of the longest chain times the number of patterns. If the design comes to you “ATPG-Ready” (with only a few long chains), you should rip up these chains and replace them with many short chains.

The [Setup Scan Identification](#) command defines whether the tool is configured to insert scan chains (“full_scan” command option), or analyze the circuit for test point insertion (“none” command option). To achieve many short scan chains, use either the -Number switch to set the number of scan chains the tool is to insert, or use the -Max_length switch to determine the length of the longest scan chain.

While the Setup Scan Identification command determines which task the tool is to perform, the tool doesn't actually insert any of the circuitry into the netlist until you issue the [Insert Test Logic](#) command.

Controlling Inputs and Correcting Internal X-Generators

Thinking ahead in the BIST development flow, when you are finally able to run the BIST test on your manufactured chips, you must have a way of preventing the unknown states from being captured into a scan chain and possibly corrupting the MISR. So, at this point in the BIST development of your design, you need to identify and control all the signals that could possibly result in an unknown state reaching the scan chains. To control a signal, the tool adds the required logic (multiplexers or other gating circuitry) to ensure that an unknown value cannot propagate through that gate.

i **Tip:** For multicycle path considerations, see “[Applying X-Bounding and Capture Windows for Multicycle Paths](#)”.

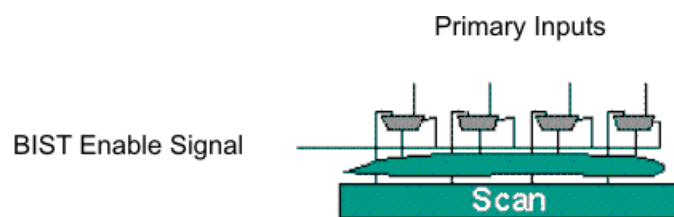
Unknown states can come from many areas within your design, but they fall into one of two main categories:

- **Unknown Primary Input Values** - There is an outside pin constraint that is at an unknown (X) value, which the tool then applies directly to a primary input. See [Figure 2-6](#).

Many of these unknown states come directly from the primary input pins, so you must isolate the primary input pins to the BISTed core from the rest of a design. By default, the tool controls all primary input pins (referred to as *input bounding*), by placing CX pin constraints on the inputs so that when the tool performs signal bounding, it automatically treats primary inputs as X sources and controls them as well.

All input pins that you do not explicitly constrain to a C0 or C1, the tool treats as possible X sources and controls them using the currently defined bounding method (unless you override that action using the [Setup X_bounding -Dont_bound](#) option).

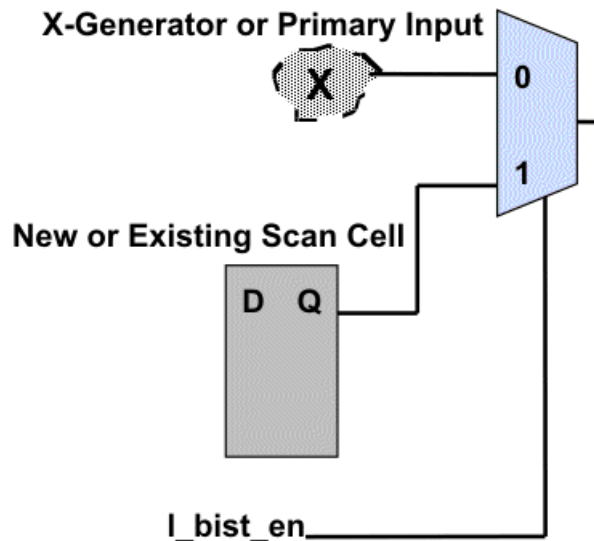
Figure 2-6. Input Bounding with Muxes



The muxes ensure that the primary inputs to the core are stimulated with pseudorandom patterns, as each input is controlled to both '0' and '1', providing maximum fault coverage. The LBISTArchitect tool multiplexes each input with the output of a scan cell, controlled by the bist enable signal. This method requires a multiplexer on each input and an associated delay.

- **Internal X-Generators** - The design has internal circuitry that could produce unknown values (X-generator) that can then propagate throughout the rest of the design into the MISR.

Figure 2-7. Controlling X Sources with Muxes



There are three methods that the tool can use when bounding the X sources, which you can control using the Setup X_bounding command. By default, the tool controls X sources by connecting it to a nearby scan cell (which supplies pseudorandom patterns) in test mode. The second method is for the tool to control the X source by connecting it to a new scan cell, instead of using an existing nearby scan cell. You can also instruct the tool to tie the X source to power or ground. This last method will result in lower fault coverage and is therefore not recommended.

If you do not issue the [Setup X_bounding](#) command prior to issuing the [Setup Test_point Identification](#) command, the tool automatically turns on input bounding and X -bounding and uses the command defaults.

Inserting Test Points Using MTPI

The LBISTArchitect tool uses a patented method to identify and insert test points into the design to overcome random pattern resistance. This method is known as Multiphase Test Point Insertion, or MTPI. This method typically results in fewer test points, but more importantly, less routing and timing impact on the core design.

Note



You cannot use the [Add Test Points](#) command in the BIST-Ready phase in conjunction with multi-phase test point insertion. However, you can use the Add Test Point and the Insert Test Logic commands in one run, go back into Setup mode, and then start the multi-phase test point insertion analysis.

MTPI test point insertion relies on the following sequence of steps:

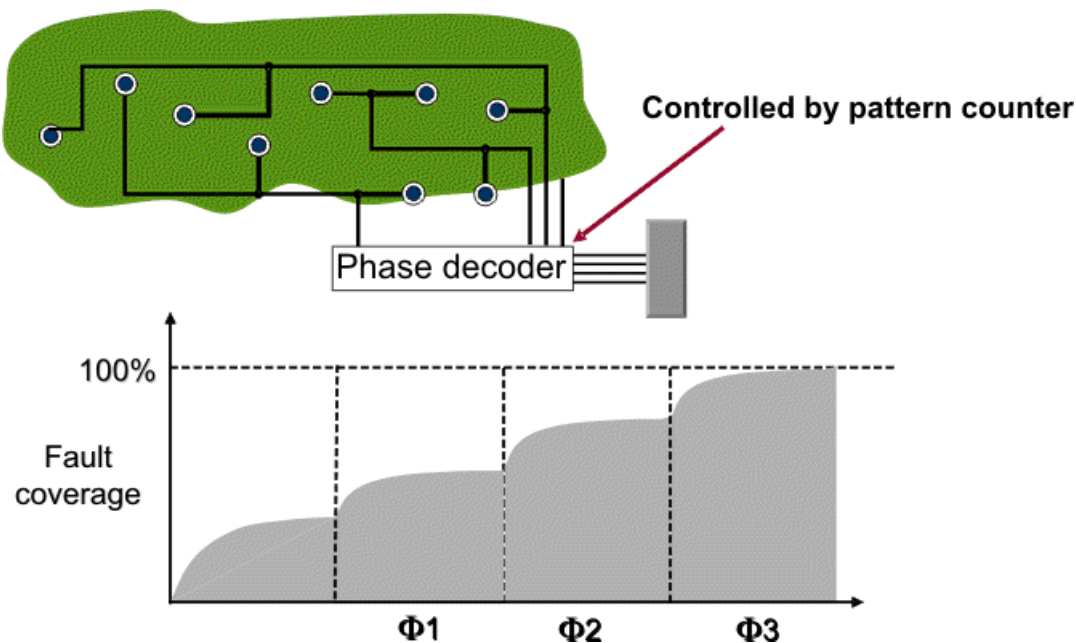
1. Use the [Setup Scan Identification](#) command to enable LBISTArchitect to identify the optimum test point locations.
2. Use the [Setup Test_point Identification](#) command to set the number of each type of test point LBISTArchitect is to locate.
3. Use the [Set System Mode](#) command to enter DFT mode.
4. Use the [Run](#) command to perform the test point analysis. While performing the analysis, LBISTArchitect lists the test point locations that it identifies.

As shown in [Figure 2-8](#), MTPI divides the pseudorandom pattern sequence into a number of phases. For example, a sequence of 32K patterns could be divided into four phases of 8K patterns each. In each phase, LBISTArchitect asserts different control points and drives them to constant known values. Unlike other approaches, MTPI uses fault simulation to identify the control points and, in each phase, only the remaining undetected faults from the previous phase are analyzed for test point insertion. LBISTArchitect identifies the test point locations based on the stuck-at fault model although this process tends to improve the transition fault coverage as well.

LBISTArchitect performs the MTPI analysis in phases. In phase 0, the tool adds all of the observe points. The tool then re-fault simulates the design to determine the undetected faults to forward onto phase 1.

For the remaining phases, the required number of control points is divided equally between each phase. The tool asserts the control points for each phase and runs a fault simulation to determine the undetected faults to be passed on to the next phase. At the end of the analysis, the tool performs a final fault simulation to determine the estimated fault coverage for the core.

Figure 2-8. MTPI Overview

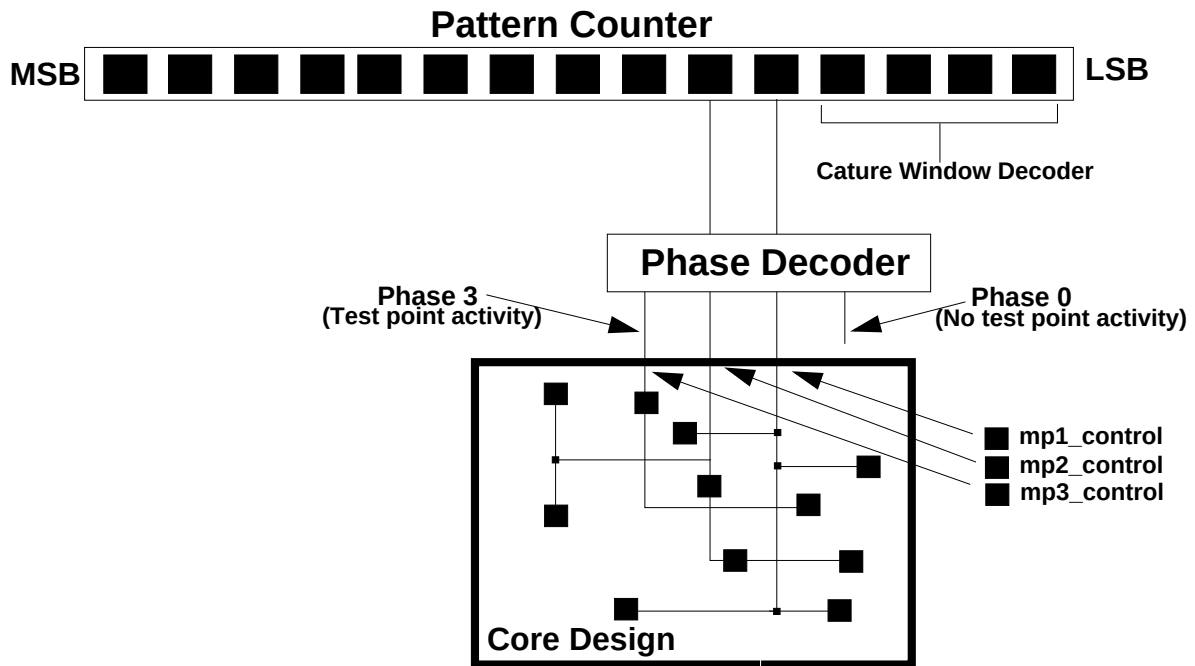


LBISTArchitect generates the MTPI phase decoder. The phase decoder translates the value of the pattern counter and asserts a sequence of control signals, as shown in [Figure 2-9](#). There is one control signal per phase, except for the first phase (phase 0) where there is none. LBISTArchitect adds the MTPI control inputs to the core inputs and you can use these to increase the top-up ATPG coverage.

The test point selection algorithm computes the control points for each phase serially. For example, if you are targeting 8192 LBIST patterns, LBISTArchitect simulates 2048 patterns for phase 0 (no control points), selects the control points for phase 1, simulates 2048 patterns, selects control points for phase 2, simulates 2048 patterns, and continues until all four phases are processed.

However, the actual LBIST patterns will consist of a much smaller number of phase 0 patterns followed by some phase 1 patterns, continuing through phase 3, and then starting over with phase 0. This approach helps to smooth out the stair-step coverage increase shown in [Figure 2-8](#) and provides better fault coverage results when you are simulating a subset of the full pattern space. For example, in [Figure 2-9](#), the phase decoder is connected to bits 4 and 5 of the pattern counter. This means LBISTArchitect will simulate 16 patterns in phase 0, 16 patterns in phase 1, 16 patterns in phase 2, 16 patterns in phase 3, and then start over.

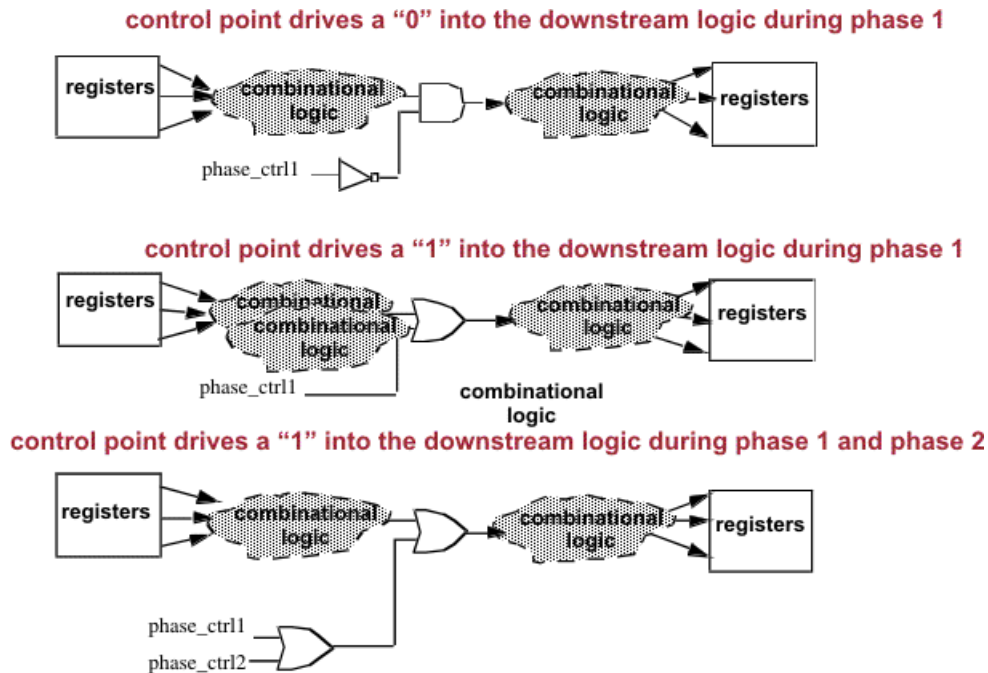
Figure 2-9. Test Point Control



MTPI inserts observe and control test points to achieve an acceptable fault coverage target. The BIST-Ready phase includes special algorithms that identify control points using a unique (and patented) approach that inserts them into the design for each phase, except phase 0. Control points are targeted to be “active” during a given subset of phases. The Mentor Graphics approach to test point insertion is unique because control points are not driven by pseudorandom data. All control points inserted into a phase are simultaneously activated using one enable signal.

As shown in [Figure 2-10](#), control points that are intended to drive a low into downstream logic require the tool to add a single AND gate into the design. Control points driving a high into the downstream logic consist of a single OR gate. If it is active during only a single phase the control gate can be driven directly by a global enable signal, (one per phase). Signals that are active during multiple phases require additional gates in order to merge the phase enable signals together. Because these extra gates are not in the functional path, they will not add any further to the functional timing caused by the addition of a test point.

Figure 2-10. MTPI Control Point Logic



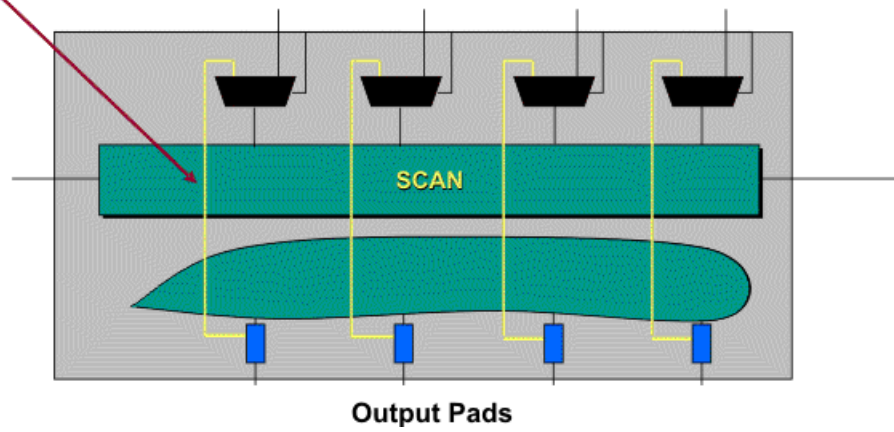
For more information on MTPI, see section “[Multi-Phase Test Point Insertion \(MTPI\)](#)” on page 126”.

Observation Points for Primary Outputs

The tool supports output bounding which is the automatic addition of observe points to primary outputs. Using output bounding provides the benefit of higher fault coverage by observing logic paths between the last scan cell and a primary output. By default, the tool may not add observe point to all primary output pins. So, to increase your fault coverage, you can use the `Setup Test_point Identification -Primary_outputs` command to add specific output pins. Note that how the tool interprets the `-Primary_outputs` switch depends on the switches you may have used with the [Setup Test_point Insertion](#) command.

Figure 2-11. Output Bounding

Output Pin Observe points entering scan chain



Chapter 3

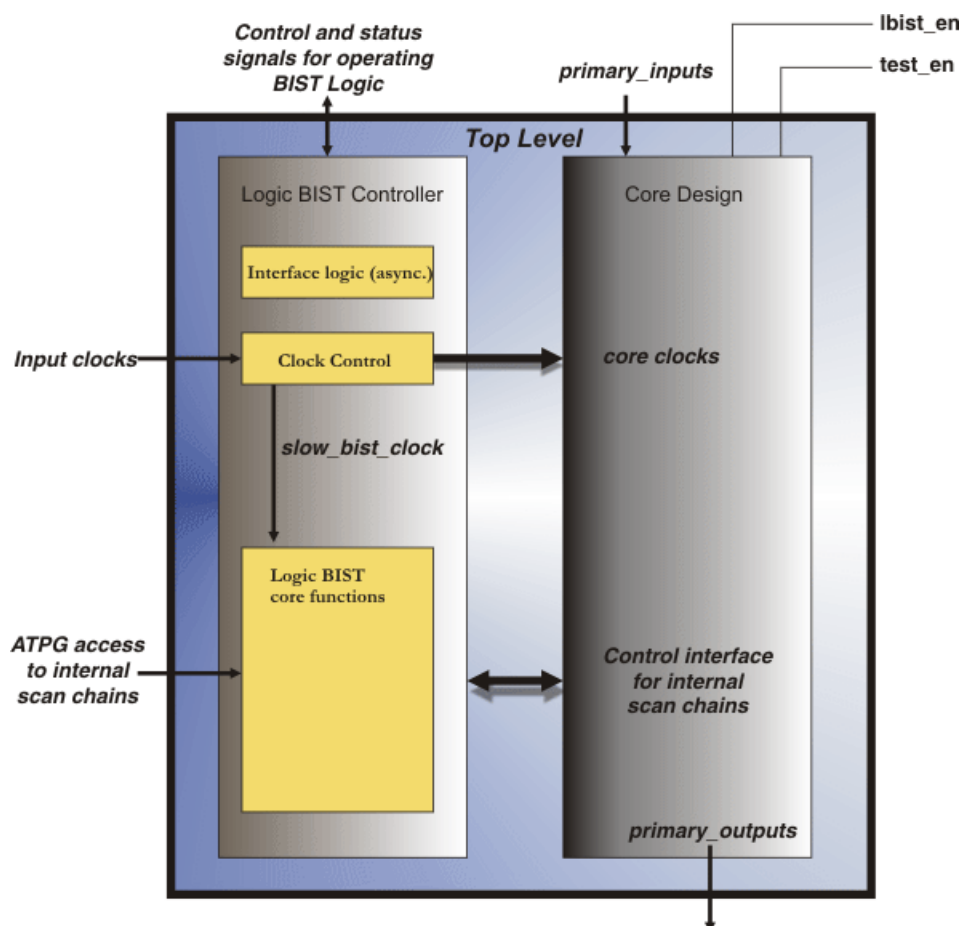
Generating the BIST Controller

Understanding the BIST Controller

Once you have created a BIST-ready core design netlist and associated configuration files, you proceed to the next LBISTArchitect tool phase: generation of the BIST controller.

Figure 3-1 illustrates the interaction between the LBISTArchitect BIST controller and your core design. At the conclusion of this tool phase, LBISTArchitect creates an HDL Verilog model of the BIST circuitry and an interface connecting this BIST circuitry to your core design.

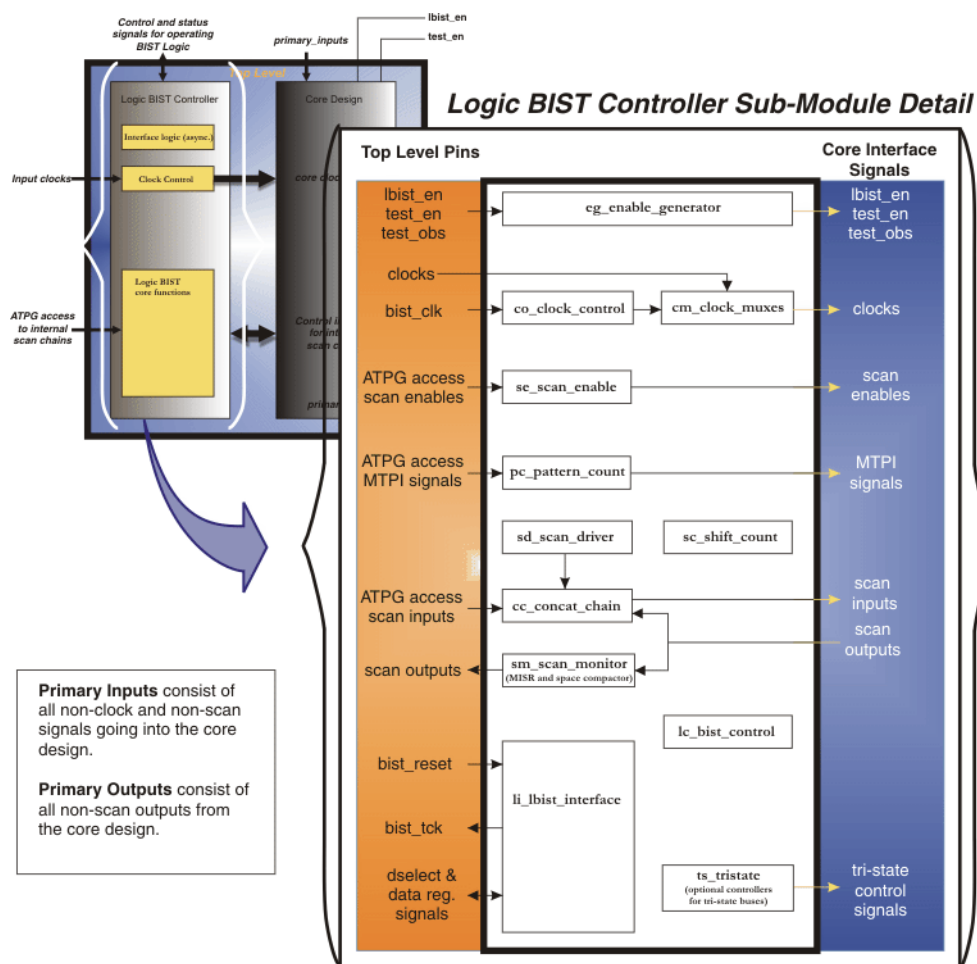
Figure 3-1. BIST Controller and Core Design Relationship



Examining the BIST Controller Module

Figure 3-2 illustrates the module-level detail of the BIST controller LBISTArchitect produces during BIST controller generation. This model is an RTL description of the BIST circuitry and its connections to your core design. Additionally, LBISTArchitect produces additional output files; refer to “BIST-Controller Phase Outputs” for a complete list.

Figure 3-2. Logic BIST Controller Sub-Modules



Sub-Module Summary

Table 3-1 summarizes each of these LBISTArchitect logic BIST controller modules in the model.

Table 3-1. Logic BIST Controller Module Summary

Module	Corresponding BIST Controller Circuitry
eg_enable_generator	test mode control signals for the core

Table 3-1. Logic BIST Controller Module Summary

Module	Corresponding BIST Controller Circuitry
co_clock_control	control signals for the BIST clock generators
cm_clock_muxes	BIST clock generator and muxing logic
se_scan_enable	scan enable signal
pc_pattern_count	Pattern Counter
sd_scan_driver	PRPG and Phase Shifter
sc_shift_count	Shift Counter
cc_concat_chain	see “ Lockup Latch Timing During Shifting ”
sm_scan_monitor	MISR and Space Compactor
lc_lbist_control	state machine for controlling BIST test
li_lbist_interface	asynchronous control interface

i **Tip:** The “[Logic BIST Controller Data Sheet](#)” section of this document contains detailed information regarding the BIST controller.

PRPG and Phase Shifter

The *sd_scan_driver* module contains both the PRPG (Pseudorandom Pattern Generator) and the phase shifter.

- **PRPG** — A [Linear Feedback Shift Register](#) and the source of pseudorandom patterns. In contrast to a typical LFSR, you can load initial values into the PRPG.
- **Phase Shifter** — Consists of XOR gates and removes phase differences between the pseudorandom patterns.

MISR and Space Compactor

The *sm_scan_monitor* module contains both the MISR (Multiple Shift Input Register) and the space compactor.

- **MISR** — A [Linear Feedback Shift Register](#) and captures the outputs of the scan chains. The last output from the MISR determines a subsequent pass or fail result. In contrast to a typical LFSR, you can load initial values into a MISR, and the MISR retains these values during the first pattern.
- **Space Compactor** — An XOR network responsible for merging multiple scan outputs into a single bit when the number of scan outputs exceeds the number of bits in the MISR.

Pattern Counter

The *pc_pattern_count* module contains the pattern counter. The pattern counter counts the total number of applied patterns and, consequently, pulses the capture (or set/reset) clocks. Using the pattern counter, you can specify start and stop values. The pattern number determines and applies state of the phase controlled signals and the clock sequence.

Shift Counter

The *sc_shift_count* module contains the shift counter. The shift counter monitors the number of cycles for shifting a single pattern into your core design and keeps the *scan_enable* signal high during the shift phase.

BIST-Controller Phase Inputs

Table 3-2 lists the input files you must supply to LBISTArchitect during the BIST controller generation phase. Normally, you create these files during the BIST-Ready tool phase, and LBISTArchitect automatically creates the configuration file (*.lbcfg*); see “[Making the Design Ready for Logic BIST](#)” for complete information.

Table 3-2. BIST-Controller Phase Inputs

File Name	Description
<i>design_name.v</i>	BIST-Ready gate-level core netlist
<i>design_name.v.lbcfg</i>	configuration file from the BIST-Ready phase—contains information including the scan groups and chains, test points, and observation points

BIST-Controller Phase Outputs

Table 3-3 lists the files LBISTArchitect outputs after a BIST controller generation run.

Table 3-3. BIST-Controller Phase Outputs

File Name ¹	Description	Input for
<i>design_name_bist.v</i>	RTL netlist for LBIST controller	Synthesize RTL of BIST Controller to gate-level LBISTArchitect TBGen Utility to generate test

Table 3-3. BIST-Controller Phase Outputs

File Name ¹	Description	Input for
<i>design_name_bist.v.lbcfg</i>	ATPG setup file that is generated with, and refers to, the LBIST RTL controller netlist	LBISTArchitect TBGen Utility to generate test
<i>design_name_bsda_in.v</i>	RTL netlist for top level as required by Boundary Scan insertion	Boundary scan using BSDArchitect—see the BSDArchitect Process Guide
<i>design_name_bsda.do</i>	BSDArchitect dofile	
<i>design_name_lbist_bscan.test_seq</i>	BSDArchitect test sequence file	
<i>design_name_lbist_bscan_tb.v</i>	BSDArchitect test bench file	
<i>design_name_lbist_bscan.wgl</i>	BSDArchitect test vector file	

Table 3-3. BIST-Controller Phase Outputs

File Name ¹	Description	Input for
<i>design_name_faultsim.do</i>	fault simulation phase dofile	LBISTArchitect Fault Simulation Phase
<i>design_name_core_faultsim.do</i>	core level fault simulation phase dofile	
<i>design_name_bist.testproc</i>	stand-alone mode test procedure file	
<i>design_name_bist.ctestproc</i>	core-level test procedure file	
<i>design_name_trans_core_faultsim.do</i>	core-level fault simulation phase dofile for transition fault grading	
<i>design_name_trans_faultsim.do</i>	fault simulation phase dofile for transition fault grading	
<i>design_name_trans_bist.ctestproc</i>	core-level test procedure file for transition faults	
<i>design_name_trans_bist.testproc</i>	stand-alone mode test procedure file for transition fault grading	
<i>design_name_bist.pat.gz</i>	controlled by <i>design_name_core_faultsim.do</i> ; writes patterns at the conclusion of fault simulation	
<i>design_name_bist.lfsr.trace.gz</i>	controlled by <i>design_name_core_faultsim.do</i> ; writes LSFR values at the conclusion of fault simulation	
<i>design_name_topup_atpg.do</i>	Tessent FastScan topup ATPG dofile	Topup ATPG—see “ Generating Test Patterns for Different Fault Models and Fault Grading ” in the <i>Scan and ATPG User’s Manual</i>
<i>design_name_topup_atpg_faults</i>	topup ATPG fault simulation driver	
<i>design_name_topup.testproc</i>	the top-level scan chain test procedure file	

Table 3-3. BIST-Controller Phase Outputs

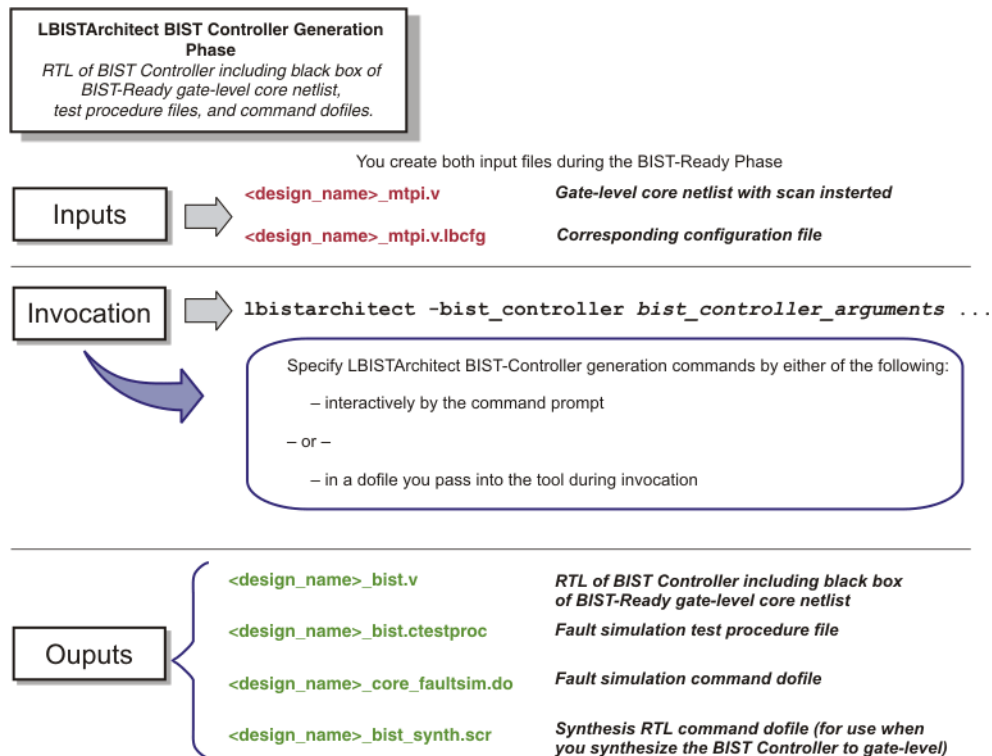
File Name ¹	Description	Input for
<i>design_name_bist_synth.scr</i>	sythesis script to synthesis the BIST controller RTL and merge it with the core design	Synthesize RTL of BIST Controller to gate-level
<i>design_name_tb.v</i>	test bench for simulating the LBIST controller. This will be used for both gate and RTL simulations, so it's left in the local directory	LBISTArchitect TBGen Utility to generate test
<i>design_name.flat</i>	flat model	
<i>design_name_bist.wgl</i>	template for test vector signature	
<i>design_name_bist.stil</i>	creates STIL file	

1. These are the default names LBISTArchitect produces. You can use different naming conventions — see “[Output File Naming](#)”.

Examining BIST Controller Generation

[Figure 3-3](#) provides, in general terms, the procedure you use for generating the BIST controller with LBISTArchitect.

Figure 3-3. BIST Controller Generation Phase



Invoking and Using LBISTArchitect

You invoke LBISTArchitect for BIST controller generation using the following command in a shell:

```
<Tessent_Tree_Path>/bin/lbistarchitect -bist_controller \  
bist_controller_arguments
```

You use the `bist_controller_arguments` for specifying the invocation options including the following:

- the gate-level BIST-Ready netlist name, location, and format
- the log file name for the run
- the dofile name and location if you use one

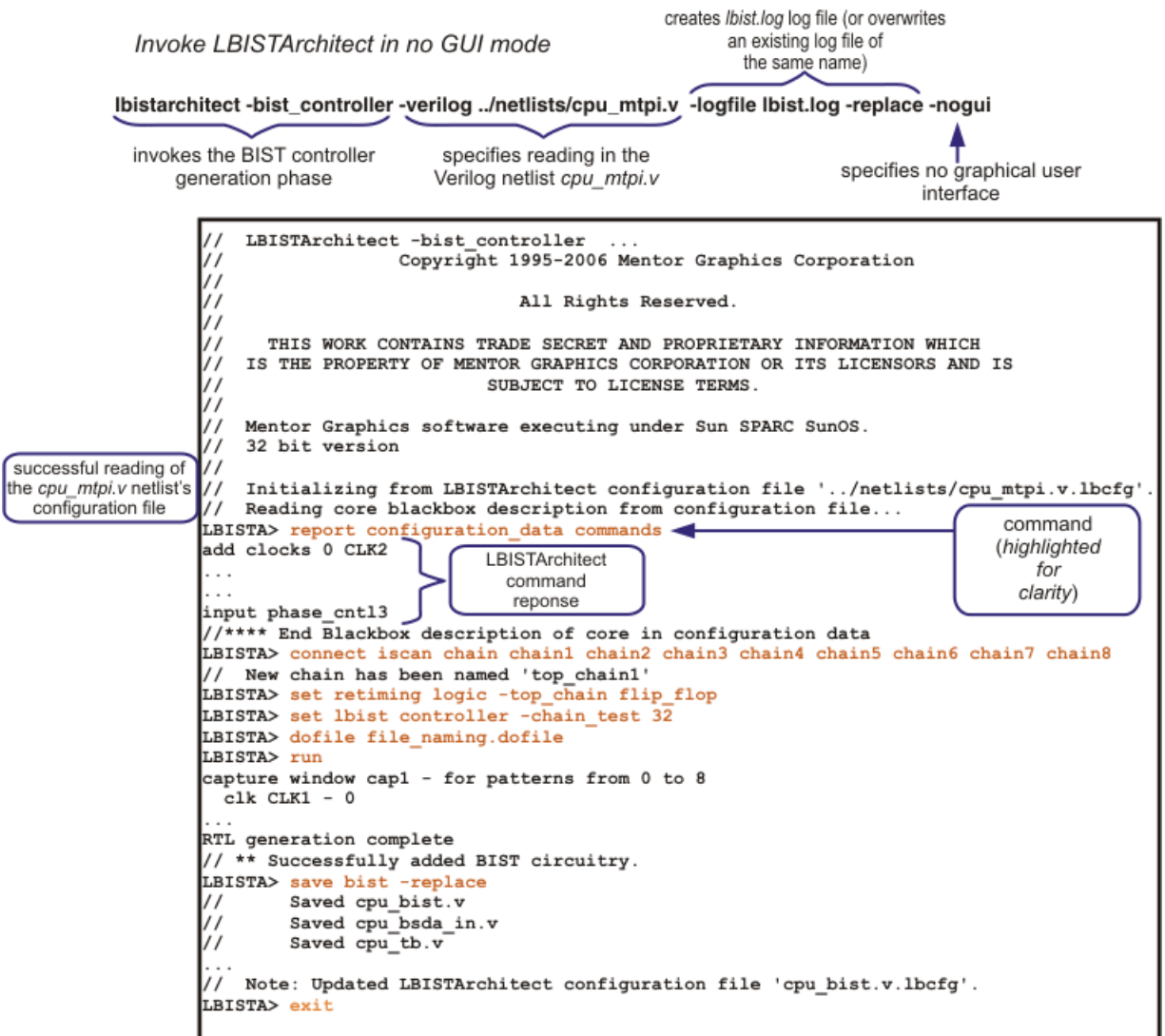
Once you have invoked LBISTArchitect, you direct the BIST Controller generation through the use of commands. When using LBISTArchitect, you can either interactively issue these commands in the LBISTArchitect session, or create a dofile containing the commands and read the dofile into LBISTArchitect during invocation.

Examining an Example BIST Controller Generation Run

Interactive Example

[Example 3-1](#) illustrates performing BIST controller generation interactively with LBISTArchitect.

Example 3-1. Interactive LBISTArchitect BIST Controller Run Example



Dofile Example

You can achieve the identical goals in the previous example by using a dofile, which you specify to LBISTArchitect during invocation. [Example 3-2](#) illustrates using a dofile with the same information.

Example 3-2. Dofile LBISTArchitect BIST Controller Run Example

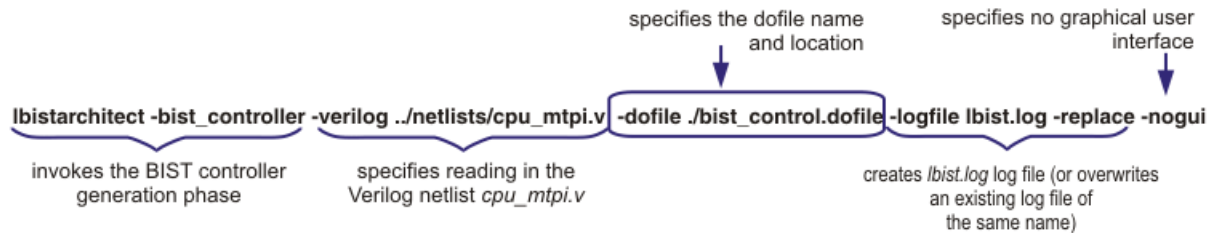
- 1 Create the dofile with an ASCII text editor

Dofile name: *bist_control.dofile*:

Dofile contents:

```
report configuration_data commands
connect iscan chain_chain1 chain2 chain3 chain4 chain5 chain6 chain7 chain8
set retiming logic -top_chain flip_flop
set lbist controller -chain_test 32
dofile file_naming.dofile
run
save bist -replace
exit
```

- 2 Invoke LBISTArchitect in no GUI mode specifying the dofile



- 3 Examine the log file (*bist.log* in this example)

Loading the Input Gate-Level Netlist

LBISTArchitect requires a gate-level BIST-Ready netlist and corresponding configuration file (*.lbcfg*) for input when you invoke the tool; LBISTArchitect automatically reads this configuration file. Additionally, the configuration file must reside alongside the gate-level BIST-Ready netlist.

Note Upon BIST controller generation invocation, LBISTArchitect automatically generates the BIST controller *using* the parameters you had entered during the LBISTArchitect BIST-Ready phase.

Once you invoke LBISTArchitect in the BIST controller generation mode, you can determine what commands and parameters are in the configuration file by issuing the following command: [Report Configuration_data Commands](#).

Output File Naming

The “[BIST-Controller Phase Outputs](#)” section identifies the output files and their *default* file names LBISTArchitect creates. You can change both the file name and where LBISTArchitect writes these files by using the “[Setup File Naming](#)” command. This command provides the flexibility of specifying both a different name and directory location. You can also keep this command in a separate dofile and reference the file in your primary dofile by using the [Dofile](#) command.

Using Log Files

When running LBISTArchitect in any phase, it is good practice if you capture a log of the run. This is especially imperative for debugging purposes. At invocation time, you can automatically write a transcript of the run by specifying the `-logfile <file_name>` invocation argument.

Performing BIST Controller Generation

When you perform BIST controller generation, LBISTArchitect creates the BIST controller using the parameters (for example: scan chains, pin constraints, test points) you used *when you created* the BIST-Ready core netlist with the inserted scan.

[Example 3-3](#) provides the general steps you perform when performing BIST controller generation using LBISTArchitect—however, each BIST controller run is unique and dependent of design style and outcomes.

Example 3-3. BIST Controller Generation Run Example

- 1 Invoke LBISTArchitect in BIST controller generation mode.

```
lbistarchitect -bist_controller -verilog cpu.v -logfile bist_controller_run.log -nogui
```

Report configuration file contents:

```
LBISTA> report configuration_data commands
```

- 2 Perform setup operations (if any).

```
LBISTA> connect iscan chain chain1 chain2 chain3 chain4 chain5 chain6 chain7 chain8
LBISTA> connect iscan chain chain9 chain10 chain11 chain12 chain13 chain14 chain15 chain16
LBISTA> connect iscan chain chain17 chain18 chain19 chain20 chain21 chain22 chain23 chain24
LBISTA> connect iscan chain chain25 chain26 chain27 chain28 chain29 chain30 chain31 chain32
LBISTA> set retiming logic -top_chain flip_flop //added
LBISTA> set lbist controller -flush_test 32
```

- 3 Perform the BIST controller generation.

```
LBISTA> run
```

*You must complete any
setup commands before
issuing the Run command.*

- 4 Write the output files.

```
LBISTA> save bist -replace
```

What Commands Should You Use?

Before issuing the LBISTArchitect “Run” command, you can specify, add, or modify certain parameters by using commands and their associated arguments. The following sections contain typical LBISTArchitect BIST controller generation activities and their associated commands:

- [Setting Up the BIST Controller](#)
- [Connecting Internal Scan Chains](#)
- [Capturing Window Waveforms for At-speed Testing](#)
- [Running the BIST Controller Generation](#)
- [Saving and Writing the Output Files](#)

Note



You should review each LBISTArchitect command in detail before implementing the command in your BIST generation flow. The “[BIST Controller Command Dictionary](#)” of the *LBISTArchitect Reference Manual* provides complete descriptions, arguments, and examples of the available commands for the BIST controller generation phase.

Setting Up the BIST Controller

The primary method for setting up the BIST controller is the [Set Lbist Controller](#) command. This command provides a method for tailoring the following BIST parameters to your specifications:

- methods for driving scan enable signals during testing including flush test patterns
- setup for multiplexing MTPI phase decoder signals to the top level for use by ATPG tools
- the active high or low signal state for the BIST reset signal

[Table 3-4](#) lists the related commands you use for setting up the BIST controller, specifically how the resultant test hardware will run.

Table 3-4. BIST Controller Generation Phase Controller-Related Commands

Command	Description
Set Testpoint Parameters	Defines parameters for controlling test points using the multi-phase test point process—use if you change port names or phases.
Set Retiming Logic	Provides a method for specifying the use of lockup latches or flip-flops for retiming logic within various sections of the BIST controller.

Setting Up an Asynchronous BIST Test

When setting up the BIST controller with the [Set Lbist Controller](#) command, you can optionally specify that the BIST controller includes an asynchronous test trigger signal to activate the test, and provide direct access to the MISR value using an output vector (misr_value). Configuring the BIST controller for this operation results in a simplified method for determining the pass/fail status of test.

Note



This interface provides no support for diagnostics, or any other advanced features requiring changes to the internal configuration registers from the defaults loaded when the bist_reset signal is toggled.

You setup the BIST controller for this operation by choosing from one of the following mutually exclusive Set Lbist Controller arguments:

- **-Parallel_interface** — Creates an interface for triggering a test and direct access to the MISR output.
- **-ON_chip_comparator** — Creates an on-chip comparator to compare the reference signature to the actual MISR signature bits.

Using either of these arguments reduces the LBISTArchitect BIST controller test sequence to the following sequential operations:

1. Reset the BIST controller by toggling the `bist_reset` input.
2. Activate the test by driving the `bist_run_input` signal high.
3. Wait for the `bist_done` signal to go high.
4. Check the MISR value using the on-chip comparator, or externally through the direct access bus.

When performing BIST controller generation with this option, the test bench files LBISTArchitect modifies and creates contain the trigger for the BIST test using the `bist_run_input` signal. Once the test completes, the `bist_done` and `mistr_value` signals display. The test bench also attempts to unload, and compares the final MISR value using the serial interface.

The input and output signals that provide the default (serial) interface to the BIST controller will not be removed from the interface. This operation creates additional BIST controller input and output signals; see “[Asynchronous Logic BIST Test Signals](#)” for complete descriptions.

Setting Up Scan Insertion for the BIST Controller

One common challenge with logic BIST testing is verification of the hardware inside the BIST controller. Conceptually, many faults in the BIST controller are detected by implication when the BIST test runs; that is, many failures in the BIST controller result in a signature mismatch and are thereby detected. However, it is difficult to determine the fault coverage of the hardware in the BIST controller using this methodology.

A different approach is to add scan chains inside the BIST controller and use a separate ATPG run to perform structural testing of the BIST IP. Adding scan chains to the BIST controller can be challenging because of the following characteristics of the BIST controller:

- Logic operates on an internally generated clock.
- Controller includes clock gating logic that is in the path of the clocks feeding the core design.

- Controller includes multiplexing logic that is in the scan path for scan chains in the core design;
- Controller includes logic that may drive the scan enable for the core design;
- Controller includes some scan cells that are driven by an internally generated reset.

When setting up for BIST controller generation using the [Set Lbist Controller](#) command, you can work around these issues by (optionally) specifying that an additional top-level `bist_ip_scan_mode` signal is added to the BIST controller. When the `bist_ip_scan_mode` signal is active, LBISTArchitect does the following:

- Bypasses internal logic that generates the `slow_bist_clk` signal; the `fast_bist_clk` is connected directly to the `slow_bist_clk`.
- Modifies scan chain concatenation logic in the `cc_concat_chain` block so that the internally concatenated scan chains (not single chain mode) are connected to the top-level pins.
- Modifies logic that drives the scan enable signal (`se_scan_enable`) so that the top-level scan enable pin drives the core scan enables.
- Modifies the logic that takes control of the core clocks in `cm_clock_muxes` to bypass them (top-level clock pins are driving the clock signals into the core).
- Modifies flip-flops in the BIST controller driven by an internally generated Reset signal to force the Reset inactive.

For more information on this option, see the `-Control_logic_for_scan_mode` switch for the [Set Lbist Controller](#) command in the *LBISTArchitect Reference Manual*.

Connecting Internal Scan Chains

[Table 3-5](#) lists the commands you use for connecting multiple internal scan chains at the BIST controller level to form a single scan chain at the top level for interfacing to ATPG topup patterns.

Table 3-5. BIST Controller Generation Phase Internal Scan Chain Commands

Command	Description
Connect Iscan Chains	Connects internal scan chains to form single internal scan chains.
Disconnect Iscan Chains	Disconnects top-level scan chains that you created with the Connect Iscan Chains command.

The primary reason you should use the Connect Iscan Chains command is for creating fewer scan chains. By default, LBISTArchitect connects all internal scan chains to top-level ports.

You can reduce the number of these I/O pins by using the Connect Iscan Chains command and stitch together the multiple internal chains.

Capturing Window Waveforms for At-speed Testing

To perform at-speed testing, you must complete the following steps:

1. Add a capture window using the [Add Capture Window](#) command.
2. Define waveforms using the [Set Capture Waveform](#) command.
3. Define the shift rate as a fraction of the bist_clk signal using the [Set Shift_rate Divisor](#) command.

[Table 3-6](#) lists the commands you use for waveform capture when testing designs at-speed.

Table 3-6. BIST Controller Phase Capture Window-Related Commands

Command	Description
Add Capture Window	Creates custom capture window waveforms.
Report Capture Window	Displays a list of the custom capture window waveforms that you currently have defined.
Delete Capture Window	Removes custom capture window waveforms that you created with the Add Capture Window command.
Set Capture Waveform	Defines the waveform that the tool applies to the core when the named capture window is active.
Set Shift_rate Divisor	Defines the rate to drive the scan chains in relation to the bist_clk signal.

For complete information about at-speed testing, see section “[At-speed Testing of Designs with High Speed Clocks](#)”.

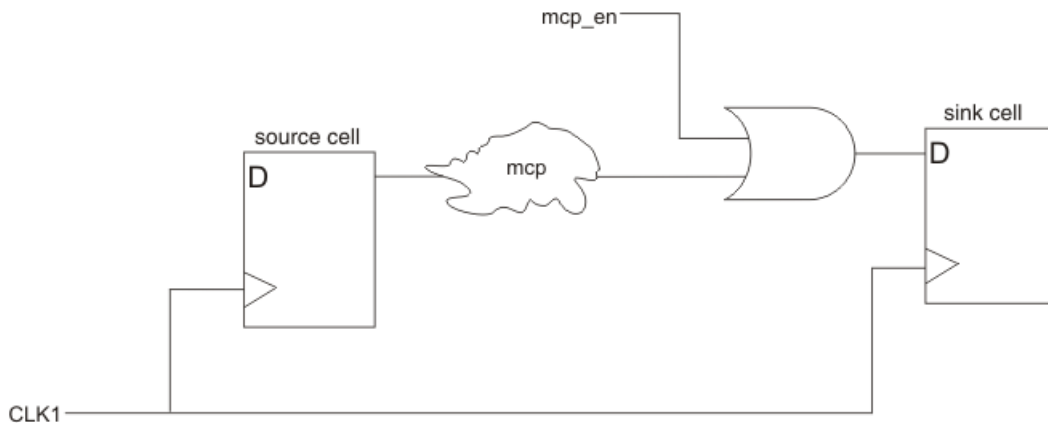
Applying X-Bounding and Capture Windows for Multicycle Paths

Implementing a BIST controller for a design containing multicycle paths is a 2-phase process you do during the following LBISTArchitect phases:

- **BIST-Ready Phase** — Identify and control x-generators by adding a mux at the destination point using the [Add Multi_cycle Path](#) command—see “[Controlling Inputs and Correcting Internal X-Generators](#)”.
- **BIST Controller Generation Phase** — Create *special* capture windows (Add Capture Window and Set Capture Waveform) and specify the correct control of the OR gate with the [Set Dynamic X_bounding](#) command.

Figure 3-4 provides an example multicycle path. The `mcp_en` pin is the select pin for blocking the path during at-speed testing. You specify this pin during the *BIST-Ready* phase using the Add Multi_cycle Path command.

Figure 3-4. Multicycle Path Example Circuit



The following LBISTArchitect command sequence example demonstrates the BIST-Controller Generation phase activities: creating the special capture windows and setting the OR gate's controls:

```
// High speed patterns
add capture window cap1 0 1
set capture waveform cap1 CLK1 pulse "0110"
set capture waveform cap1 CLK2 pulse "0000"
set capture waveform cap1 CLK3 pulse "0000"

// High speed patterns
add capture window cap2 2 11
set capture waveform cap2 CLK1 pulse "0000"
set capture waveform cap2 CLK2 pulse "0110"
set capture waveform cap2 CLK3 pulse "0000"

// High speed patterns
add capture window cap3 12 14
set capture waveform cap3 CLK1 pulse "0000"
set capture waveform cap3 CLK2 pulse "0000"
set capture waveform cap3 CLK3 pulse "0110"

// Multi-cycle pattern
add capture window cap4 15 15
set capture waveform cap4 CLK1 pulse "1001"
set capture waveform cap4 CLK2 pulse "0000"
set capture waveform cap4 CLK3 pulse "0000"

// Bounding high speed patterns
set dynamic x_bounding mcp_en cap1 cap2 cap3
```


In this example, patterns 0 through 1 with CLK1 are for at-speed fault detection of faults except on the mcp; pattern 15 detects faults on the mcp.

Running the BIST Controller Generation

When you have completed defining any BIST setup commands, you can generate the BIST controller with LBISTArchitect using the “[Run](#)” command.

Saving and Writing the Output Files

[Table 3-7](#) lists the commands you use *after* the “Run” command for saving and writing the output files and patterns.

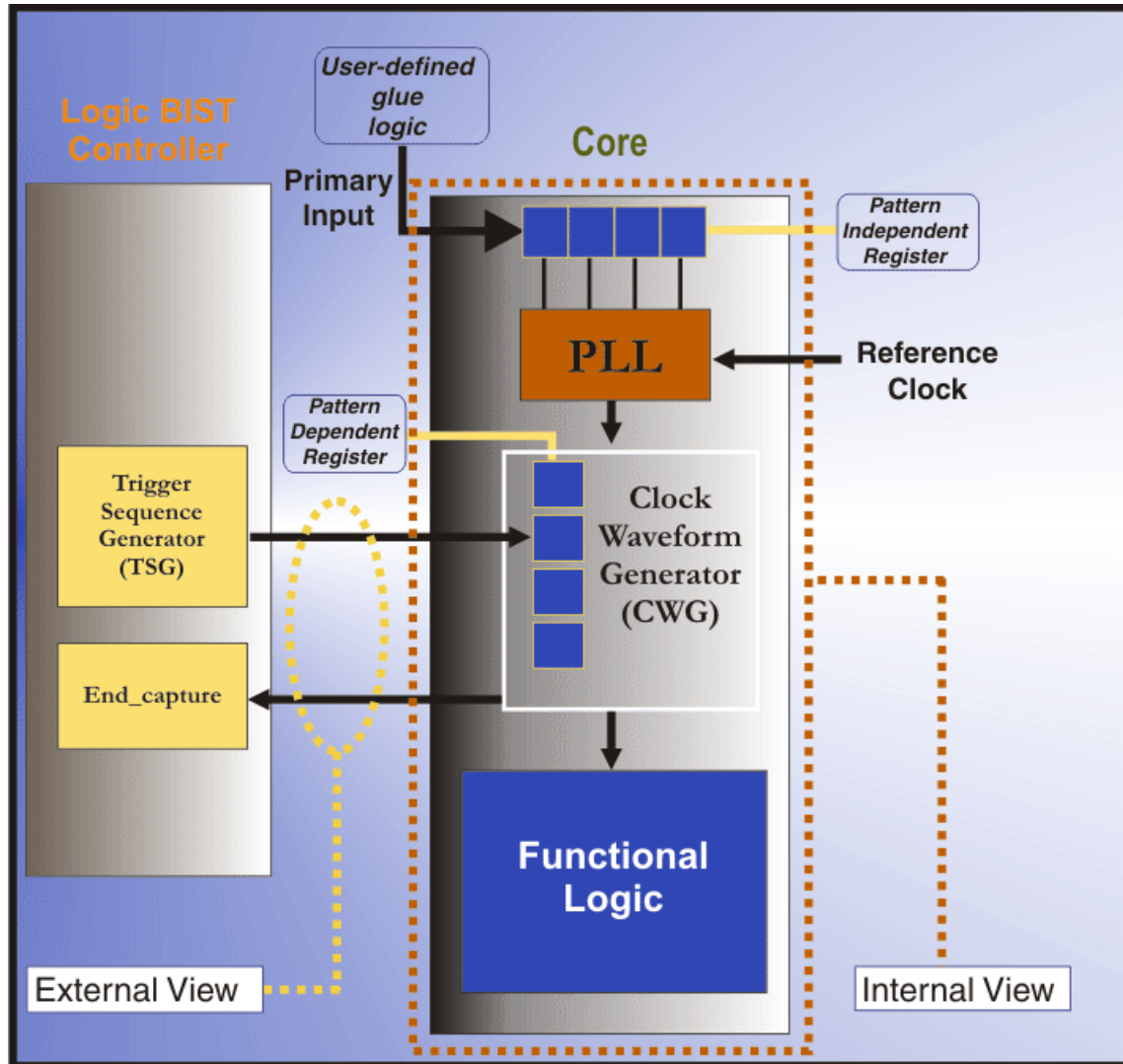
Table 3-7. BIST Controller Generation Save Commands

Command	Description
Save BIST	Saves the BIST output files, using the same language format as the input netlist.
Save Sequential Patterns	Saves the specified number of sequential patterns to the specified file for sequential fault simulation of the BIST controller with FlexTest.

Using On-Chip PLL Clocks for BIST Capture

You can alternatively configure the BIST controller to drive on-chip PLL clock generators instead of the BIST controller clocks and use the PLL clocks directly during for the BIST capture. Figure 3-5 illustrates the generic interface for the BIST controller and PLL circuit.

Figure 3-5. LBISTArchitect PLL Generic Interface



On-Chip Registers

Typically, the following on-chip registers control the PLL and Clock Waveform Generator (CWG):

- **Pattern Independent Registers** — Controlled through the core's primary input. In Figure 3-5, the registers are serially loaded through a scan chain connecting to the core's

primary input. The contents of Pattern Independent Registers do not change for an entire BIST run.

- **Pattern Dependent Registers** — Controlled directly or indirectly by the BIST controller. The contents of the Pattern Dependent Registers can change with patterns.

Internal and External Views

As [Figure 3-5](#) shows, the interface contains the following views in relation to the BIST controller and the core:

- **Internal View** — The clock sequences the PLL/CWG generates. These clock sequences drive the clocks of the functional logic in the core.

During fault simulation, LBISTArchitect runs the simulation on the internal view of the clock sequences. Consequently, you must provide the correct timeplate and capture procedure for fault simulation. See “[Limitations For Using PLL Clocks](#)” for more information.

- **External View** — The trigger signals between the BIST controller and the PLL/CWG, including the associated signal waveforms. In the external view, the BIST controller has no access to the patterns or information regarding the top-level clock (free running reference clock) that drives the PLL.

Limitations For Using PLL Clocks

The following list describes the limitations for using use on-chip PLL clocks for BIST capture:

- **Reference Clocks** — [Figure 3-5](#) shows a free-running reference clock driving the PLL. You must connect the reference clock to the appropriate top-level clock tree or clock input.
- **Test Bench** — Because LBISTArchitect has no access to the top-level clock or PLL behavior, you must provide the appropriate testbench (or test vectors) to initialize the pattern independent PLL control registers before the start of the BIST test.
- **Pattern Generation** — Because LBISTArchitect has no access to the patterns in the external view, the tool cannot generate a complete test bench or WGL/STIL patterns.
- **Fault Simulation** — You must provide the internal clocks and capture procedure for fault simulation. Additionally, you must provide information regarding the top-level clock that drives the PLL and information regarding PLL behavior.
- **PLL and Glue Logic** — For correct operation, you must embed the PLL in the core under test. You must design glue logic to shift in values to the Pattern Independent Registers. Additionally, these latches must be separate from other scan chains in the core functional logic so that the values of these control latches remain unchanged during scan chain load/unload.

The BIST Controller PLL Interface Setup Process

Table 3-8 provides the steps you use for setting up the BIST controller interface to the PLL.

Table 3-8. BIST Controller PLL Interface Setup Process

Step	Description
1	Specify the input signals for the PLL or CWG used to trigger specific clock sequences and waveforms— see “ Step 1 — Define the Triggers ”
2	Define the trigger signals to be applied to the core using the PLL signals—see “ Step 2 — Define the Capture Windows ”
3	Define a specific waveform for each PLL trigger signal—see “ Step 3 — Specify the Capture Waveforms ”
4	Optionally specify the name of the core design’s primary input pin to define the end of the capture window—see “ Step 4 — Add Optional End Capture Signals ”
5	Define the internal clock signals from the PLL and CWG—see “ Step 5 — Define the Internal Clock Signals ”
6	Reads a core-level test procedure file and verifies name correlation between the capture window and the named capture procedure—see “ Step 6 — Load and Verify the Test Procedure File ”
7	Generate the BIST controller—see Step 7 — Run the BIST Controller Generation

Input Files

You must supply a valid test procedure file containing timeplates and one named capture procedure for each Add Capture Window command (see “[Step 2 — Define the Capture Windows](#)”). In addition, you must use the same input files, see “[BIST-Controller Phase Inputs](#),” you use for standard BIST-controller generation.

BIST Controller PLL Interface Commands

Table 3-9 lists the commands you use for creating the BIST controller PLL interface.

Table 3-9. BIST Controller PLL Interface Setup Commands

Command	Description
Add Capture Window	Creates custom capture window waveforms for testing designs with multiple clocks.
Add Clocks	Specifies the names and inactive states of the core design’s primary input pins to be considered for multi-clock domain BIST.

Table 3-9. BIST Controller PLL Interface Setup Commands

Command	Description
Add Pll End_capture	Specifies the name of the core design's primary output pin, which indicates to the BIST controller a capture window is ended.
Add Pll Triggers	Specifies the names and inactive states of the core design's primary input pins for consideration as on-chip PLL-driven BIST.
Delete Pll End_capture	Removes the core design PLL End_capture signal.
Delete Pll Triggers	Removes all core design PLL trigger signals or the specified PLL trigger signals.
Read Procfile	Reads the core-level test procedure file and checks the file's syntax.
Report Pll End_capture	Displays the PLL End_capture signal you have added with the Add Pll End_capture command.
Report Pll Triggers	Displays a list of all PLL trigger signals you have added with the Add Pll Triggers command.
Set Capture Waveform	Defines the waveform that the tool applies to the core when the named capture window is active.

You setup the BIST controller to PLL interface during the BIST controller generation phase of the LBISTArchitect flow using the following steps in either a dofile or directly from the tool's command line interface. See [“Invoking and Using LBISTArchitect”](#) and [“Examining an Example BIST Controller Generation Run.”](#)

You can find detailed examples of this process in the following sections:

- [“Example 1 — No End Capture Signal”](#)
- [“Example 2 — End Capture Signal”](#)

Step 1 — Define the Triggers

In either the dofile or from the command line, identify and add the names and inactive states of the core design's primary input pins for use as on-chip PLL driven BIST using the [Add Pll Triggers](#) command. This command uses the following syntax:

```
add pll triggers off_state primary_input_pins...
```

For example:

```
add pll triggers 0 PLL_trigger1
```

Step 2 — Define the Capture Windows

In either the dofile or from the command line, add the capture windows using the [Add Capture Window](#) command. This command uses the following syntax:

```
add capture window capture_window_name pattern_start_value pattern_stop_value
```

For example:

```
add capture window cap1 0 9
```

Step 3 — Specify the Capture Waveforms

In either the dofile or from the command line, define the specify waveforms the for each PLL trigger signal using the [Set Capture Waveform](#) command. This command uses the following syntax:

```
set capture waveform capture_waveform_name signal_name { PULSE | STRETCH | HOLD }  
"waveform string"
```

For example:

```
set capture waveform cap1 PLL_trigger1 HOLD "000"
```

Note



The length of the capture cycles is based on either the End_capture signal (see “[Step 4 — Add Optional End Capture Signals](#)”) or by the counter size based on the Set Capture Waveform command.

Step 4 — Add Optional End Capture Signals

You can optionally set a handshaking signal from the core to the BIST controller that specifies the end of the capture window. You add the end_capture signal using the [Add Pll End_capture](#) command. When the BIST controller receives the end_capture signal, then the controller terminates the capture window and load/unload scan chains—see “[Example 2 — End Capture Signal](#)”.

In either the dofile or from the command line, specify the end_capture signal using the following command:

```
add pll end_capture primary_output_pin
```

For example:

```
add pll end_capture /core1/end_capture
```

If you omit using an end_capture signal, then you must specify that the BIST controller holds to certain cycles using the Set Capture Waveform command (see “[Step 3 — Specify the Capture Waveforms](#)”).

Step 5 — Define the Internal Clock Signals

In the external view, LBISTArchitect has no access to the internal clock that drives the PLL and CWG. You must identify this clock with the [Add Clocks ... -Internal](#) command. In either the dofile or from the command line, define the internal clocks using the following command:

```
add clocks off_state primary_input_pin -internal
```

For example:

```
add clocks 0 /u_pll/pll_out1 -internal
```

```
add clocks 0 /u_pll/pll_out2 -internal
```

Step 6 — Load and Verify the Test Procedure File

In the test procedure file, you must define a named capture procedure for each window you specify with the Add Capture Window command. The capture window names you use with the Add Capture Window command must be consistent with the named capture procedures in the test procedure file. You verify by loading the test procedure file using the [Read Procfile](#) command.

In either the dofile or from the command line, load the test procedure file for verification using the following command:

```
read procfile test_procedure_file
```

Step 7 — Run the BIST Controller Generation

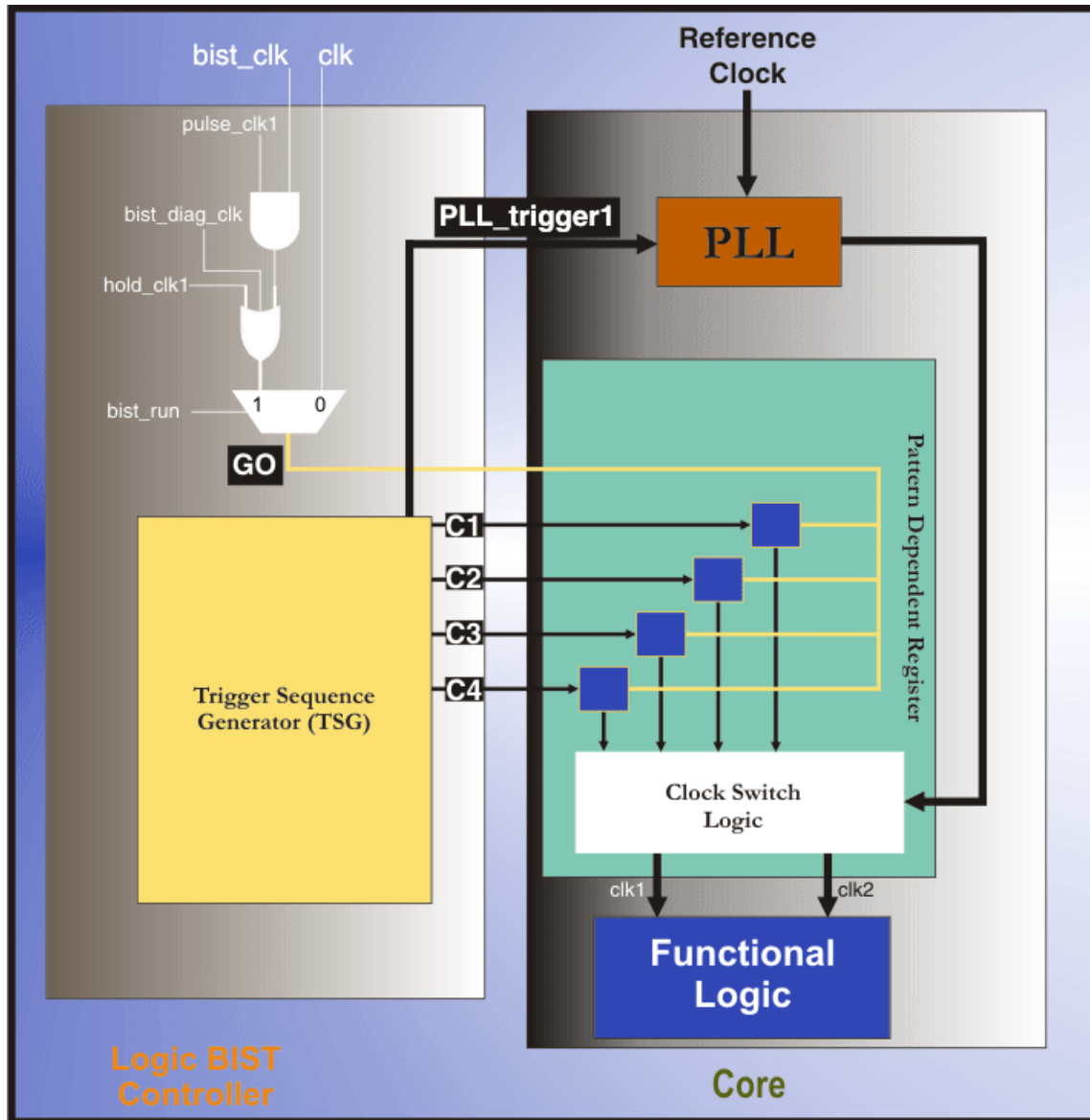
When you have completed defining any BIST setup commands, you can generate the BIST controller with LBISTArchitect using the “[Run](#)” command using either the dofile or the command line.

BIST Controller PLL Interface Generation Examples

Example 1 — No End Capture Signal

Figure 3-6 illustrates an example BIST controller PLL interface with no end capture signal.

Figure 3-6. Example BIST Controller PLL Interface with no End Capture



In this example, there are four Pattern Dependent Control latches you can set directly from the core inputs with the following sequences:

- **Latches set to “0101”** — `clk1` has 200 MHz and `clk2` has 100MHz.
- **Latches set to “1010”** — `clk1` has 100 MHz and `clk2` has 200MHz.

Assume `bist_clk` is 25MHz. You must define the following trigger signals:

- **PLL_trigger1** — The signal that turns the PLL on.
- **C1, C2, C3, and C4** — The signals that are loaded in parallel into the CWG control latches.
- **GO** — The signal to enable the PLL clock output to drive `clk1` and `clk2`.

In functional mode, `clk` drives the CWG control latches; in test mode, `bist_clk` is used. The clock control logic is same as when BIST controller directly controls the core clocks. In this application, however, instead of controlling functional clocks directly, it controls the PLL/CWG.

You implement these signals in the dofile (or interactively using the command interface) using the following command sequence:

1. **Define the PLL Signals** — You must first define external PLL trigger signals using the Add Pll Triggers command.
2. **Define the Capture Windows** — You use the Add Capture Window command to two capture windows—`cap1` and `cap2`.
3. **Define the Waveforms** — Use Set Capture Waveform to define `cap1` and `cap2` with specific waveforms.
4. **Specify the Patterns** — The Add Bist Capture command specifies patterns 0-25 are using `cap1`, patterns 26-31 are using `cap2`, and patterns 32-57 are using `cap1` again, and so forth.

In this example, `clk1` and `clk2` are held for one `bist_clk` cycle to synchronize before firing when `GO` signal is active, and they only active for one `bist_cycle`. In the fault simulation mode, use the named capture procedures with defined internal mode to simulate internal clock sequences. [Figure 3-7](#) shows the waveform for the capture window `cap1`.

The following example dofile shows the command sequence:

```
add pll triggers 0 PLL_trigger1
add pll triggers 0 C1 C2 C3 C4
add pll triggers 0 GO

add capture window cap1 // clk1 200MHz, clk2 100MHz
add capture window cap2 // clk1 100MHz, clk2 200MHz

set capture waveform cap1 PLL_trigger1 HOLD "111" // bist_clk 25MHz
set capture waveform cap1 C1 HOLD "000"
set capture waveform cap1 C2 HOLD "111"
set capture waveform cap1 C3 HOLD "000"
set capture waveform cap1 C4 HOLD "111"
set capture waveform cap1 GO HOLD "010"

set capture waveform cap2 PLL_trigger1 HOLD "111" // bist_clk 25MHz
```

```
set capture waveform cap2 C1 HOLD "111"
set capture waveform cap2 C2 HOLD "000"
set capture waveform cap2 C3 HOLD "111"
set capture waveform cap2 C4 HOLD "000"
set capture waveform cap2 GO HOLD "010"

add bist capture cap1 0 25 //patterns 0-25 use cap1
add bist capture cap2 26 31 //patterns 26-31 use cap2 and so on
// the sequence automatically repeats
// so patterns 32-57 will again use cap1
// and 58-63 use cap2...
```

In the test procedure file, you specify the two named capture procedures (cap1 and cap2) for use in fault simulation. The following example shows the contents of the test procedure file:

```
set time scale 1 ns;

timeplate tp1 =
force_pi 0;
double_pulse /core1/clk1 2 2 6 2; // clk1 200MHz
pulse /core1/clk2 2 4; // clk2 100MHz
measure_po 9;
period 10;
end;

timeplate tp2 =
force_pi 0;
double_pulse /core1/clk2 2 2 6 2; // clk2 200MHz
pulse /core1/clk1 2 4; // clk1 100MHz
measure_po 9;
period 10;
end;

procedure capture cap1 =
    mode internal =
        timeplate tp1;
        cycle =
            pulse /core1/clk1;
            pulse /core1/clk2;
        end;
cycle =
    pulse /core1/clk1;
    pulse /core1/clk2;
end;
cycle =
    pulse /core1/clk1;
    pulse /core1/clk2;
end;
cycle =
    pulse /core1/clk1;
    pulse /core1/clk2;
end;
end;
end;
```

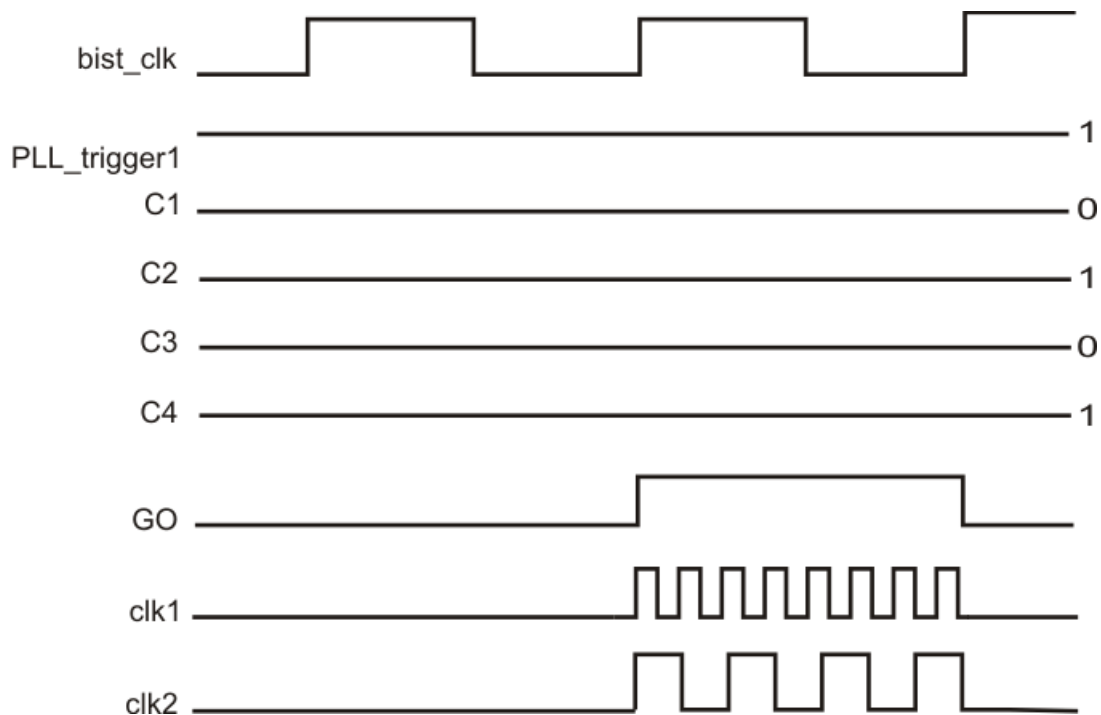
```

procedure capture cap2 =
  mode internal =
    timeplate tp2;
    cycle =
      pulse /core1/clk1;
      pulse /core1/clk2;
    end;
  cycle =
    pulse /core1/clk1;
    pulse /core1/clk2;
  end;
  cycle =
    pulse /core1/clk1;
    pulse /core1/clk2;
  end;
  cycle =
    pulse /core1/clk1;
    pulse /core1/clk2;
  end;
end;
end;

```

Figure 3-7 show the resulting waveforms for cap1 for this example.

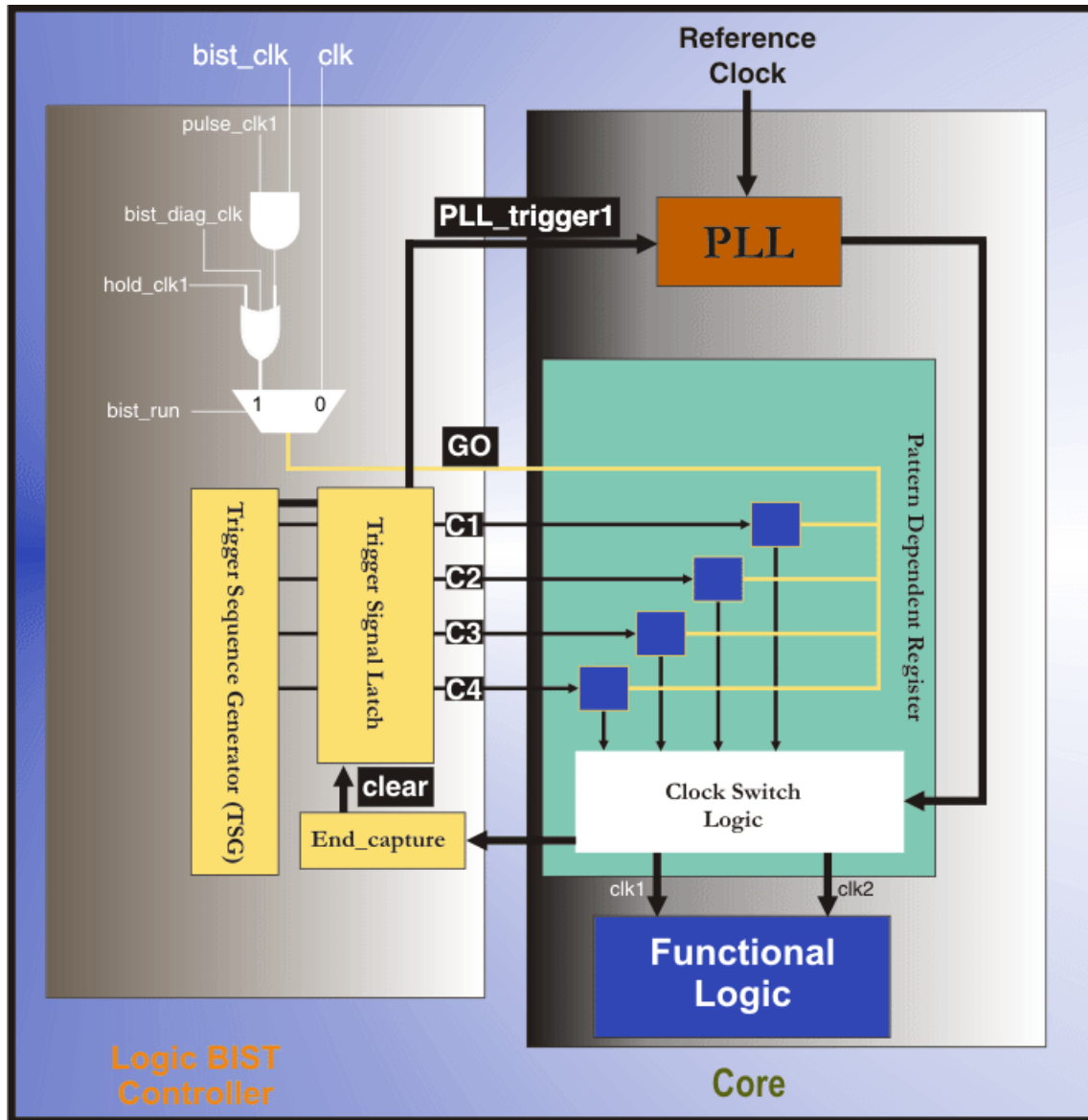
Figure 3-7. Example Waveform for Capture Window cap1



Example 2 — End Capture Signal

Figure 3-8 illustrates an example BIST controller PLL interface with an end capture signal.

Figure 3-8. Example BIST Controller PLL Interface with End Capture



As in the previous example, this example uses four CWG Pattern Dependent control latches you can set directly from the core inputs with the following clock sequences:

- **Latches set to “0101”** — clk1 has 200 MHz and clk2 has 100MHz.
- **Latches set to “1010”** — clk1 has 100 MHz, clk2 has 200MHz.

The internal clock clk1 and clk2 have to be active for several bist_clk cycles. To save additional hardware for BIST counter for an extended capture window and assuming there is an

end_capture signal provided from the core, you use the signal to terminate capture window, instead of using BIST controller counter. To make sure all trigger signals are still valid when the waveform (from the Set Capture Waveform commands) is done, you add a Trigger Signal Latch for each signal.

You do this by using commands in the previous example and plus an the following additional command:

add pll end_capture /core1/end_capture

This command specifies the end_capture signal.

In this example, the BIST controller still initializes the PLL clocks using the values you set with the Set Capture Waveform command, and uses a latch to hold each signal. With the end_capture signal, the BIST controllers waits until the End_capture signal is active and clears all the latches so all the trigger signals are inactive.

Chapter 4

At-speed Testing of Designs with High Speed Clocks

Generating Clock Signals

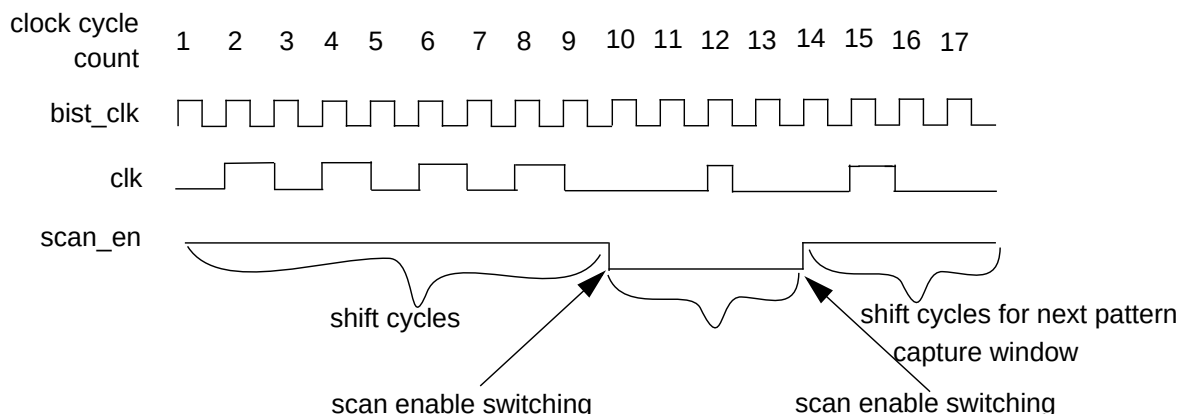
Logic BIST testing is based on the fully autonomous application of pseudo-random patterns through a scan interface. This means the on-chip hardware must control the clock signals during scan chain shifting (pattern loading), and also during the application of the pattern in functional mode (the capture window).

A single global signal (scan enable) controls the operating mode of the circuit (pattern loading or capture). This signal may operate at relatively slow speed which means we must also consider clock control during this time period; we refer to this clock control as "scan enable switching". This section focuses on LBIST clock control during one of these three modes: pattern loading, scan enable switching, and capture.

Pattern loading involves shifting data into every flip-flop in the design. Each pattern is shifted into the core's scan chains through a sequence of shift cycles which loads every flip-flop in the core design. The shift cycles are followed by a capture window which operates the flip-flops in "normal" or "functional" mode based on the scan enable signal. The value that is stored in the flip-flops during the capture window is then retrieved from the scan, or "functional" outputs, as the next pattern is serially shifted in.

The diagram in [Figure 4-1](#) shows the operation of the system clock and the scan_enable signal during the loading and subsequent capture window. This example shows the waveforms during all three modes.

Figure 4-1. Load and Capture Cycle



High-level Architecture

The LBISTArchitect hardware must control the core clocks during the logic BIST test. The controller generates all the clock signals that are required from a single input clock that is called `bist_clk` at the top level but is referred to as `fast_bist_clk` internally. The following modules are responsible for clock control.

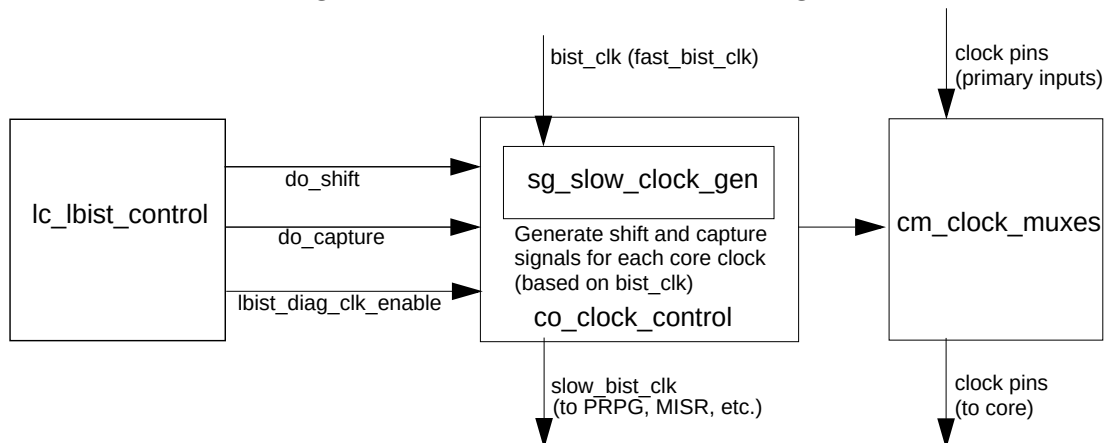
Table 4-1. Clock Control Modules

Module Name	Description
<code>co_clock_control</code>	Generate a “shift” and a “capture” clock signal for each core clock.
<code>sg_slow_clock_gen</code>	Generate a slow speed clock (<code>slow_bist_clk</code>) that is used to operate all logic BIST circuitry except the clock generators in <code>co_clock_control</code> .
<code>cm_clock_muxes</code>	Take control of the core clock signals by inserting a mux. The <code>bist_run</code> signal drives the select line of the mux. During logic BIST, the core clock is created by merging the <code>lbist_shift_clock</code> , <code>lbist_capture</code> , and the <code>lbist_diag_clk</code> signal.
<code>lc_lbist_control</code>	State machine controlling operating mode (pattern loading, scan enable switching, capture window).

lc_lbist_control Block

The high-level operation of the logic BIST controller is defined by the state transitions in the state machine which are defined in the `lc_lbist_control` block (see [Figure B-2](#) on page 125). This state machine is driven from `slow_bist_clk` and generates the following three important signals that feed into the `co_clock_control` block as shown in [Figure 4-2](#): `do_shift`, `do_capture`, and `lbist_diag_clk_enable`.

Figure 4-2. Clock Control Drive Signals

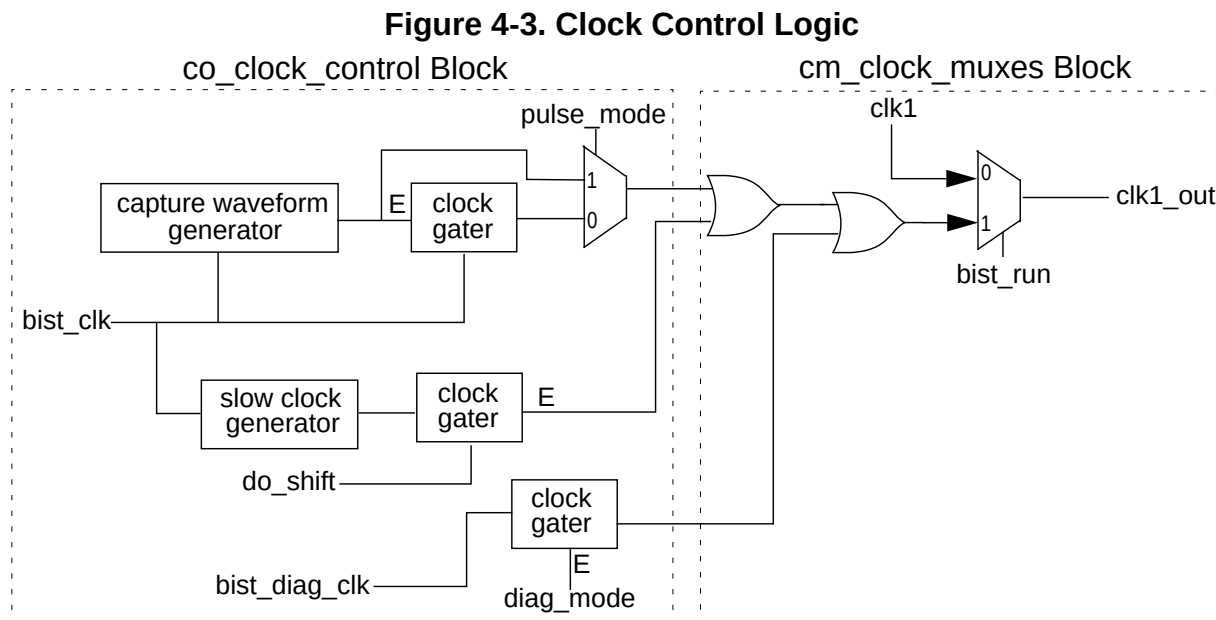


The `do_shift` signal is used to drive `slow_bist_clk` into the core during scan chain load/unload operations. The `do_capture` signal is used to drive capture waveforms into the core. Note that these two signals will never activate simultaneously.

The `lbist_diag_clk_enable` signal is used to block the `lbist_diag_clk` signal except during the diagnostic state, when `lbist_diag_clk` can be used to shift all the core scan chains using a single primary input pin.

Clock Control Low-level Architecture

In order to support the generation of a slow shift cycle and, also, to generate capture waveforms with clocks running at multiple frequencies, we must modify the clock gating logic in the core as shown in [Figure 4-3](#).



The Logic BIST Controller operates from a single “fast” clock input that has previously been referred to as “`bist_clk`”. However, the majority of the logic within the controller is actually driven from an internally generated clock that operates at the same rate as the shift cycles that are applied to the core. This makes it practical to drive the controller from a very high speed signal since only a small block of logic within the controller must actually operate at this rate. The logic within the `co_clock_control` block is responsible for generating the “`slow_bist_clk`” signal and also for generating the pulse and hold signals that drive high speed waveforms into the core. All logic within this block has been designed to minimize the amount of combinational logic between register stages.

Logic within the bist controller uses the `pulse_clk` input to drive a single cycle of `bist_clk` into the core. The `hold_clk1` signal is used to drive slow waveforms into the core. In addition to the pulse and hold inputs, a separate signal called “`bist_diag_clk`” is used to drive the core clock

inputs from an asynchronous clock source during diagnostics. The `bist_diag_clk` input is used to unload the scan chain contents after applying a failing pattern.

Generating Clock Signals During Pattern Loading

This section discusses clock waveforms and control signal generation during pattern loading. During pattern loading, the core clocks are driven by a gated version of the slow bist clock signal. The `sg_slow_clock_gen` module generates the `slow_bist_clk` signal.

Generating the `slow_bist_clk` Signal

The `sg_slow_clock_gen` module functions as a clock divider. The top-level input pin (`fast_bist_clk`) is fed into this module and used to generate the `slow_bist_clk` output. A mux is used to directly connect the input (`fast_bist_clk`) to the output (`slow_bist_clk`) when shifting at full speed. The clock output is driven by an output bit from a `shift_register` when the circuit is configured to function as a clock divider. The full circuit is shown in [Figure 4-4](#), and the various operating modes are described in detail in the next section.

Figure 4-4. Contents of the sg_slow_clock_gen Block

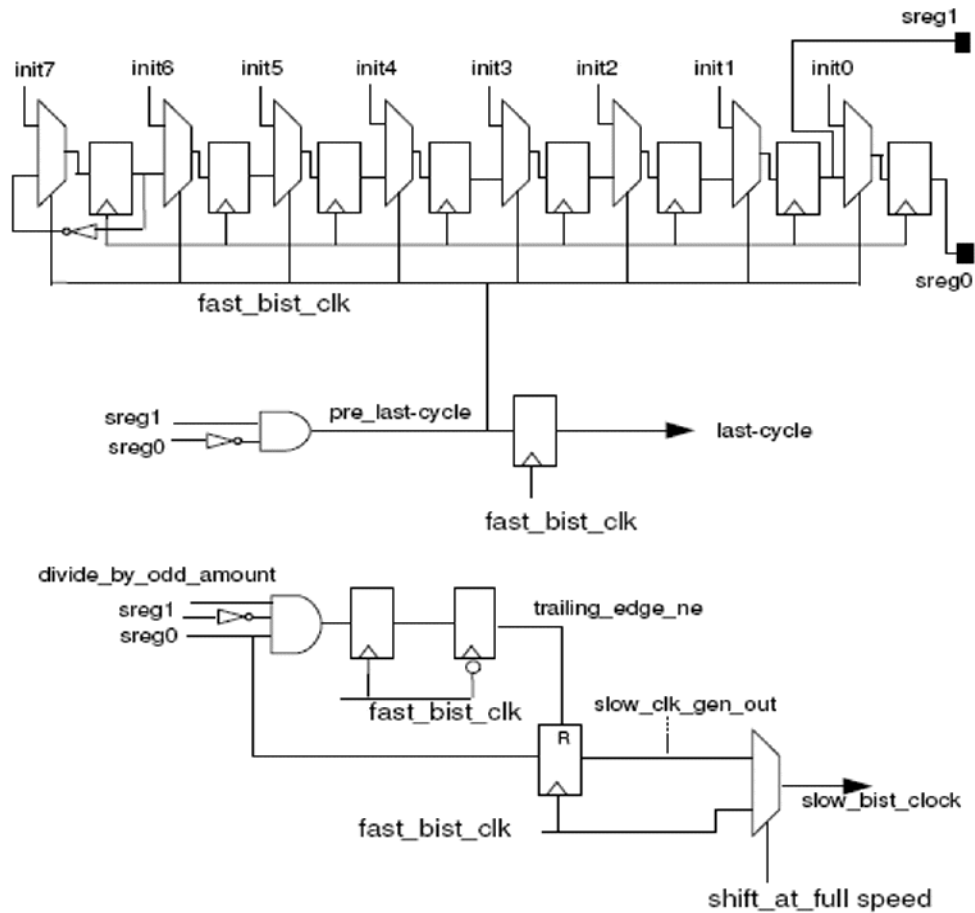


Table 4-2. Decoding the shift_speed register

shift_speed register	Init7 to Init0	divide_by_odd_amount	shift_at_full_speed
000 - divide by 1	00000010	1	1
001 - divide by 2	00000101	0	0
010 - divide by 3	00001011	1	0
011 - divide by 4	00010011	0	0
100 - divide by 5	00100111	1	0
101 - divide by 6	01000111	0	0
110 - divide by 7	10001111	1	0
111 - divide by 8	00001111	0	0

When operating at full speed, the mux is used to directly connect `slow_bist_clk` and `fast_bist_clk`.

When operating as a clock divider, the shift register at the top of [Figure 4-4](#) is initialized such that a “01” combination in the two least significant bits indicates that it is time to reload the shift register. Note that the most significant bit of the shift register is loaded with the inverse of its current output value. This guarantees that the reload signal, labeled `pre_last_cycle`, will eventually force a reload of the register.

Note



The most significant bit of the shift register is loaded with the inverse of its current output value (refer to [Figure 4-4](#)). During actual application, the “01” combination in two least significant bits is generated even though the initial values of the shift register are not known.

When dividing by an even amount, the software delays the shift register output (`sreg0`) by one cycle to create the output clock in the `slow_clk_gen_out` register.

When dividing by an odd amount, the software loads the shift register such that the output waveform would have an asymmetric output (it would be high half of a `fast_bist_clk_cycle` longer than ideal). In order to generate a symmetric waveform, the software identifies the potential trailing edge in `sreg1` and `sreg0`, and then uses a re-timing register to create an asynchronous reset for the `slow_clk_gen_out` signal. This technique creates an output waveform that transitions from low-to-high at the positive edge of `fast_bist_clk`, and transitions from high-to-low at the negative edge of `fast_bist_clk`.

Caution



In the circuitry of [Figure 4-4](#), the generated clock signal uses a different path for the positive edge transition than for the negative edge transition. The timing of the negative edge transition is dependant on the proximity of the “trailing_edge_ne” flip-flop to the “slow_clk_gen_out” flip-flop. If you have tight timing requirements for the negative edge transition, you must modify the synthesis script to reflect this. The timing of the negative edge will also require special attention in the timing constraints file during timing verification.

Glitch Recovery in `sg_slow_clock_gen` Circuit

The `slow_clk_gen` registers are controlled by “`fast_bist_clk`”, but the registers are initialized from combinational logic that is driven by `tck` which means there is a possibility of loading a “invalid” value into the registers when the JTAG interface is used to load a new value into the shift speed control register. Fortunately, the `slow_clock_generator` circuit is designed to recover from this type of glitch within a few `fast_bist_clk` cycles. The RTL for the circuit, and a brief description of how this works is provided below:

```

//////////////////////////////////// .
// simple shift register that resets when the two LSB registers
// reach 10 (effectively next rising edge).
//
// Added are two extra bits at the top of the shift register
// and always force them to 10, so it is known that even if
// slow_clk_gen_init contains junk (this should never happen)
// it will still get to the last_cycle eventually, and reset.
//////////////////////////////////// .

reg [8:0] slow_clk_gen;
reg      slow_clk_gen_ne;

wire last_cycle;
assign last_cycle = (slow_clk_gen[1:0] == 2'b10) || pulse_slow_clk;

// when shifting down, load msb with ~msb, so that we know the
// last_cycle will eventually trip - protect against lockup.

always @(posedge fast_bist_clk or posedge bist_reset)
    if (bist_reset == 1'b1)
        slow_clk_gen <= 9'b000000010;
    else
        if (last_cycle)
            slow_clk_gen <= slow_clk_gen_init;
        else
            slow_clk_gen <= {~slow_clk_gen[8],slow_clk_gen[8:1]};

```

If the init value is 000010011, it will require 3 cycles to reach 10 of the LSB:

```

000010011 - init
100001001 - cycle 1
010000100 - cycle 2
101000010 - cycle 3 - 10 at LSB

```

As can be observed, the MSB is always inverted at every cycle, and the value will be shifted to LSB. This will ensure that the controller will generate the appropriate slow clock once it detects 10 at LSB.

Clock Path and Waveforms When Loading Patterns at bist_clk Frequency

The complete clock path when shifting at speed is shown in [Figure 4-5](#) (the actual path is highlighted in green). The path when shifting at a slower frequency is shown in [Figure 4-7](#).

Figure 4-5. Clock Generation When Shifting At Full Speed

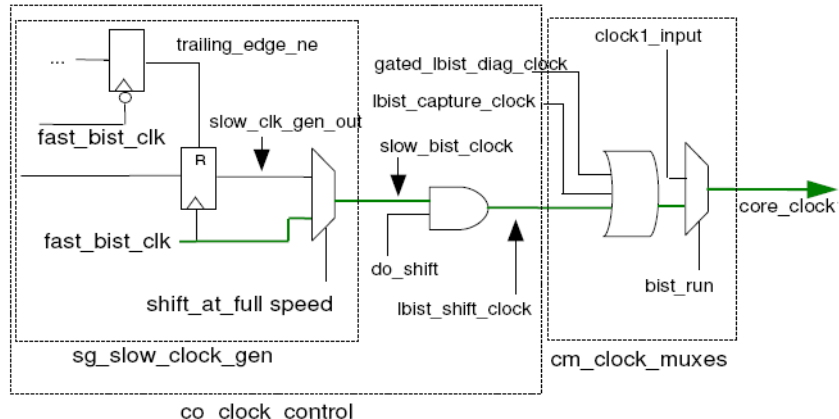
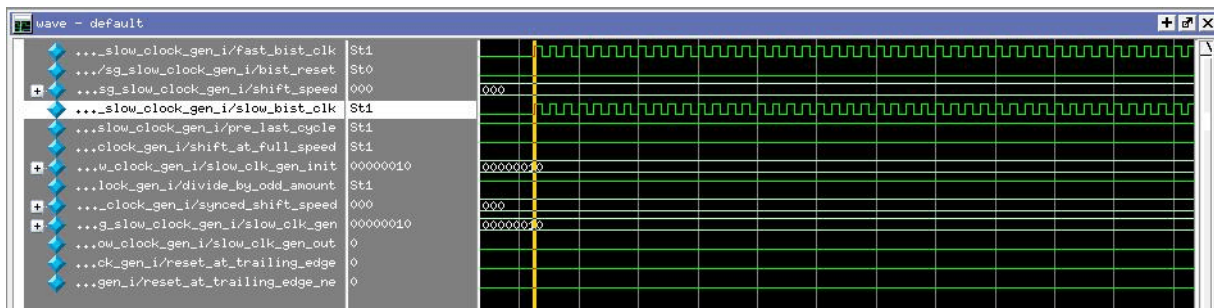


Figure 4-6 shows a picture of the simulation waveforms when shifting at full speed.

Figure 4-6. Clock Waveforms When Shifting At Full Speed



Clock Path and Waveforms When Loading Patterns at 1/n bist_clk Frequency

The `sg_slow_clk_gen` module provides the ability to generate `slow_bist_clk` at fractions of the incoming `bist_clk` signal. This signal is used to shift the core scan chains at lower frequencies thereby reducing timing closure and power consumption. Figure 4-7 shows that the output of the slow clock generator is driving the core clocks when shifting anything less than full speed.

Figure 4-7. Clock Generation When Shifting At Slow Speed

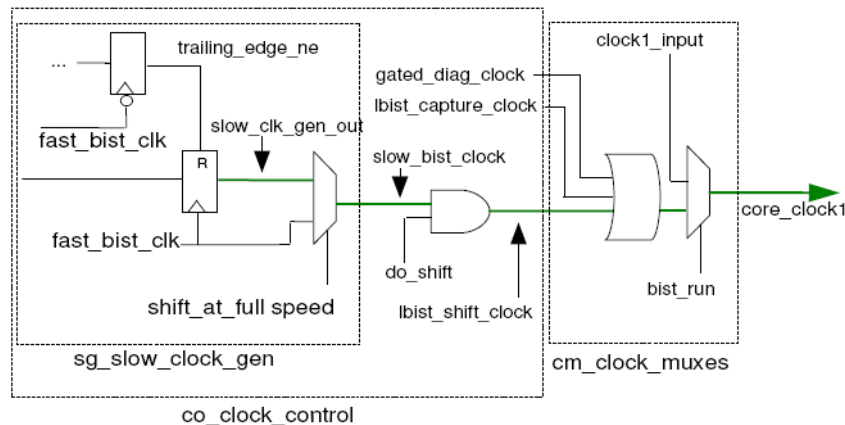


Figure 4-8 shows the waveforms when shifting at 1/2 of the bist_clk frequency.

Figure 4-8. Slow Shift Cycles

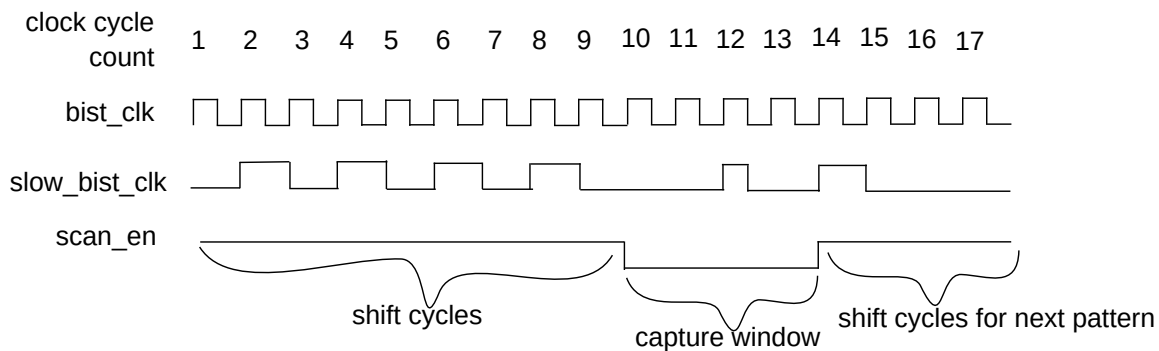
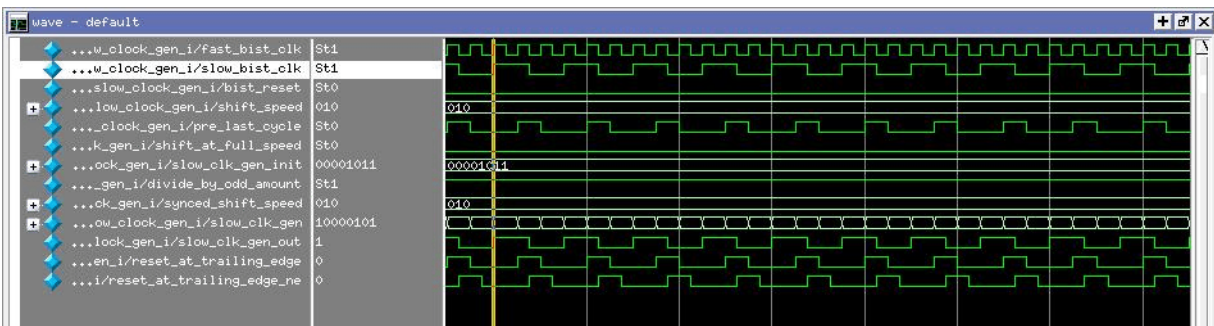


Figure 4-9 shows a picture of the simulation waveforms when shifting at 1/3 of the bist_clk frequency.

Figure 4-9. Clock Waveforms When Shifting At 1/3



For information on generating shift cycles relative to the bist_clk frequency, see the [Set Shift_rate Divisor](#) command in the *LBISTArchitect Reference Manual*.

Generating Scan Enable Signals

This section describes the methodology that LBISTArchitect uses for driving scan enable signals during testing.

Chain Test Patterns

The default behavior of the BIST controller is to apply a few patterns at the start of the test without any clock activity during the capture window. These patterns are called chain test patterns since they are designed to verify the integrity of the scan chain connections and the ability of the BIST controller to shift data through the scan chains.

These patterns do not significantly add to the test coverage, but they are extremely valuable when debugging a signature mismatch. A signature mismatch when unloading a chain test pattern immediately isolates the cause of the problem to the shift path.

Clock Signals During Scan Enable Switching

The scan enable signal changes during a window where the core clocks are gated off. This provides a time period of multiple fast `bist_clk` cycles for the signal transition to propagate through the netlist. Otherwise, scan enable timing can limit the maximum speed of operation for the BIST controller.

Note



The BIST controller is designed to provide a runtime-configurable delay period (with all core clocks disabled) for scan enable switching. As described in section “[Layout of the Scan Enable Control Register](#)” on page 138, the delay between the rising edge of the scan enable signal, and the first shift cycle can be configured between 0 to 15 `slow_bist_clk` cycles (scan enable on delay). The delay between the falling edge of the scan enable and the start of the capture window can also be configured between 0 and 15 `slow_bist_clk` cycles. The default value for both transitions is a delay of one `slow_bist_clk` cycle. This programmable delay period can be used to compensate for slow scan enable transitions that are typically associated with a global signal that must propagate to every scan element (flip-flop) in the design.

Generating Clock Signals During Capture Window

This section describes how LBISTArchitect generates high speed clocks during capture clock waveforms and the slow waveforms during shift cycles.

Stuck-at Testing

Stuck-at testing fault models occur when a terminal of a gate is permanently stuck at either 0 or 1.

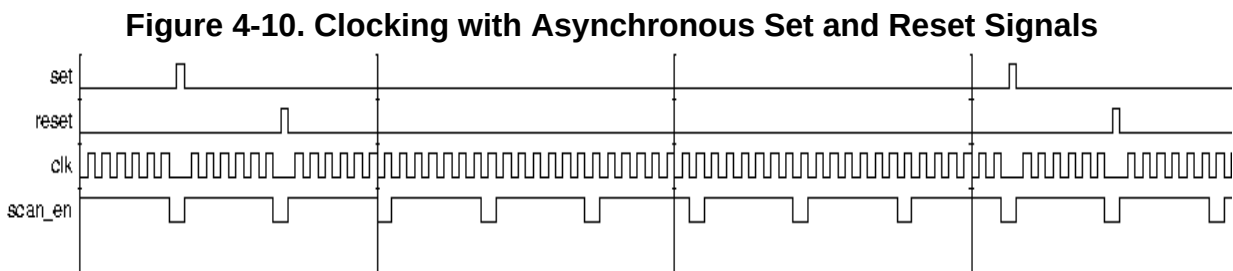
Testing Set and Reset Signals

Set/reset signals must be handled differently than other signals in the core. For most patterns, the set/reset signals should be held in a non-active state. This verifies the operation of the other clock signals in the design, but it does not verify that the set/reset lines work properly when activated.

You test the set/reset signals by choosing a subset of patterns and pulsing only a single set/reset signal during the capture window. Note that set/reset signals cannot be pulsed during the shift cycles because they would reset the flip-flops in the scan chain.

The following diagram shows the use of three different capture windows. The first capture window tests the “set” signal. The next capture window tests the “reset” signal, and the next six capture windows test the main “clk” signal.

The following diagram is for a core design with a single system clock (clk), an asynchronous set line (set), and an asynchronous reset (reset). All of these signals are active high. The maximum scan chain length is six flip-flops so only six shift cycles are needed to re-load the scan chains.



Managing Clock Skew

Another feature of many modern designs is the use of multiple clock domains. This type of design has several balanced clock trees within the core. Even when the clocks are all operating at the same frequency there may be an unknown amount of skew between clock edges, so that it is not possible to pulse both clocks simultaneously during the same capture window and capture predictable values.

The solution to this problem is very similar to the solution for set/reset clocks. Pulse only a single clock during each capture window, and switch between the clocks in a user-configurable manner. Note that all clocks must be pulsed during the shift cycles; otherwise you cannot get data to flow through the scan chains.

Figure 4-10 is for a design with two asynchronous set/reset lines but only a single system clock, so LBISTArchitect connects bist_clk to clk. This approach works equally well with multiple internal clock domains. Each clock is fed through the suppression logic and is activated during selected capture windows through clock gating logic controlled by the bist_clk. Asynchronous set and reset lines are typically pulsed for 1/16th or 1/32nd of the total patterns. Designs with multiple clocks will split the clock activation based on the loading of each clock.

For example, a circuit with two system clocks and an asynchronous reset might use the following cycle, which repeats after the application of 16 patterns.

- **Pattern 0** — Pulse the reset line.
- **Patterns 1 to 12** — Pulse system_clock #1.
- **Patterns 13 to 15** — Pulse system clock #2.

Note that system clock #1 pulses for 12 out of every 16 patterns whereas system clock #2 pulses for only three of the 16 patterns. This is based on a scenario where system clock #1 drives much more logic than system clock #2. LBISTArchitect automatically computes the optimal duty cycle during the BIST_READY phase.

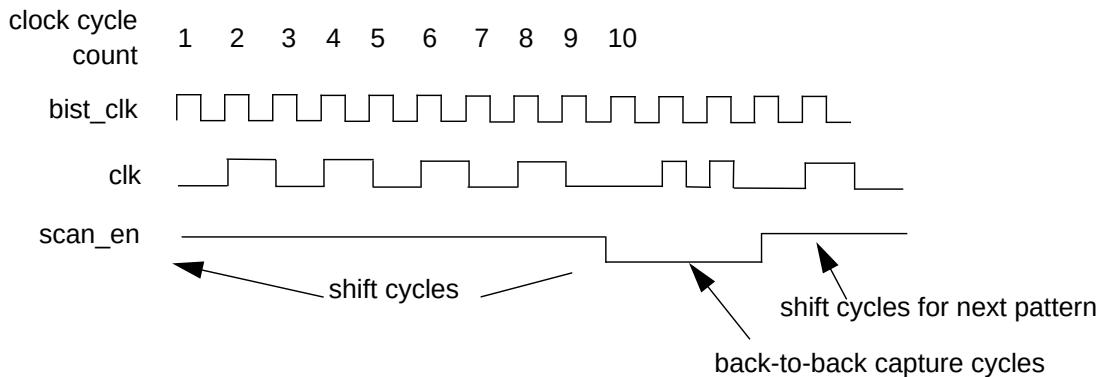
At-speed Testing (Transition Faults)

One problem with the use of slow shift cycles and slow scan enable switching is that the time interval between the last shift cycle and the capture cycle no longer matches functional mode. As a result, we are no longer testing paths at-speed. The solution to this problem is to generate more than one clock pulse during the capture window. This operates the logic at full speed and signal transitions that are captured during the second clock pulse and their associated faults have been tested at functional speed.

The following commands and diagram illustrate using the larger capture window and back-to-back clock pulses to test for transition faults.

```
add capture window cap1 0 12
set capture waveform cap1 clk pulse "11"
set scan_enable switching slow
```

Figure 4-11. Transition Fault Testing



Hardware and Clock Waveforms for Generating At-speed Pulses

The LBISTArchitect tool can generate a variety of clock pulses during the capture window. A `<clk_name>_bist_capture` signal is created for each core clock. This signal is low except when the controller state machine asserts the `do_capture` signal. The hardware is implemented using a shift register. The output bit from the shift register is gated using the `do_capture` signal. Finally, this "gated" version of the shift register output can either directly drive `fast_bist_clk` into the core using another AND gate, or it can feed through one more register stage to produce a slower clock waveform. This hardware is very similar to the clock divider logic in the `sg_slow_clock_gen` block.

The following examples use eight shift register stages. This would be consistent with a capture window size of eight `bist_clk` cycles.

Note



The `init0` through `init7` signals on these diagrams are defined by the capture waveforms in the LBIST dofile, and are not related to the `shift_speed` signals.

Figure 4-12 shows a mode clock waveform during capture is generated using a gated version of `bist_clk`.

Figure 4-12. Hardware for Pulse Mode

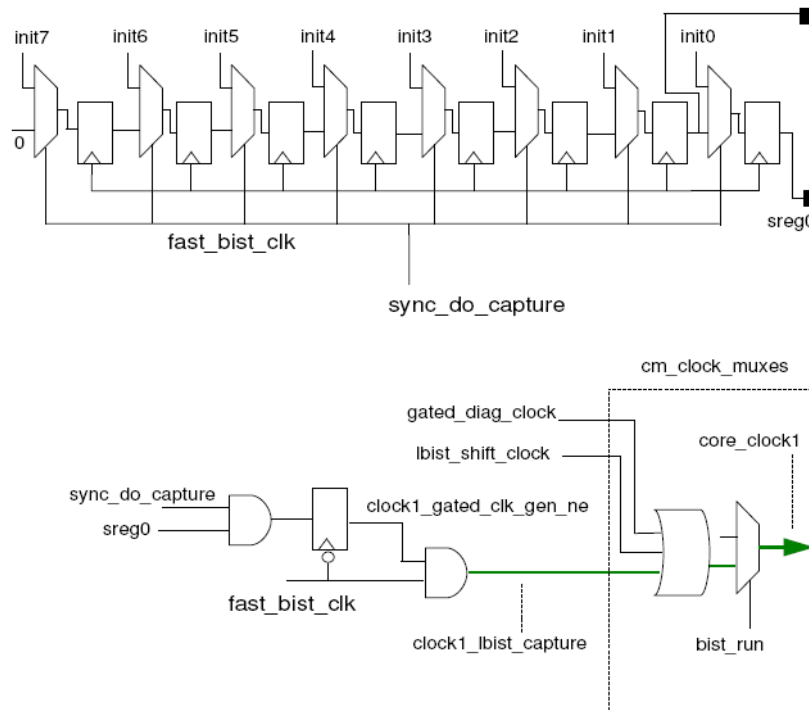


Figure 4-13 shows a picture of the relevant signals when the waveform is defined using the following command:

set capture waveform <cap_window_name> pulse "01001100"

This command results in the following assignments: (init0 = 0, init1 = 1, init2 = 0, init3=0, init4=0, init5=1, init6=1, init7=0);

Figure 4-13. Clock Waveforms During Pulse Mode

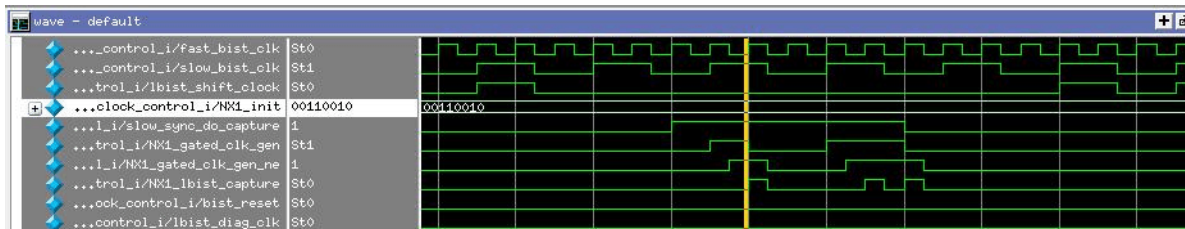


Figure 4-14 shows the Hold Mode. The clock waveform during capture is generated from register output.

Figure 4-14. Hardware for Hold Mode

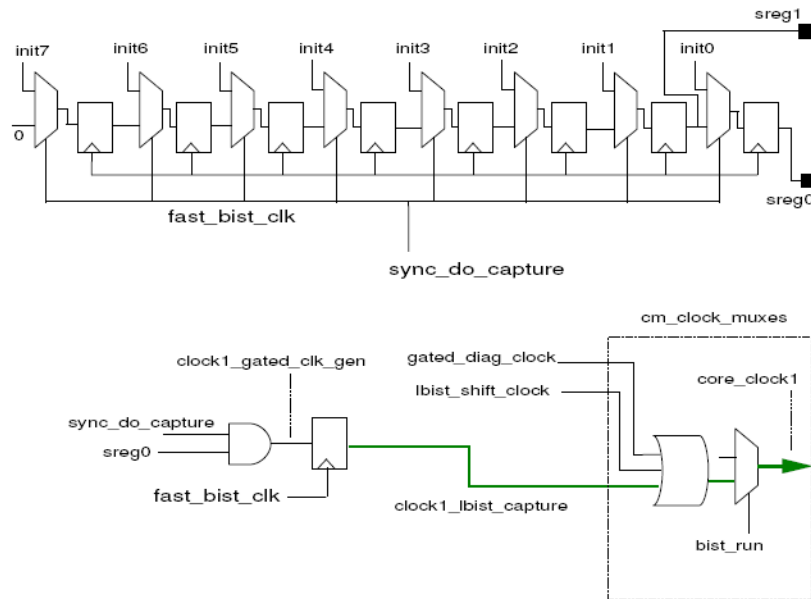


Figure 4-15 shows the clock generated waveform when the waveform is defined using the following command:

Set Capture Waveform <window name> hold “11001100”;

Figure 4-15. Waveforms during Hold Mode

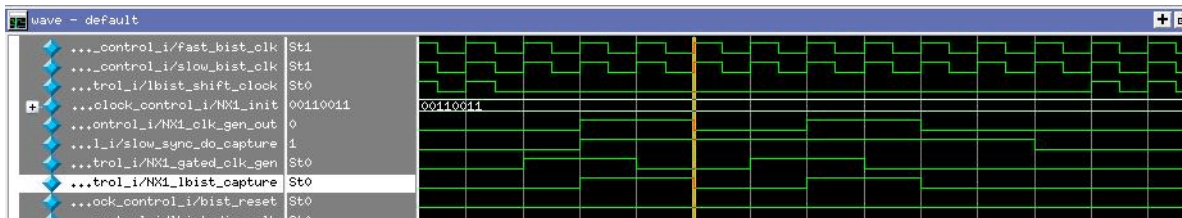


Figure 4-16 shows the Stretch Mode. The clock output generated from register output, but high-to-low transitions occur at negative edge of fast_bist_clk.

Figure 4-16. Hardware for Stretch Mode

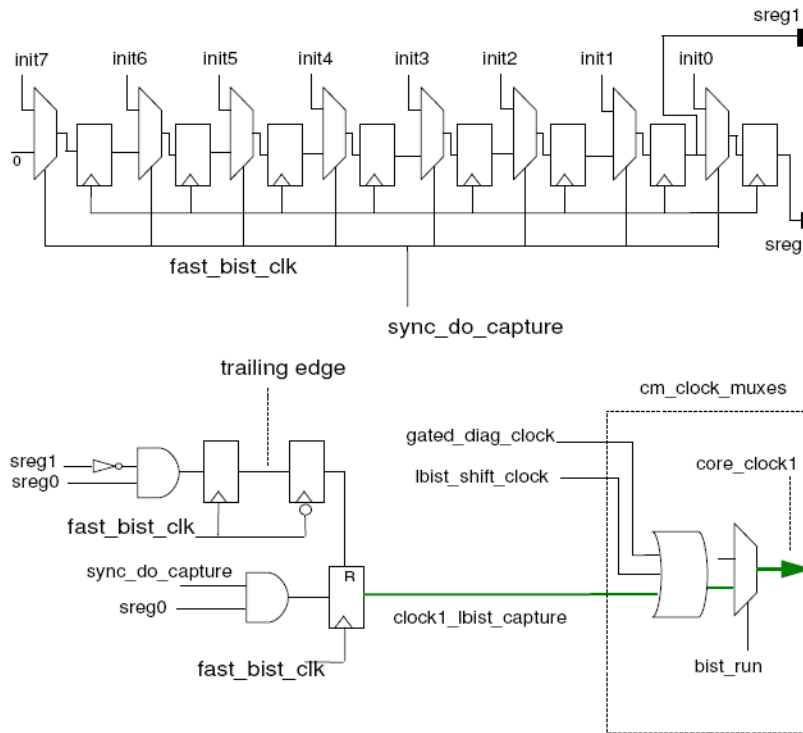
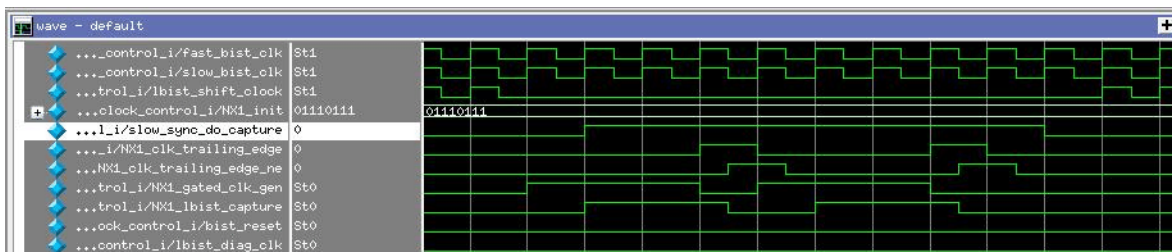


Figure 4-17 shows the clock generator waveforms when the waveform is defined using the following command:

```
set capture waveform <window_name> stretch "11001100";
```

Figure 4-17. Waveforms During Stretch Mode



Testing Paths Between Multiple Clock Domains

The techniques described in the previous sections focused on pulsing a single clock within each capture window. This provides transition fault testing for each clock domain but it does not provide at-speed testing of logic between clock domains. Testing these paths requires capture windows that pulse multiple clocks within the capture window and may even require several capture windows to cover all possible paths.

The following commands and diagram show the use of three different capture windows to test all clock domains (and all inter-clock domain paths) at-speed. This example assumes that all clocks are leading edge triggered.

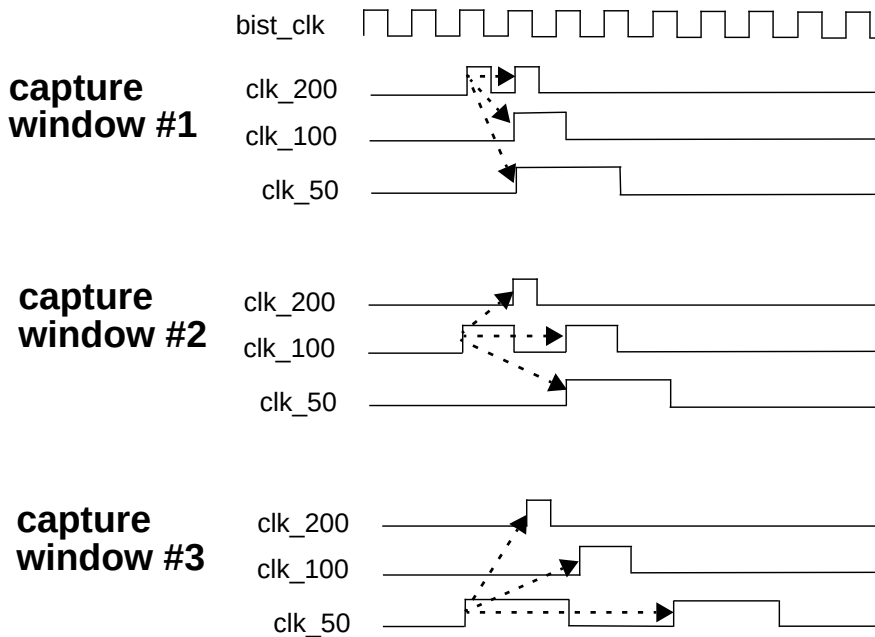
```
add capture window cap1 0 12
set capture waveform cap1 clk pulse "1100"
set capture waveform cap1 clk hold "0100"
set capture waveform cap1 clk hold "0110"
add capture window cap2 0 12
set capture waveform cap2 clk pulse "0100"
set capture waveform cap2 clk hold "1010"
set capture waveform cap2 clk hold "0011"
add capture window cap3 0 12
set capture waveform cap3 clk pulse "0100"
set capture waveform cap3 clk hold "0010"
set capture waveform cap3 clk hold "1100"
```

Note

Please refer to TechNote mg38304 on SupportNet for more information about at-speed testing.

Figure 4-19 shows unloading scan chain contents during diagnostics testing.

Figure 4-18. Multiple Capture Windows



Timing Closure

Performing timing analysis for the Logic BIST hardware will require a separate analysis for each of the following three operating modes.

- [LBIST Scan Chain Shifting](#)
- [LBIST At-speed Capture](#)
- [LBIST Diagnostics Scan Chain Unload](#)

All three of these modes are LBIST specific, so they need to be constrained in the `bist_run` register (in the [li_lbist_interface](#) block) to output a constant 1. For some designs, it may also be necessary to analyze the timing of the scan enable signal. This is discussed in section “[LBIST Scan Enable Switching](#)”.

Note



If you are using Synopsys PrimeTime, LBISTArchitect can optionally generate timing constraint files that define clock frequencies and constraints necessary to test each of these three modes.

LBIST Scan Chain Shifting

The following constraints place the BIST hardware in scan chain shifting mode, and define the clock sources appropriately. The `do_shift` signal must be constrained to 1, the `do_capture` signal

constrained to 0, and the `lbist_diag_clk_enable` signal constrained to 0 (registered outputs from the `lc_lbist_controller` state machine). This set of constraints allows the `slow_bist_clk` signal to pass directly into the core.

If the design must support shifting at full speed, the `shift_speed` register must be constrained to 0. The top level `bist_clk` input is directly connected to the `slow_bist_clk` signal when the `shift_speed` register is constrained to 0. Therefore, the `fast_bist_clk` signal becomes the source of all the internal clocks during scan chain shifting.

If the design does not require shifting at full speed, you can simply constrain the `shift_speed` register to the maximum required shift speed. In this mode, the `slow_clk_gen_out` register (in the `co_clock_control` block) becomes the source of the `slow_bist_clk` signal (which is an internally generated clock operating at the appropriate frequency).

LBIST At-speed Capture

The following constraints are required when LBIST drives at-speed capture sequences into the core clock pins. Constrain the `do_shift` signal to 0, the `do_capture` signal to 1, and the `lbist_diag_clk_enable` signal to 0. The `slow_speed` logic in the BIST controller is basically dormant during the at-speed capture. Each core clock is either driven at full speed, or driven using an internally generated slow speed clock.

When a clock is driven at full speed, the `<clock>_lbist_capture` signal is driven from the `<clock>_and_gate_i` (AND gate) that acts as a clock gate to allow `fast_bist_clk` pulses to pass into the core. (This is shown in [Figure 4-7](#)).

When a clock is driven at a slower rate, the `<clock>_clock_gen_out` register is the source of an internally generated clock. The timing of the clock waveform is based on the values you specify in the Set Capture Waveform command in the LBISTArchitect `bist_controller` phase dofile. This mode of operation is shown in [Figure 4-9](#) and [Figure 4-12](#). The actual clock source for both the “hold” mode, and the “stretch” mode are the same.

MTPI Signals

The phase control signals are used to activate the MTPI control points embedded in the core during test point insertion. These signals are decoded from the pattern counter, and updated during the first shift cycle. These signals are registered, and any path from the register outputs into the core may be treated as a false path (or multi-cycle path with the number of cycles equivalent to the number of shift cycles in the design minus one).

PRPG/MISR Interface

By default, the LBIST controller includes retiming latches at the PRPG outputs, and also at the MISR inputs. The goal is to provide one-half of a `slow_bist_clk` period for skew tolerance. This

means these signals must propagate within one-half of a `slow_bist_clk` period (including clock skew between the edge of `slow_bist_clk`, and the next edge of the core clock signal).

LBIST Scan Enable Switching

The timing of the scan enable signal is typically not critical because it switches during a period of suppressed core clock cycles. However, the timing of this signal must still be verified and can be an issue for design that do not use slow scan enable switching.

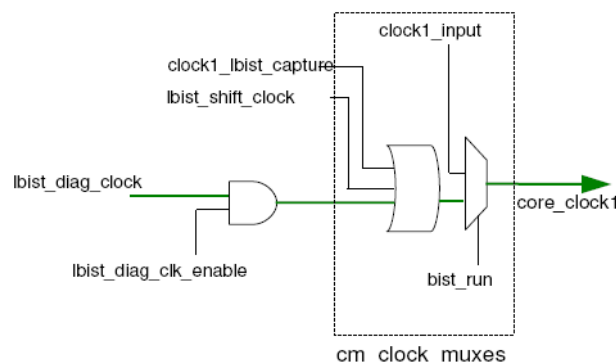
The source of the `scan_enable` signal depends on a registered output from the `lc_lbist_control` block called `do_negedge_scan_enable`. For designs that must support launch-off-shift timing (no suppressed clock cycles between shift and capture), the `do_negedge_scan_enable` signal must be constrained to 0, and the scan enable becomes a half-cycle path (changes on the negedge of `slow_bist_clock`, and is used at the next posedge).

For designs that use suppressed cycles, the `do_negedge_scan_enable` signal can be constrained to 0, and the `scan_enable` changes at the positive edge of `slow_bist_clk`. The `scan_enable` signal becomes a multi-cycle path when more than a single `slow_bist_clk` cycle is suppressed.

LBIST Diagnostics Scan Chain Unload

During LBIST diagnostics, the state machine will enter a “diagnostic hold” state after applying a single pattern. In this diagnostics hold state, the `do_shift`, and `do_capture` signal are both 0, but the `lbist_diag_enable` signal is 1. This allows the `lbist_diag_clk` signal to pass into the three input OR gate in the `cm_clock_mux` block, and drive the clocks into the core. Perform timing analysis for this state by constraining `do_shift` and `do_capture` to 0, and `lbist_diag_clk_enable` to 1.

Figure 4-19. Unloading Scan Chain Contents During Diagnostics Testing

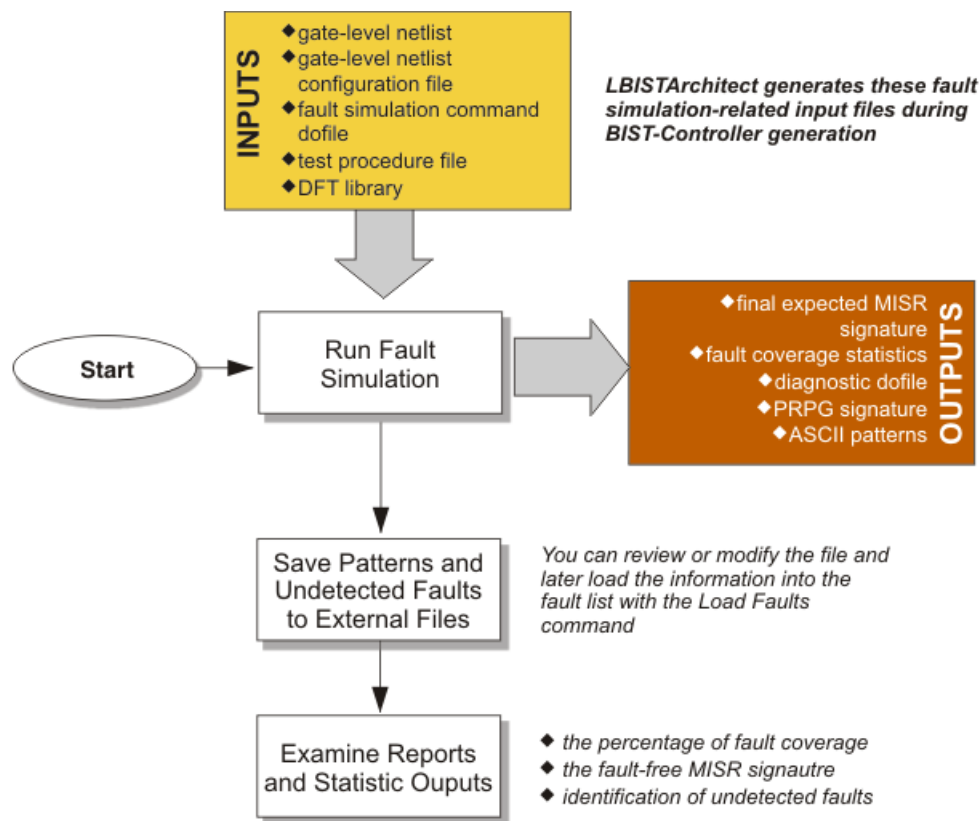


Chapter 5

Performing Fault Simulation

Understanding Fault Simulation

The Fault Simulation Process



Understanding Fault Simulation

LBISTArchitect fault simulation is a functional test you use for ascertaining the following for your design:

- The percentage of [Test and Fault Coverage](#)
- The fault-free [PRPG Output and MISR Signature](#)

During fault simulation, LBISTArchitect emulates the BIST hardware of your design using the actual connections of the PRPG, MISR, and scan chains.

Fault Simulation Types

When using LBISTArchitect, you can choose to perform both [Core Fault Simulation](#) and [Design Fault Simulation](#). Either of these types of fault simulation require additional input files—see [“Fault Simulation Process Inputs”](#) for complete coverage of the inputs for either fault simulation type.

 **Note** The process for performing the fault simulation is identical, regardless of whether you do core fault simulation or fault simulation of the design: the key difference at runtime is the input files to LBISTArchitect you provide at runtime.

Core Fault Simulation

Fault simulation exclusively covering the core. Typically, you perform this type of fault simulation early in your design cycle. This type of fault simulation uses the netlist you created with LBISTArchitect during the BIST-ready process. See [“Inputs and Outputs of Second Run of BIST-Ready Phase”](#) in the [“Preparing the Design for Logic BIST”](#) section.

Design Fault Simulation

Fault simulation using the final netlist and covering the entire design, both the core and the emulated BIST controller. This type of fault simulation requires a gate-level netlist pre-synthesized from the RTL netlist of the BIST controller and the core from a synthesis application.

Test and Fault Coverage

Fault simulation identifies which faults the test patterns detect using either the stuck-at fault model or transition fault model. The stuck-at fault models the behavior occurring if the terminals of a gate are stuck at either a high (stuck-at-1) or low (stuck-at-0) voltage. Transition faults model large delay defects at gate terminals in the circuit under test. The slow-to-rise transition fault models a device pin that is defective because its value is slow to change from a 0 to a 1. The slow-to-fall transition fault models a device pin that is defective because its value is slow to change from a 1 to a 0. The fault sites for both fault models include the pins of primitive instances.

Fault Classes and Coverage

At the conclusion of fault simulation, LBISTArchitect lists faults into a fault class by a 2-letter code. [Table 5-1](#) lists these codes and the corresponding fault class coverage.

Table 5-1. Fault Class Codes and Names

Fault Class Codes	Fault Class Names	Fault Class Coverage
FU	Full	TE+UT
TE	TEstable	DT+PD+OS+HY+AU+UD
DT	DETEcted	DS+DI+DR
DS	DET_Simulation	
DI	DET_Implication	
PD	POSDET	PT+PU
PU	POSDET_Untestable	
PT	POSDET_Testable	
AU	Atpg_untestable	
UD	UNDEtected	UC+UO
UC	UNControlled	
UO	UNObserved	
UT	UNTestable	UU+TI+BL+RE
UU	UNUsed	
TI	TIed	
BL	Blocked	
RE	Redundant	

Coverage Statistics

[Figure 5-1](#) shows the typical statistical output of LBISTArchitect at the conclusion of a fault simulation run.

Figure 5-1. Typical Fault Simulation Statistics Report

test_coverage

$$\frac{\#DT + (\#PD * \text{posdet_credit})}{\#testable} \times 100$$

fault_coverage

$$\frac{\#DT + (\#PD * \text{posdet_credit})}{\#full} \times 100$$

atpg_effectiveness

$$\frac{\#DT + \#UT + \#AU + \#PU + (\#PT * \text{posdet_credit})}{\#full} \times 100$$

```
// command: report statistics
```

Statistics report		
fault class	#faults (coll.)	#faults (total)
FU (full)	5930	12886
UC (uncontrolled)	2	2
UO (unobserved)	37	79
DS (det_simulation)	5090	11730
DI (det_implication)	588	784
PT (posdet_testable)	4	14
RE (redundant)	206	274
TI (tied)	3	3
test_coverage	99.28%	99.30%
fault_coverage	95.78%	97.17%
atpg_effectiveness	99.31%	99.32%
#test_patterns		167
#simulated_patterns		209
CPU_time (secs)		43.7

Test Coverage: the percentage of all testable faults detected by a pattern set.

Fault Coverage: the percentage of *all* faults (both testable and untestable) detected by the pattern set.

ATPG Effectiveness: a measure of the ability of the ATPG tool to either provide a test to detect a fault, or prove a test cannot be created.

PRPG Output and MISR Signature

During fault simulation, LBISTArchitect computes the actual PRPG output and MISR signature. [Figure 5-2](#) shows an example output and signature.

Figure 5-2. PRPG Output and MISR Signature

```
list of LFSRs
prpg PRPG length=16 shift=serial in_stream_taps=11,13,14
  prpg_current_value=0001001000001001
misr MISR length=32 shift=serial in_stream_taps=10,30,31
  misr_current_value=10000110000111110010110001110000
```

Note



Faults have no effect on the PRPG. LBISTArchitect provides the actual PRPG value for verification and completeness.

Supplemental Fault Simulation Information

Table 5-2 provides a listing of supplemental fault simulation-related information in Mentor Graphics Design-for-Test documentation.

Table 5-2. Supplemental Fault Simulation References

Topic	Refer to
Fault Classes	<i>Scan and ATPG User's Manual</i>
Fault Collapsing	
Functional Test	
Test Types and Associated Fault Models	
Debugging DRC Violations	<i>LBISTArchitect Reference Manual</i>

Alternatively, you can search the Mentor Graphics extensive SupportNet Knowledge Base at the following URL:

<http://supportnet.mentor.com>

The Fault Simulation Process

The fault simulation process is a process for determining test and fault coverage, and generating the PRPG output and MISR signature for your BIST'd design.

Note



Under normal circumstances, you use the *as is* fault simulation dofile LBISTArchitect created when [Generating the BIST Controller](#). This dofile contains the necessary commands for automatically running fault simulation.

The following figure shows the stages in the process. [Table 5-3](#) describes each of the stages.

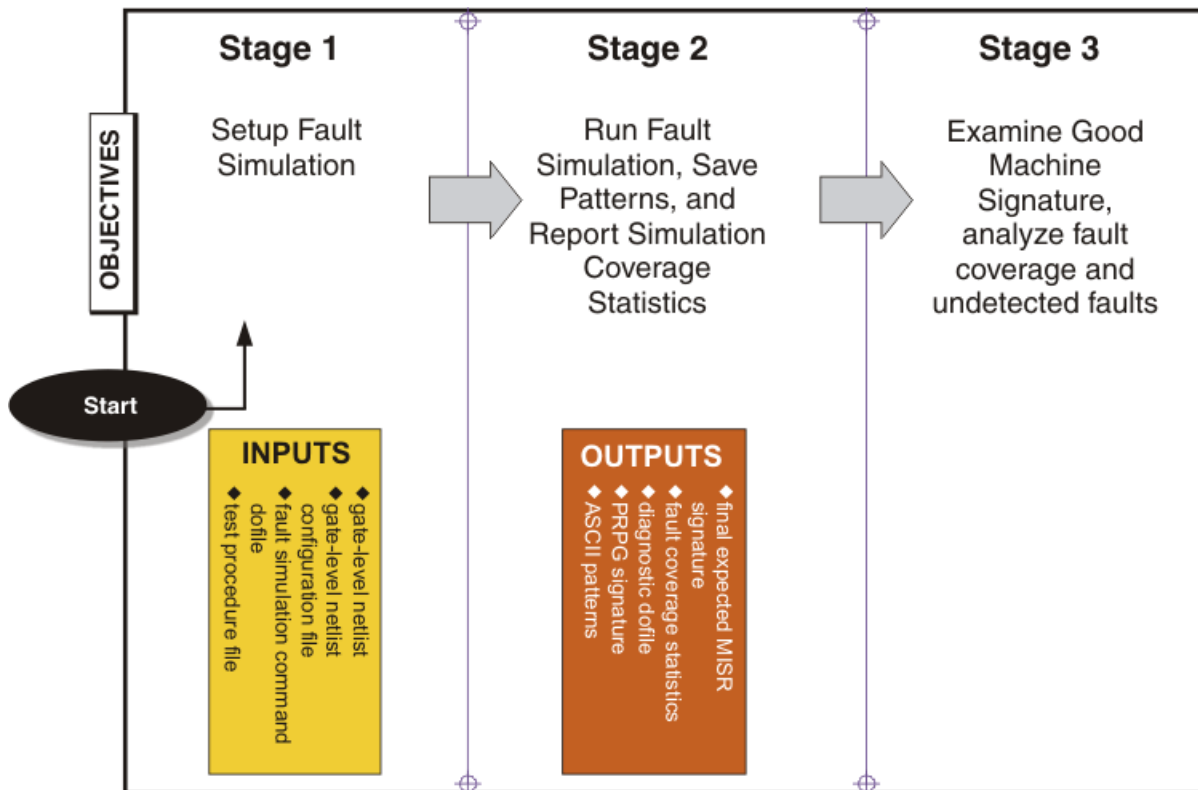


Table 5-3. Fault Simulation Process

Stage	Description
1	Setup Fault Simulation: collect the required input files. See “Fault Simulation Process Inputs.”
2	Run Fault Simulation: invoke LBISTArchitect in fault simulation mode, run the simulation, report the results, and save the patterns. See “Performing Design Fault Simulation” or “Performing Core Fault Simulation.”
3	Review the Simulation Results: inspect the PRPG output and MISR signature. See “Examining Fault Simulation Results.”

Fault Simulation Process Inputs

During BIST controller generation, LBISTArchitect creates the majority of the input files you provide to the tool for fault simulation, whether you are performing [Design Fault Simulation](#) or [Core Fault Simulation](#). LBISTArchitect assigns these files default names unless you modified the output names using the [“Setup File Naming”](#) command—see [“Output File Naming”](#) in the [“Generating the BIST Controller”](#) section.

Design Fault Simulation Input Files

[Table 5-4](#) identifies and describes the input files you must have for *design fault simulation* process.

Table 5-4. Inputs for the Design Fault Simulation Process

Input	Default Name from BIST-Controller Generation
Gate-level BIST controller and core netlist pre-synthesized from the RTL netlist	
Test procedure file	<i>entity_bist.testproc</i> <i>entity_trans.testproc</i>
Design fault simulation dofile ¹	<i>entity_faultsim.do</i> <i>entity_trans_faultsim.do</i>
ATPG DFT Library (for example, atpglib)	

1. See “[Deconstruction of Fault Simulation Dofile](#)”

Core Fault Simulation Input Files

[Table 5-5](#) identifies and describes the input files you must have for *core fault simulation* process.

Table 5-5. Inputs for the Core Fault Simulation Process

Input	Default Name
RTL netlist of the BIST controller	<i>entity_bist.v</i>
Test procedure file	<i>entity_bist.testproc</i> <i>entity_trans_core.testproc</i>
Core fault simulation dofile ¹	<i>entity_core_faultsim.do</i> <i>entity_trans_corefaultsim.dofile</i>
ATPG DFT Library (for example, atpglib)	

1. See “[Deconstruction of Fault Simulation Dofile](#)”

The Importance of a Fault Simulation Dofile

Under most circumstances, you use *as is* the appropriate simulation dofile LBISTArchitect had automatically created during the BIST Controller generation file. These dofiles contain the precise commands LBISTArchitect uses for successful fault simulation. See “[Example Fault Simulation Dofile](#)” for a line-by-line examination of an example fault simulation dofile.

Fault Simulation Process Dependencies

The fault simulation process uses certain design parameters you specified during the BIST-Ready process including the following:

- DRC and scan insertion operations
- Design clock specification
- Input- and X-bounding identification
- Control and test point insertion

Consequently, if you encounter any of these issues during fault simulation, you should examine the outputs and logs from the BIST-Ready process. For a further description of these parameters, see “[Flow of First Run of BIST-Ready Phase](#)” and “[Flow of Second Run of BIST-Ready Phase](#)” in the “[Preparing the Design for Logic BIST](#)” section.

Fault Simulation Process Outputs

At the conclusion of a successful fault simulation run, LBISTArchitect outputs the following information, normally to the transcript:

- Good machine signature
- Fault simulation results
- Undetected fault list

[Table 5-6](#) lists the additional LBISTArchitect outputs from the fault simulation process.

Table 5-6. Simulation Process Outputs

Output	Default Name
Test patterns	<i>entity_bist.pat.gz</i>
Fault file	user defined

Performing Design Fault Simulation

You use the following procedure for running [Design Fault Simulation](#)—see also “[Design Fault Simulation Input Files](#).”

Running Design Fault Simulation

You invoke LBISTArchitect for design fault simulation using the following command in a shell:

```
<Tessent_Tree_Path>/bin/lbistarchitect -fault_simulation
fault_sim_arguments
```

You use the *fault_sim_arguments* for specifying the following:

- **Netlist** — The gate-level netlist pre-synthesized from the RTL netlist of the BIST controller and the core.
- **Library** — The ATPG DFT Library.
- **Logfile** — The logfile name and location for the run transcription.
- **Design Fault Simulation Dofile** — The dofile name and location of the design fault simulation dofile LBISTArchitect created during the BIST controller generation process. LBISTArchitect provides two different versions of this dofile: the *entity_faultsim.do* file is used to perform stuck-at-fault grading and the *entity_trans_faultsim.do* file is used for transition fault grading.

Example 5-1. Design Fault Simulation Example Invocation

```
lbistarchitect -fault_simulation design_netlist.v -verilog -lib atpglib -nogui -log faultsim.log -rep \
-dof design_faultsim.do
```

Invocation Explanation



lbistarchitect -fault_simulation	invokes the fault simulation mode
design_netlist.v -verilog	loads the design Verilog netlist <i>design_netlist.v</i>
-lib atpglib	specifies the ATPG DFT Library
-log faultsim.log -rep	creates <i>faultsim.log</i> log file
-dof design_faultsim.do	specifies the design fault simulation dofile
-nogui	specifies no graphical user interface

Concluding Design Fault Simulation

At the conclusion of the run, you close LBISTArchitect by issuing the [Exit](#) command.

Performing Core Fault Simulation

You use the following procedure for running [Core Fault Simulation](#)—see also “[Core Fault Simulation Input Files](#).”

Running Core Fault Simulation

You invoke LBISTArchitect for core fault simulation using the following command in a shell:

```
<Tessent_Tree_Path>/bin/lbistarchitect -fault_simulation \  
fault_sim_arguments
```

You use the *fault_sim_arguments* for specifying the following:

- **Netlist** — RTL netlist of the BIST controller LBISTArchitect created during the BIST controller generation process.
- **Library** — The ATPG DFT Library.
- **Logfile** — The logfile name and location for the run transcription.
- **Core Fault Simulation Dofile** — The dofile name and location of the core fault simulation dofile LBISTArchitect created during the BIST controller generation process. LBISTArchitect provides two different versions of this dofile: the *entity_core_faultsim.do* file is used to perform stuck-at-fault grading and the *entity_trans_corefaultsim.do* file is used for transition fault grading.

Example 5-2. Core Fault Simulation Example Invocation

```
lbistarchitect -fault_simulation cpu_bist.v -verilog -lib atpglib -log faultsim.log -rep \  
-dof cpu_core_faultsim.do
```



Invocation Explanation

lbistarchitect -fault_simulation	→	invokes the fault simulation mode
cpu_bist.v -verilog	→	loads the core Verilog netlist cpu_bist.v
-lib atpglib	→	specifies the ATPG library
-dof cpu_core_faultsim.do	→	specifies the core fault simulation dofile

Concluding Core Fault Simulation

At the conclusion of the run, you close LBISTArchitect by issuing the [Exit](#) command.

Examining Fault Simulation Results

LBISTArchitect provides the fault simulation results in both the runtime transcript and files the tool created at the conclusion of the fault simulation run.

Interpreting the Transcript

Figure 5-3 shows information you find at the end of a fault simulation run—see also Table 5-7.

Figure 5-3. Interpreting the Fault Simulation Transcript Results

```

1 command: save patterns cpu_bist.pat.gz -scan_test -ascii -replace
// command: analyze redundancy
// -----
2 Performing fault redundancy identification ...
// -----
// Number redundant faults identified = 82.
// command: update implication detection
// 4 faults were identified as detected by implication.
// -----
3 command: report lfsrs
list of LFSRs
prpg PRPG length=16 shift=serial in_stream_taps=11,13,14
prpg_current_value=0001001000001001
misr MISR length=32 shift=serial in_stream_taps=10,30,31
misr_current_value=10000110000111110010110001110000
// -----
4 command: report statistics

Statistics report
-----
fault class          #faults  #faults
                      (coll.)   (total)
-----
FU (full)            5930      12886
-----
UC (uncontrolled)      2         2
UO (unobserved)       37        79
DS (det_simulation)   5090     11730
DI (det_implication)  588       784
PT (posdet_testable)   4         14
RE (redundant)        206       274
TI (tied)              3         3
-----
test_coverage         99.28%    99.30%
fault_coverage        95.78%    97.17%
atpg_effectiveness    99.31%    99.32%
-----
#test_patterns                167
#simulated_patterns           209
CPU_time (secs)               43.7
-----
5 command: write faults cpu_bist_topup.faultfile -class ds -class di -noeq

```

Table 5-7 provides an explanation of each numbered area in Figure 5-3.

Table 5-7. Explanation of Transcript Results

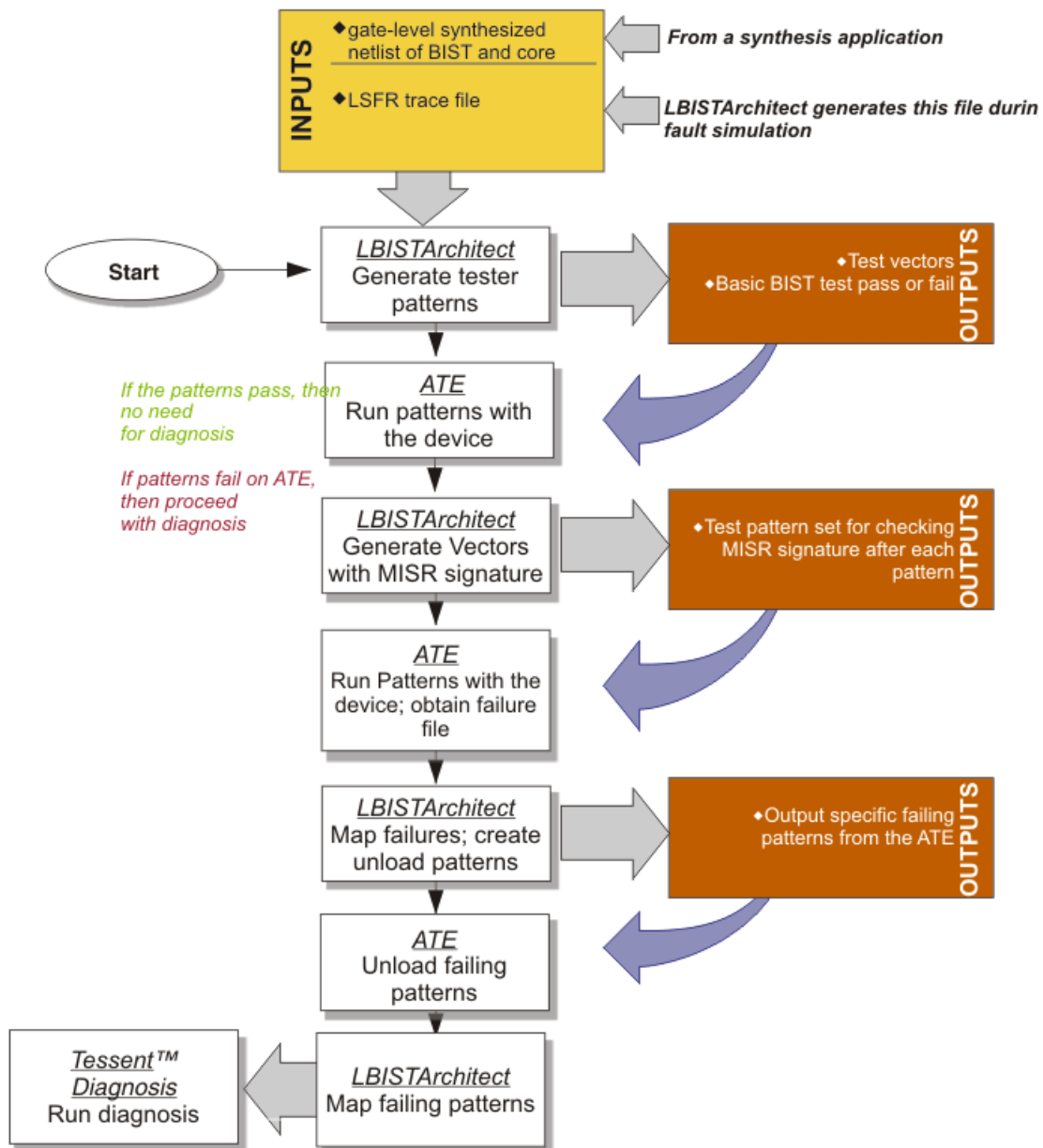
	Details
①	Save Test Patterns —writes the test pattern set to the file <i>cpu_bist_pat.gz</i>
②	Fault Redundancy Identification —lists faults among current the faults whose redundancy has not yet been determined
③	LSFRs and MISR Signature —a list of definitions for all the current Linear Feedback Shift Registers (LFSRs), and the MISR signature
④	Statistics Report —see “ Test and Fault Coverage ”
⑤	Save Faults to a File —writes fault information from the current fault list to a file

Chapter 6

Performing Manufacturing Diagnosis

Understanding the Diagnostic Flow

The Diagnostic Process



Understanding the Diagnostic Flow

The LBISTArchitect diagnostic flow aids you in locating failures in the device when the BIST test fails when testing the actual device.

Single-Step Simulation and MISR Signature Access

LBISTArchitect uses single-step simulation for diagnosis. The following lists the sequence of events during a single-step run:

1. The test vectors set the `single_step` bit in the control register.
2. The test vectors start the test by loading the `dselect` register with `run_bist`.

You can directly access the MISR signature through the data register (see “[Layout of Data Register](#)”) whenever the `run_bist` code (0001) is loaded into the `dselect` register. The `dselect` register remains unchanged while the per-pattern MISR signatures are unloaded/reloaded from the data register. During this time, the controller stops and waits for the data register to be reloaded before applying the next pattern.

3. The `bist_diag_clk` is directly connected to the core scan chain clocks during the period when the controller is waiting for the next MISR signature.

Note



Do not pulse the `bist_diag_clk` during this period. Additionally, do not change the control registers during this period.

See “[Layout of Control Register Bits within Data Register](#)” and “[Logic BIST Controller Data Sheet](#)” for complete information.

TAP Controller and Interconnects

If you use a TAP controller and interconnects, then you must generate these with [BSDArchitect](#). BSDArchitect generates the BSDL file when it generates the boundary scan controller and the interconnects—see “[Using a TAP Controller and Interconnects](#)” and “[Boundary Scan Insertion](#).”

Basic Diagnostic Flow

Diagnosis is usually an iterative process involving the exchange of data files (for example, patterns and failure logs) between LBISTArchitect and your ATE. You subsequently use the outputs from this flow as inputs into the Tessent Diagnosis application.

Basic BIST Test

Perform the following steps to determine if you must use the diagnostic flow:

- **Tester Pattern Creation** — Using LBISTArchitect Test Vector Generator (tvgen), create the tester patterns for stimulating the BIST controller on the ATE.
- **Run Basic BIST Test on ATE** — Using the hardware device on the ATE, run the tester patterns that activate the BIST controller.
 - If the Basic BIST Test passes, then no diagnosis is needed.
 - If the Basic BIST Test fails on the ATE, then proceed with the [Diagnostic Flow](#).

Diagnostic Flow

Perform the following steps if you have run the [Basic BIST Test](#) and have failing devices:

- **Generate Vectors with MISR Signature** — Using LBISTArchitect Test Vector Generator, generate an investigation pattern set for the BIST controller. This pattern set tests the MISR signature after each pattern.
- **Run Per-Pattern BIST Test on ATE** — Run the investigation pattern set against the BIST controller device on the ATE. Save the failure log and reformat the information in the log so LBISTArchitect can map the failing cycles into failing BIST patterns numbers.
- **Map Failures and Create Unload Patterns** — Using LBISTArchitect, input the formatted MISR failure log, design netlist, and investigation pattern set. This produces test vectors that unload the scan chain contents for the failing patterns.

Note



Typically, 10 to 20 failing patterns are sufficient to diagnose the failure site or failure sites.

- **Run Failing Pattern Unload on ATE** — Run the unload pattern list on the ATE. The failure file contains sufficient information for detailed diagnosis.
- **Map Failing Unload Patterns to BIST Test Patterns** — Using LBISTArchitect and the failure file, map the failing cycles from the ATE to failing scan cells in the BIST patterns. This information is stored in the failure file for diagnosis.
- **Run Fault Simulation in Diagnosis Mode** — Using [Tessent Diagnosis](#) and the failure file, run diagnosis either manually or by using a dofile the tool created during the BIST controller generation phase.

Before You Begin Diagnosis

Before performing diagnosis, you must have successfully taken your design through the following LBISTArchitect phases:

- Inserted your scan chains, performed X bounding, and inserted test points—see [“Preparing the Design for Logic BIST.”](#)
- Created and inserted the LBIST controller—see [“Generating the BIST Controller.”](#)
- Performed both design and core fault simulation, and obtained a good machine fault signature—see [“Performing Fault Simulation.”](#)

The Diagnostic Process

The diagnostic process is a method for identifying the potential fault sites using the BIST hardware. During this process, you create test patterns using the LBISTArchitect Test Vector Generator (tvgen) and then exercise the BIST controller on the ATE using these patterns.

The following figure shows the steps in the process, and [Table 6-1](#) describes each step of the process.

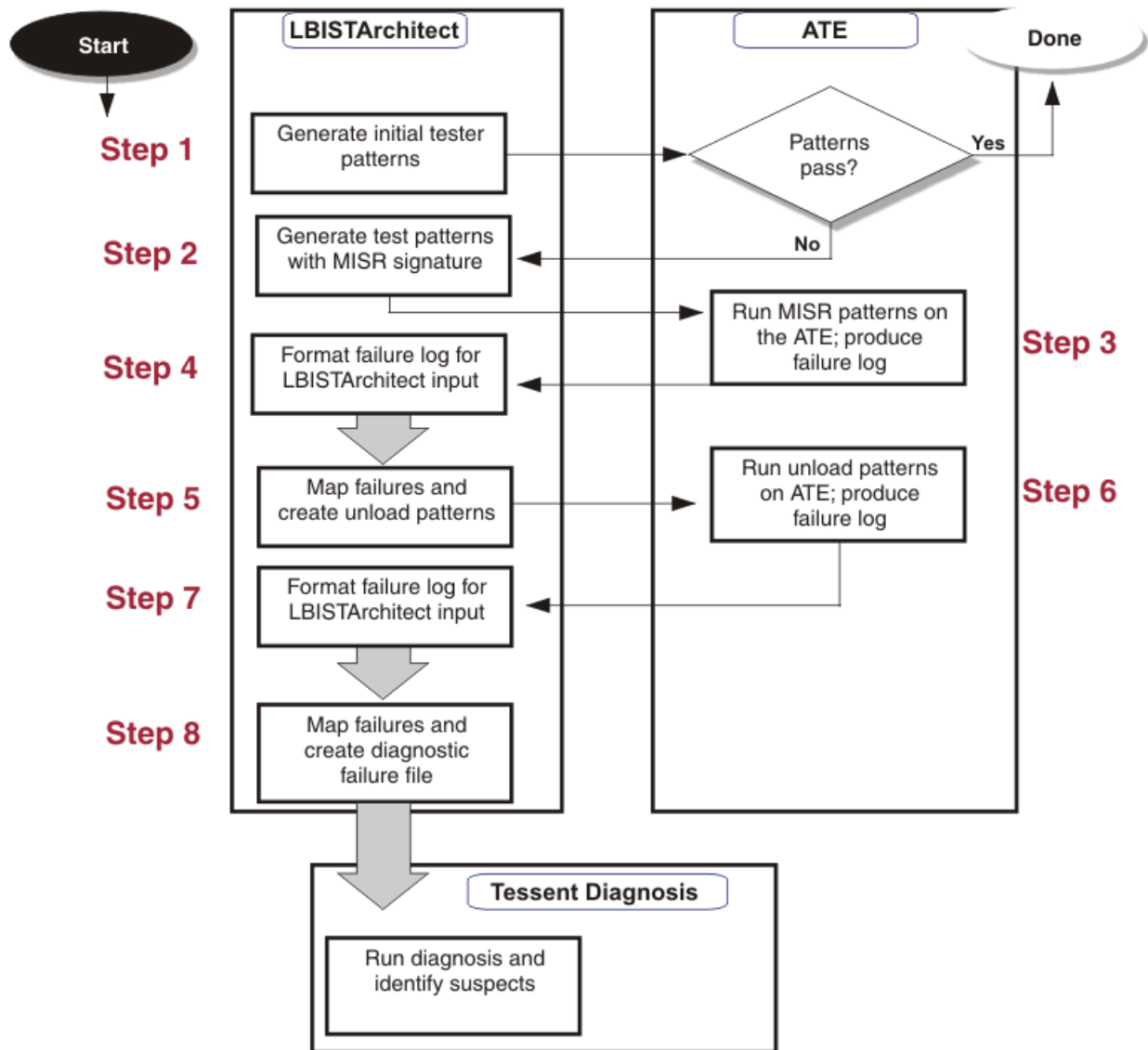


Table 6-1. Diagnostic Process

Step	Description
1	Create the tester patterns and run using the BIST controller on the ATE—see “ Step 1 — Generate Basic BIST Test Patterns and Test on the ATE ” and “ Run Basic BIST Test on the ATE .”
2	Generate an investigation pattern set for the BIST controller—see “ Step 2 — Generate Failure Investigation Patterns .”

Table 6-1. Diagnostic Process

Step	Description
3	Run the investigation patterns with the BIST controller on the ATE and obtain the failure file—see “ Step 3 — Run Investigation Patterns on the ATE. ”
4	Put the failure file in a format LBISTArchitect reads—see “ Step 4 — Format the ATE Failure File. ”
5	Correlate the failures to specific patterns—see “ Step 5 — Map Failures and Create Unload Patterns. ”
6	Unload the patterns from the device on the ATE—see “ Step 6 — Run Unload Patterns on the ATE. ”
7	Put the final unload failure file in a format LBISTArchitect reads—see “ Step 7 — Format the Final ATE Failure File. ”
8	Map the final failure file and generate an input file for diagnosis—see “ Step 8 — Map the Failing Patterns and Creating Diagnostic File. ”
	Input the diagnostic file and perform diagnosis—see “ Running Manufacturing Diagnosis. ”

Diagnosis Process Inputs

Table 6-2 identifies and describes the input files you must have for the diagnosis process.

Table 6-2. Inputs for the Diagnosis Process

Input	LBISTArchitect Phase
The RTL-level netlist from the BIST controller generation phase (<i>design_bist.v</i>) and configuration file (<i>design_bist.v.lbcfg</i>)	BIST Controller Generation—see “ Examining BIST Controller Generation ”
Internal BIST test patterns, normally <i>entity_bist.pat.gz</i>	Fault Simulation—see “ Fault Simulation Process Outputs ”
LBISTArchitect Diagnostic dofile, normally <i>entity_diagnose.do</i> —see “ Running Manufacturing Diagnosis ” for when you use this file	BIST Controller Generation

The diagnostic process produces intermediate netlists and data files you subsequently use for input for the next step of the diagnostic process.

Diagnosis Process Outputs

At the conclusion of a successful diagnosis process, LBISTArchitect provides Tessent Diagnosis with a manageable list of failures you use during diagnosis. For more information, see “[Running Manufacturing Diagnosis.](#)”

Performing Diagnosis

Use this procedure to prepare BIST-specific failure data for input into [Tessent Diagnosis](#) for diagnosis and creating a diagnostic report for examination.

Using a TAP Controller and Interconnects

If you are using a TAP controller and interconnects, then you must specify a BSDL file during LBISTArchitect Test Vector invocation. This forces the single scan chain behavior. For specific invocation options, see the [lbistarchitect](#) command reference in the *LBISTArchitect Reference Manual*.

For BSDL specifics, consult the following sections of the BSDArchitect documentation:

- [Load Testdata](#) in the *BSDArchitect Reference Manual*.
- “[LBISTArchitect Diagnostics Test Sequence File Meta Tags](#)” in the *BSDArchitect Reference Manual*.

Step 1 — Generate Basic BIST Test Patterns and Test on the ATE

Create the Patterns

From a Linux/UNIX shell, invoke LBISTArchitect for test vector generation using the following command:

```
<Tessent_Tree_Path>/bin/lbistarchitect -tvgen test_vector_generation_arguments
```

Use the *test_vector_generation_arguments* for specifying the following:

- **Design Netlist** — The RTL-level netlist from the BIST controller generation phase (*design_bist.v*) and configuration file (*design_bist.v.lbcfg*).
- **LSFR Trace File** — The LSFR trace file you created with LBISTArchitect during the fault simulation process.

Example 6-1. Step 1 — Basic BIST Test Creation Example Invocation

```
lbistarchitect -tvgen design_bist.v -trace design_lfsr.trace.gz design.wgl -rep
```

Invocation Explanation



lbistarchitect -tvgen	→	invokes the test vector generation mode
design_bist.v	→	loads the RTL-level netlist <i>design_bist.v</i>
-trace design_lfsr.trace.gz	→	specifies and loads the LSFR trace file <i>design_lfsr.trace.gz</i>
design.wgl -rep	→	specifies name of WGL pattern set for use on the ATE

Run Basic BIST Test on the ATE

After you have created the test patterns, exercise the BIST core on the ATE using the patterns by running the Basic BIST Test. If the Basic BIST Test passes, then do not run diagnosis. If, however, the Basic BIST Test fails, then proceed with [Step 2 — Generate Failure Investigation Patterns](#).

Step 2 — Generate Failure Investigation Patterns

If the Basic BIST Test has failed on the ATE. This step of the process creates a pattern failure investigation vector set for subsequent use on the ATE with the failing device. This vector set checks the MISR for each BIST pattern.

From a Linux/UNIX shell, invoke LBISTArchitect for test vector generation using the following command:

```
<Tessent_Tree_Path>/bin/lbistarchitect -tvgen -check 1 \  
test_vector_generation_arguments
```

Use the *test_vector_generation_arguments* for specifying the following:

- **Design Netlist** — The RTL-level netlist from the BIST controller generation phase (*design_bist.v*) and configuration file (*design_bist.v.lbcfg*).

Example 6-2. Step 2 — Failure Investigation Pattern Creation Example Invocation

```
lbistarchitect -tvgen -check 1 design_bist.v design_check.wgl -rep
```

Invocation Explanation



lbistarchitect -tvgen -check1	→	invokes the test vector generation mode -check 1 creates a MISR check for each pattern
design_bist.v	→	loads the RTL-level netlist <i>design_bist.v</i>
design_check.wgl -rep	→	specifies name of investigation WGL pattern set for use on the ATE

Step 3 — Run Investigation Patterns on the ATE

After you have created the failure investigation patterns, you re-exercise the BIST core on the ATE using the patterns. Upon running these patterns on the ATE with the failing device, the ATE produces a failure file, which you reformat and use for input to LBISTArchitect for mapping the failures to patterns.

Step 4 — Format the ATE Failure File

After you have obtained a failure file from the ATE, reformat the failure file into a format LBISTArchitect reads. Use the examples in [Example 6-3](#) and [Example 6-4](#) as a guide for reformatting your ATE failure files.

[Example 6-3](#) shows the format you must use for a failing time-based BIST pattern file.

Example 6-3. Time-Based Failure File Format Example

The time can be in ps, ns, us, or ms.

```
%MISR_FAIL
# this is a comment list
# <time>      <output>
1020ns      bist_data_sout
2001ns      bist_data_sout
2020010ps  bist_data_sount
```

[Example 6-4](#) shows the format you must use for a failing cycle-based BIST pattern file.

Example 6-4. Cycle-Based Failure File Format Example

```
%MISR_FAIL
# this is a comment list
# <time>      <output>
cycle 100  sout_1
cycle 150  sout_2
cycle 300  shared_pin[0]
```

Step 5 — Map Failures and Create Unload Patterns

After you have formatted the ATE's raw failure file, do the following:

- “[Map the ATE Failures](#)”
- “[Unload the Failing Patterns](#)”

Note



You perform the failure *mapping operation* (-failuremap) two times: first for the initial mapping of the MISR failures (this step); and second once you have unloaded the failing patterns (see “[Step 8 — Map the Failing Patterns and Creating Diagnostic File](#)”).

Map the ATE Failures

From a Linux/UNIX shell, invoke LBISTArchitect for failure mapping using the following command:

```
<Tessent_Tree_Path>/bin/lbistarchitect -failuremap failure_mapping_arguments
```

Use the *failure_mapping_arguments* for specifying the following:

- **Design Netlist** — The RTL-level netlist from the BIST controller generation phase (*design_bist.v*) and configuration file (*design_bist.v.lbcfg*).
- **Investigation Patterns** — The patterns you created during “[Step 1 — Generate Basic BIST Test Patterns and Test on the ATE](#)” and subsequently ran on the ATE.
- **Reformatted ATE Failure File** — The ATE failure file you reformatted in “[Step 4 — Format the ATE Failure File](#)”.

Example 6-5. Step 5 — Map Failures Example Invocation

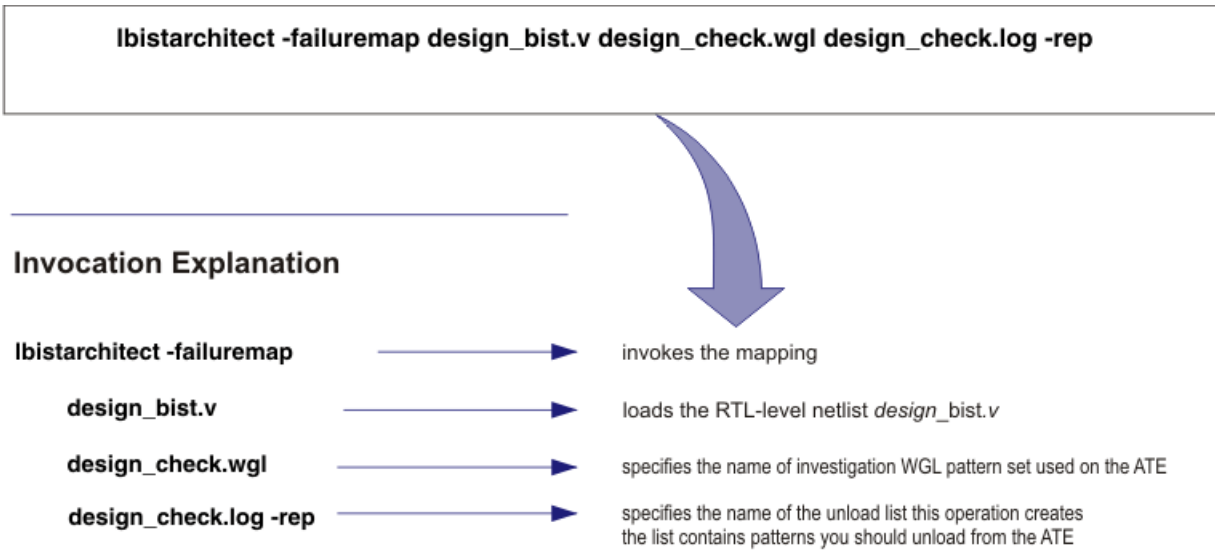


Figure 6-1 shows an example of the output from the -failuremap operation.

Figure 6-1. Mapped Failures

```
%PATTERN_UNLOAD
# bist pattern number    intent
1                        pass MTPI phase 0
2                        pass MTPI phase 0
3                        pass MTPI phase 0
4                        pass MTPI phase 0
1537                     fail
1538                     pass MTPI phase 3
1539                     fail
1540                     fail
1541                     fail
1542                     fail
1544                     fail
1545                     fail
```

Unload the Failing Patterns

From a Linux/UNIX shell, invoke LBISTArchitect for unloading the patterns using the following command:

```
<Tessent_Tree_Path>/bin/lbistarchitect -failuremap pattern_unloading_arguments
```

Use the *pattern_unloading_arguments* for specifying the following:

- **Design Netlist** — The RTL-level netlist from the BIST controller generation phase (*design_bist.v*) and configuration file (*design_bist.v.lbcfg*).
- **BIST Pattern File** — The BIST pattern file (normally, *entity_bist.pat.gz*) you created during fault simulation. See “[Fault Simulation Process Outputs](#)”.

- **LSFR Trace File** — The LSFR trace file you created with LBISTArchitect during the fault simulation process.
- **Failure Map List** — The mapped failure list you created with LBISTArchitect during the “[Map the ATE Failures](#)” step. You specify the file name in conjunction with the -funload invocation switch.

Example 6-6. Step 5 — Pattern Unload Example Invocation

```
lbistarchitect -tvgen design_bist.v \  
design_bist.pat.gz \  
-trace design_lfsr.trace.gz \  
-funload design_check.list \  
design_unload.wgl -rep
```

Invocation Explanation



lbistarchitect -tvgen	invokes the test vector generation mode
design_bist.v	loads the RTL-level netlist <i>design_bist.v</i>
design_bist.pat.gz	loads the BIST patterns from fault simulation
-trace design_lfsr.trace.gz	specifies and loads the LSFR trace file <i>design_lfsr.trace.gz</i>
-funload design_check.list	specifies the name of the unload list this operation creates the list contains patterns you should unload from the ATE
design_unload.wgl -rep	specifies name of WGL pattern set for use on the ATE

Step 6 — Run Unload Patterns on the ATE

After you have created these patterns, exercise the BIST core on the ATE. The ATE produces a failure file, however this failure file contains sufficient information for detailed diagnosis.

Step 7 — Format the Final ATE Failure File

After you have a final failure file containing the unload patterns from the ATE, reformat the failure file into a format LBISTArchitect reads. Use the examples in [Example 6-7](#) and [Example 6-7](#) as a guide for reformatting your ATE failure files.

Example 6-7 shows the format of the failing time-based BIST pattern unload file.

Example 6-7. Time-Based Unload Format Example

```
%PATTERN_FAIL
# this is a comment list
# <time> <output> <expected_value> <actual_value>
    1020ns sout_1          H          L
    2001ns sout_2          L          H
  2020010ps shared_pin[0]  L          H
```

Example 6-8 shows the format of the failing cycle-based BIST pattern unload file.

Example 6-8. Cycle-Based Unload Format Example

```
%PATTERN_FAIL
# this is a comment list
# <cycle>    <output> <expected_value> <actual_value>
  cycle_102  sout_1    H          L
  cycle_200  sout_2    L          H
  cycle_202  shared_pin[0] L          H
```

Step 8 — Map the Failing Patterns and Creating Diagnostic File

After you have unloaded the failing patterns from the ATE and reformatted the failure file to a format LBISTArchitect reads, map the failing unloaded patterns and generate the fault simulation failure map.

[Example 6-9](#) shows an excerpt from an example diagnostic failure map you create during this step. Input this file into LBISTArchitect during diagnostic fault simulation—see [“Running Manufacturing Diagnosis.”](#)

Example 6-9. Diagnostic Failure Map Example

```
// For diagnosis the last pattern is 1591
//
// pattern number      chain          bit
1536                  chain21         11 L H
1536                  chain22         14 H L
1538                  chain21         11 L H
1538                  chain22         14 L H
1539                  chain14         14 H L
...
1590                  chain12         13 H L
1590                  chain13         1 L H
1590                  chain13         3 L H
1590                  chain17         6 L H
1590                  chain17         14 L H
1590                  chain21         11 L H
1590                  chain22         14 L H
1591                  chain12         14 L H
1591                  chain14         14 L H
1591                  chain22         14 L H
last_pattern_tested 1591
```

From a Linux/UNIX shell, invoke LBISTArchitect for failure mapping using the following command:

```
<Tessent_Tree_Path>/bin/lbistarchitect -failuremap failure_mapping_arguments
```

Use the *failure_mapping_arguments* for specifying the following:

- **Design Netlist** — The RTL-level netlist from the BIST controller generation phase and configuration file (*design_name.v.lbcfg*).
- **Unload Patterns** — The patterns you created for unload during [“Step 6 — Run Unload Patterns on the ATE”](#) and subsequently ran on the ATE.
- **Reformatted ATE Failure File** — The ATE failure file you reformatted in [“Step 7 — Format the Final ATE Failure File”](#).

Example 6-10. Failure File Mapping and Diagnostic File Creation Invocation Example

```
lbistarchitect -failuremap design_bist.v \  
design_unload.wgl \  
design_unload_failure.log \  
unload_failures.diag -rep
```



Invocation Explanation

lbistarchitect -tvgen	→	invokes the test vector generation mode
design_bist.v	→	loads the RTL-level netlist <i>design_bist.v</i>
design_unload.wgl	→	specifies name of the unload WGL pattern set you used on the ATE
design_unload_failure.log	→	loads the formatted failure file with the ATE unload failures
unload_failures.diag -rep	→	specifies the name of the fault simulation diagnosis file this step creates

Running Manufacturing Diagnosis

You perform the diagnosis using the Tessent Diagnosis tool with the following input files you created using the LBISTArchitect tool:

- **Diagnosis Failure File** — You created this file in “[Step 8 — Map the Failing Patterns and Creating Diagnostic File](#)”.
- **BIST Pattern File** — You created this file during fault simulation—see “[Fault Simulation Process Outputs](#)”—normally, *entity_bist.pat.gz*.

For more information on troubleshooting the diagnosis and interpreting the results using Tessent Diagnosis, see the *Tessent Diagnosis User’s Guide*.

Invoking Tessent Diagnosis

Invoking Tessent Diagnosis requires the pathname to the [flattened design netlist](#).

From a Linux/UNIX shell, enter:

```
<Tessent_Tree_Path>/bin/tessent -diagnosis flat_model -logfile logfile_name
```

- **-diagnosis** — A switch that invokes Tessent Diagnosis in stand-alone mode.
- **flat_model** — The pathname to and name of the flattened design netlist.
- **-logfile logfile_name** — The switch and user-specified filename necessary to generate a logfile of the diagnostic session. The logfile contains all the commands and error messages issued during the session and should be generated to allow troubleshooting.

Tessent Diagnosis invokes, loads the flattened design netlist, and displays the command-line user interface. For more information on invoking Tessent Diagnosis, see the [tessent](#) shell command in the *Scan Insertion, ATPG, and Diagnosis Reference Manual*.

Manual Verses Automatic Invocation

Like most Mentor Graphics DFT tools, you can input a dofile containing commands into Tessent Diagnosis. During BIST Controller generation, LBISTArchitect creates a skeleton diagnostic dofile (normally, *design_diagnose.do*) you can use with Tessent Diagnosis.

Figure 6-2 shows a representative dofile. You use this dofile “as is”, or modify the file. See [“Generating the BIST Controller.”](#)

Figure 6-2. Sample Diagnosis Dofile

```

////////////////////////////////////
//      Copyright 2001-2007 Mentor Graphics Corporation
//
//      All Rights Reserved.
//
//      THIS WORK CONTAINS TRADE SECRET AND PROPRIETARY
//      INFORMATION WHICH IS THE PROPERTY OF MENTOR GRAPHICS
//      CORPORATION OR ITS LICENSORS AND IS SUBJECT TO LICENSE TERMS
//
//
// File Type:      diagnostic fault simulation dofile
// Date Created: Mon Jun 18 14:13:34 2007
// Tool Version: LBISTArchitect v8.2007_3.10
//
////////////////////////////////////

// NOTE: In order to use this file, you must start yiedassist with the
//      flat model that was saved during the core level fault simulation phase
set pattern source external design_bist.pat.gz -ascii
set sys mode good
diagnose failures unload_failures.diag

```


There are several ways to get help when setting up and using Tessent software tools. Depending on your need, help is available from documentation, online command help, and Mentor Graphics Support.

Documentation

The Tessent software tree includes a complete set of documentation and help files in PDF format. Although you can view this documentation with any PDF reader, if you are viewing documentation on a Linux file server, you must use only Adobe® Reader® versions 8 or 9, and you must set one of these versions as the default using the MGC_PDF_READER variable in your *mgc_doc_options.ini* file.

For more information, refer to “[Specifying Documentation System Defaults](#)” in the *Managing Mentor Graphics Tessent Software* manual.

You can download a free copy of the latest Adobe Reader from this location:

<http://get.adobe.com/reader>

You can access the documentation in the following ways:

- **Shell Command** — On Linux platforms, enter **mgcdocs** at the shell prompt or invoke a Tessent tool with the -Manual invocation switch. This option is available only with Tessent Shell and the following classic point tools: Tessent FastScan, Tessent TestKompress, Tessent Diagnosis, and DFTAdvisor.
- **File System** — Access the Tessent bookcase directly from your file system, without invoking a Tessent tool. From your product installation, invoke Adobe Reader on the following file:

```
$MGC_DFT/doc/pdfdocs/_bk_tessent.pdf
```

- **Application Online Help** — You can get contextual online help within most Tessent tools by using the “help -manual” tool command:

```
> help dofile -manual
```

This command opens the appropriate reference manual at the “dofile” command description.

Mentor Graphics Support

Mentor Graphics software support includes software enhancements, access to comprehensive online services with SupportNet, and the optional On-Site Mentoring service.

For details, refer to this page:

<http://supportnet.mentor.com/about>

If you have questions about a software release, you can log in to SupportNet and search thousands of technical solutions, view documentation, or open a Service Request online:

<http://supportnet.mentor.com>

If your site is under current support and you do not have a SupportNet login, you can register for SupportNet by filling out a short form here:

<http://supportnet.mentor.com/user/register.cfm>

All customer support contact information is available here:

<http://supportnet.mentor.com/contacts/supportcenters/index.cfm>

Appendix B

Logic BIST Controller

Data Sheet

This data sheet describes the logic BIST controller RTL code the LBISTArchitect tool generates, and contains the following sections:

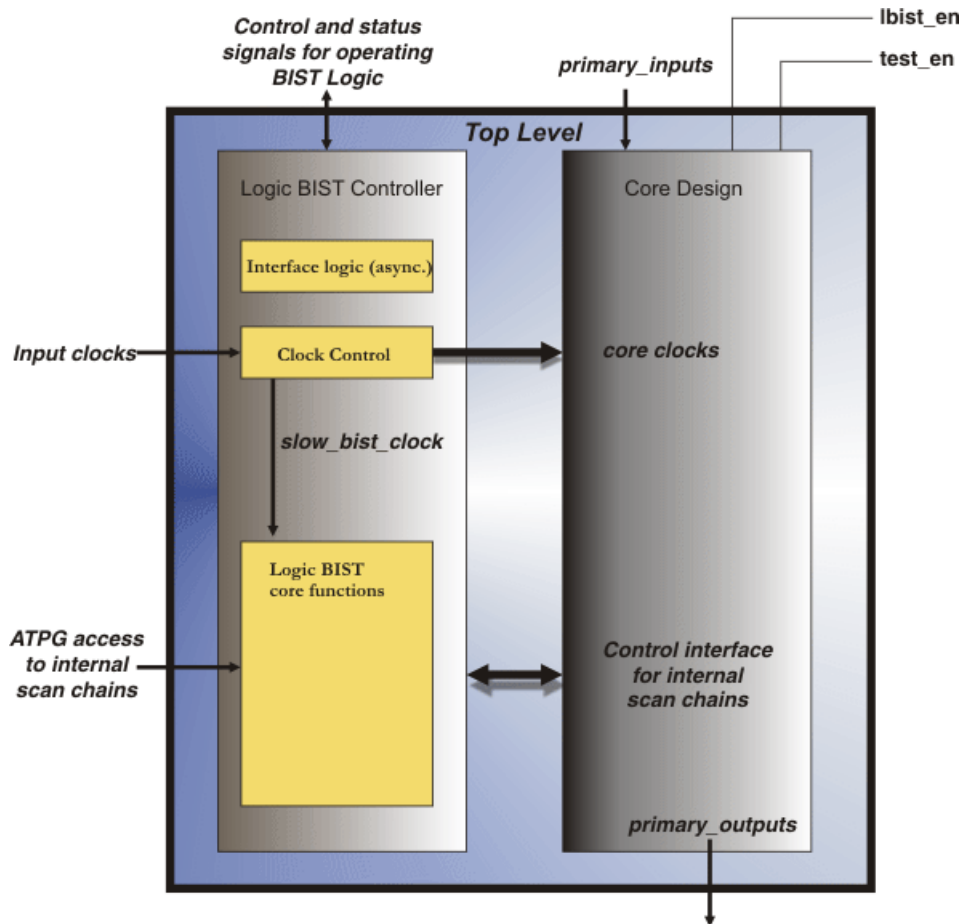
- [“Architectural Overview”](#)
- [“Multi-Phase Test Point Insertion \(MTPI\)”](#)
- [“Lockup Latch Timing During Shifting”](#)
- [“BIST Controller Interface Description”](#)
- [“Timing Between ATE and BIST Controller”](#)
- [“Timing Between BIST Controller and Core”](#)
- [“Brief Description of Sub-blocks”](#)

i **Tip:** For generation-oriented logic BIST controller information, see [“Generating the BIST Controller”](#).

Architectural Overview

[Figure B-1](#) shows the basic data paths between the logic BIST controller and the core. LBISTArchitect creates an output netlist in Verilog format. This netlist contains a top-level module that instantiates the core design and the logic BIST controller along with all the appropriate interconnects. The netlist also includes the synthesizable RTL description of the logic BIST controller.

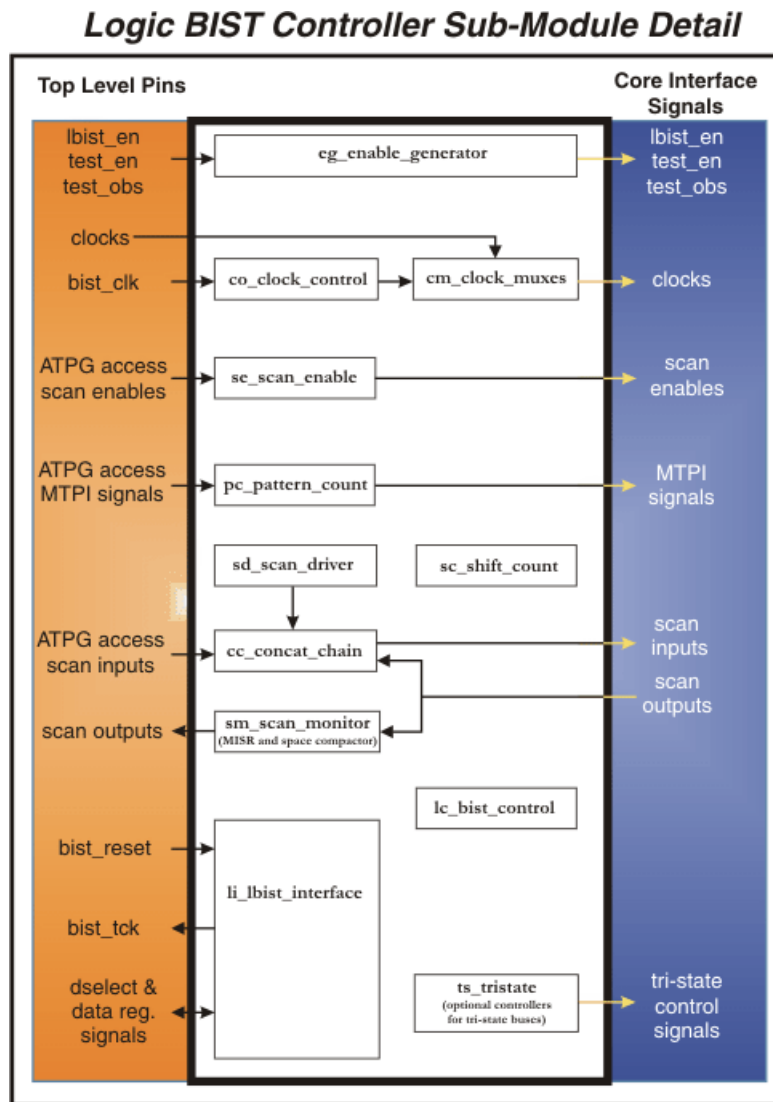
Figure B-1. Logic BIST Controller to Core Data Paths



Sub-Module Detail

Figure B-2 illustrates the basic design of the logic BIST controller.

Figure B-2. Logic BIST Controller



- **Primary Inputs** — Consist of all non-clock and non-scan signals going into the core design.
- **Primary Outputs** — Consist of all non-scan outputs from the core design. Primary inputs pass directly from the top level into the core.

General Operation

The `sd_scan_driver` module generates pseudorandom patterns using a Linear Feedback Shift Register (LFSR) and a network of XOR gates. These patterns load into the design through the core-design scan chains.

Each unique pattern is serially shifted into the design. During this time, the flip-flops in the core design operate in “scan” mode. The *shift counter* keeps track of the number of shift cycles to issue for each pattern. The control logic (in `lclbist_control`) also uses the shift counter to separate the shift cycles from the capture cycle. Once a pattern loads into the core scan chains, the BIST controller generates a single capture cycle. During this capture cycle, the flip-flops operate in “normal” mode. Running through all the patterns tests the operation of the core circuits.

The *pattern counter*, consisting of a start value, an end value, and a MISR control bit, keeps track of how many patterns have been driven through the core. The counter is loaded with a start value and counts up until it reaches a programmable end point. The test stops when the pattern counter reaches the programmed end value. The MISR control bit puts the MISR into a hold condition during loading of the first pattern. This prevents the uninitialized values that are in the scan chains prior to loading the first pattern from corrupting the MISR signature.

The `sm_scan_monitor` module uses another LFSR and XOR-gate network to monitor the captured values that shift out of the scan chains when the test is running. The scan monitor generates a unique signature based on those scan chain values.

For a more information on the various sub-blocks, see the section, “[Brief Description of Sub-blocks](#)” on page 146.

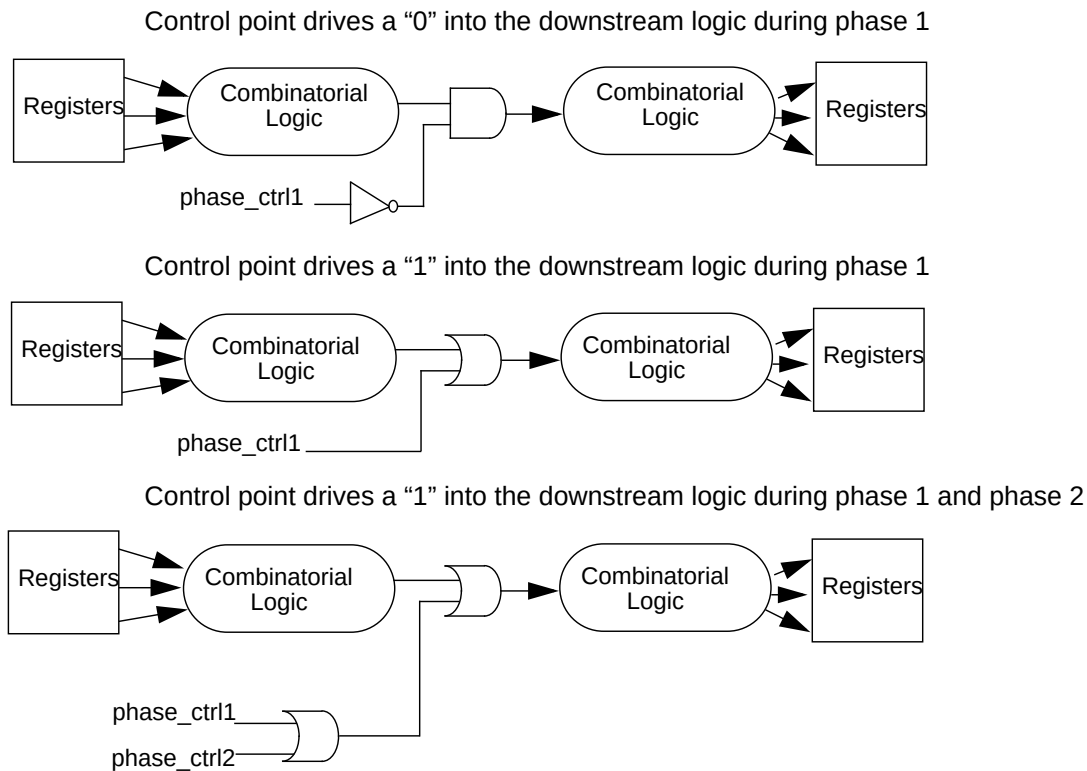
Multi-Phase Test Point Insertion (MTPI)

The BIST-Ready phase includes special algorithms that identify control points using a unique (and patented) approach that divides the test process into a number of phases (typically 4) and inserts test points into the design for each phase, except phase 0. Test points are targeted to be “active” during a given subset of phases. The Mentor Graphics approach to test point insertion is unique because test control points are not driven by pseudorandom data. All control points inserted into a phase are simultaneously activated using a single enable signal.

Control points meant to drive a high into downstream logic require the addition of a single OR gate into the design. Control points driving a low into downstream logic consist of a single AND gate. You can drive the control gate directly with a global enable signal (one per phase) if it is activated during only a single phase. Signals that are active during multiple phases require additional gates in order to merge the phase enable signals together.

[Figure B-3](#) shows several different types of control points.

Figure B-3. Control Point Gating



The logic BIST controller generates one control signal for each phase. LBISTArchitect inverts these signals inside the core when driving AND-type control points. It also merges multiple phase control signals together inside the core when a control point is active during more than one phase.

The phase control signals from the logic BIST controller are combined with the global “test_en” signal to guarantee that all control points are deactivated (in pass-through mode) during normal circuit operation. The phase control signals are decoded from the two bits of the pattern counter that are directly to the left of the most significant bit of the capture window decoder; the capture window decoder uses the least significant bits of the pattern counter. The number of phases and the number of patterns must be powers of two when using MTPI. This produces an even distribution of patterns between MTPI phases, while minimizing the amount of logic required in the controller.

Lockup Latch Timing During Shifting

This section discusses the BIST controller timing issues that occur while the controller is shifting data through the design in BIST mode, and also when an ATE or test bench accesses the concatenated scan chains.

During BIST Mode

The logic BIST controller must load and unload data from the internal scan chains during testing. The BIST controller generates the core clocks, but they feed into separate clock trees, so we must assume that some skew exists between an edge transition of the BIST clock and edge transitions of the core clocks. Latches are optionally inserted (on a per scan chain basis) to ensure that each scan_driver output changes on the opposite edge of the BIST clock relative to the capture transition for the core clock. If the first scan cell in the chain captures data on the positive edge of the core clock (roughly in line with the positive edge of the BIST clock), then a latch is inserted to ensure that the BIST controller output doesn't change until after the negative edge of the BIST clock.

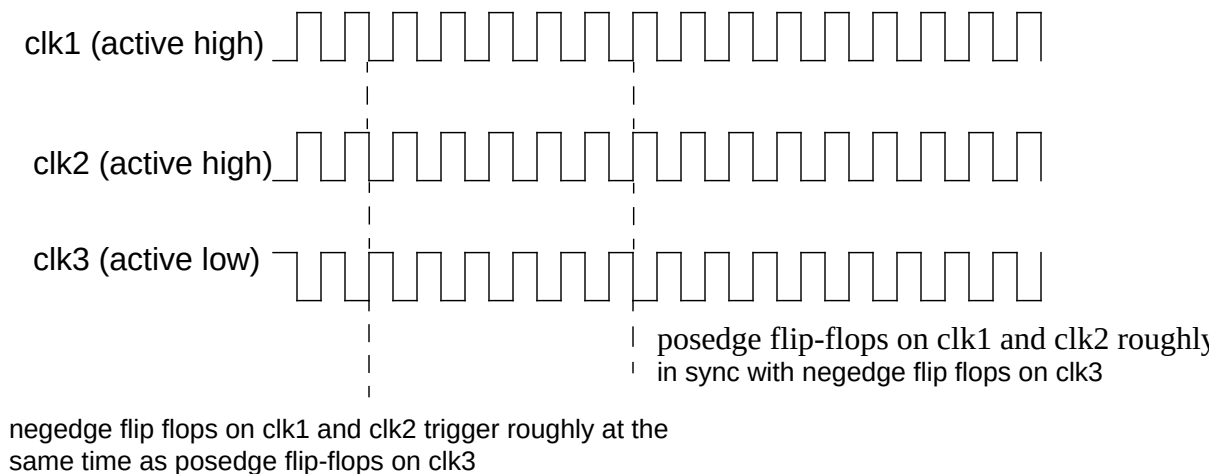
Similar problems exist for the scan monitor inputs. Latches are used to guarantee 1/2 cycle skew tolerance between the scan chain outputs and the MISR inputs.

During Scan Chain Concatenation

Clock skew between the various core clock signals also causes problems during scan chain concatenation. The BIST controller automatically inserts optional lockup latches to ensure roughly 1/2 cycle of skew tolerance when shifting data through the merged scan chains.

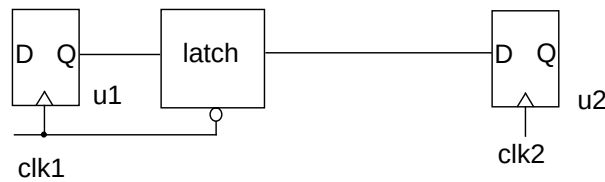
Latch insertion must be based on knowledge of the edge sensitization of the last flip-flop in the first chain and the first flip-flop in the second chain. The initial phase relationship is also important. The LBISTArchitect latch insertion algorithm assumes that active low clocks and active high clocks are roughly 1/2 cycle out of phase. [Figure B-4](#) shows this relationship.

Figure B-4. Clock Phase Relationships



[Figure B-5](#) shows the wiring for the latches.

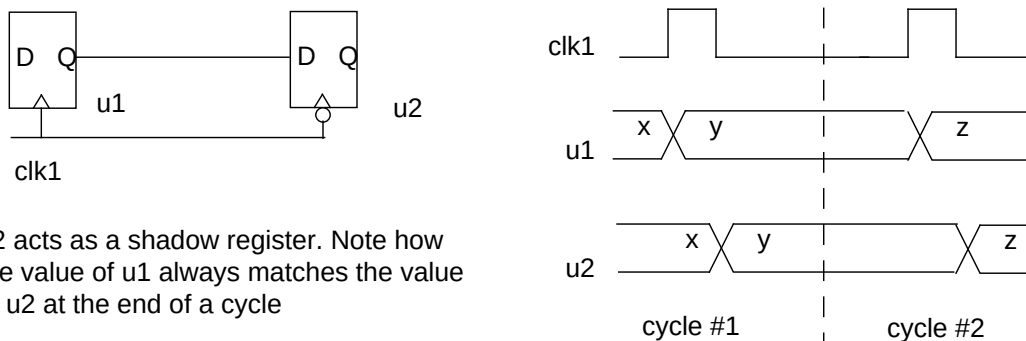
Figure B-5. Out-of-Phase Clock Wiring



The latch is enabled using an inverted version of clk1. This delays the output of u1 by approximately 1/2 clock cycle.

One final problem that must be addressed when stitching together arbitrary scan chains is the creation of shadow registers. This happens when the two flip-flops change on opposing clock edges. LBISTArchitect assumes that a clock cycle is either low-high-low, or high-low-high. For low-high-low clock transitions, a negative edge flip-flop driven by a positive edge flip-flop will result in two flip-flops that always share the same value (*shadow registers*). For high-low-high clock transitions, a positive edge flip-flop followed by a negative edge flip-flop will result in shadow registers. Figure B-6 shows a shadow register and the clock cycle transitions.

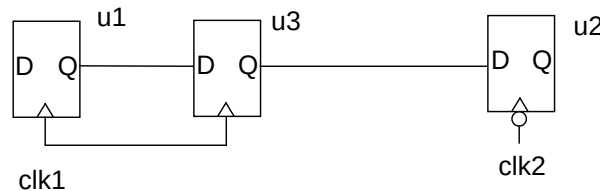
Figure B-6. Shadow Registers



u2 acts as a shadow register. Note how the value of u1 always matches the value of u2 at the end of a cycle

Shadow registers are undesirable in some circumstances because the loss of controllability may mask some faults. This problem is easily avoided by adding another flip-flop between u1 and u2. Figure B-7 shows the new register now acting as the shadow register restoring full controllability of u1 and u2. Aliasing in the newly inserted register is not a problem because it does not control any logic when the device is in functional mode.

Figure B-7. Shadow Registers and Controllability



u3 was added onto the end of chain #1 and functions as a shadow register, allowing proper control of u1 and u2.

BIST Controller Interface Description

The following table describes the signals that are used to control the basic operation of the BIST controller.

Table B-1. Basic Logic BIST Controller Interface

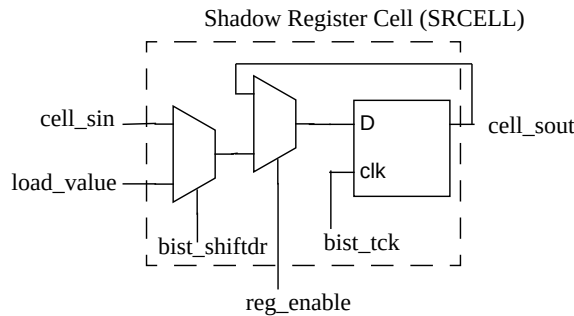
Signal Name	Mode	Description
bist_clk	Input	Clock signal from core design which drives the BIST controller (this can be controlled by the user for multi-clock designs).
bist_reset	Input	Synchronous or asynchronous reset for the BIST controller .
bist_tck	Input	Clock signal that defines timing of interfacing signals.
bist_shiftdr	Input	Switch between shifting data through the registers and loading the initial value that will be shifted out.
bist_dselect_enable	Input	Serially load/unload dselect register when enable is high.
bist_dselect_updatedr	Input	Serial loading is complete - transfer shift register into internal dselect register.
bist_dselect_sin	Input	Serial input pin for loading dselect register.
bist_dselect_sout	Output	Serial output pin for unloading dselect register.
bist_data_enable	Input	Serially load/unload data register when enable is high.
bist_data_updatedr	Input	Serial loading is complete - transfer shift register into selected internal data register.
bist_data_sin	Input	Serial input pin for loading data register.
bist_data_sout	Output	Serial output pin for unloading data register.
bist_diag_clk	Input	Special clock input - used during diagnostics to unload scan chain contents at end of logic BIST test.

Overview of Dual-Register Interface

The interface between the Logic BIST controller, and the external world (ATE, 1149.1 TAP, etc.) consists of two registers (data and control) that are driven by an asynchronous clock (bist_tck). Each bit in the register is implemented with a flip-flop and two muxes.

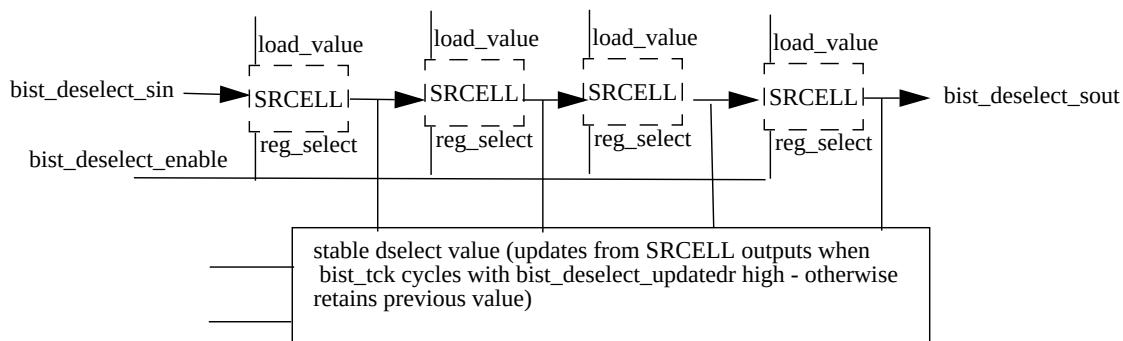
The first mux is used to keep the data from shifting through the register when it is not enabled. The second mux switches between serially shifting data through the register (`bist_shiftdr = 1`) or loading the cell from an external source. This basic building block is shown in [Figure B-8](#) and is referred to as a shadow register cell (SRCELL).

Figure B-8. Contents of SRCELL Block



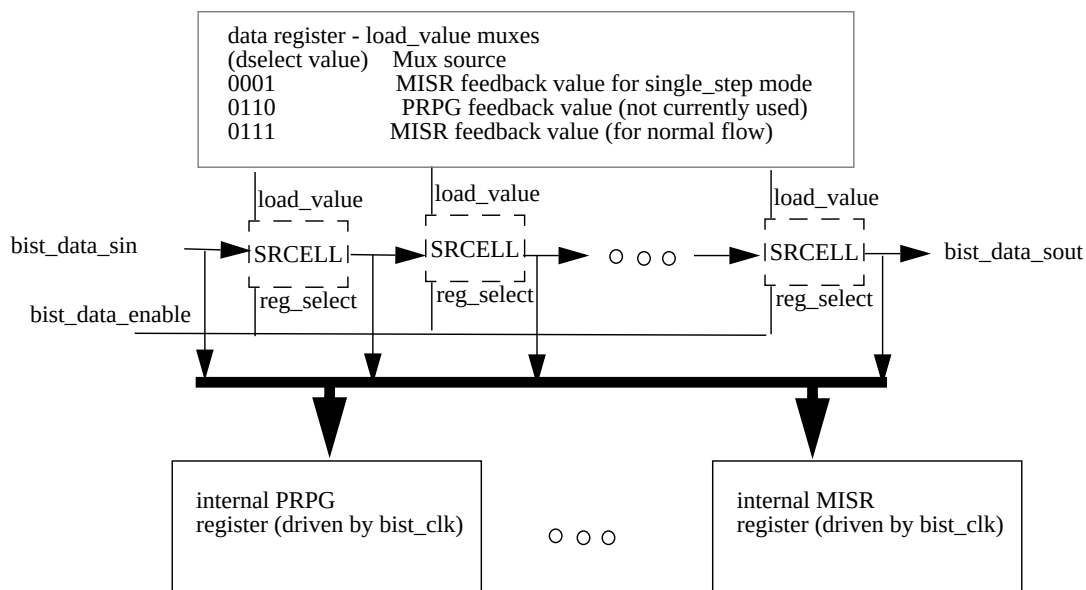
The dselect register is always implemented as a four bit register. It is constructed using four SRCELL blocks with the `cell_sout` signal from one block feeding the `cell_sin` signal of the next block. The `load_value` inputs always drive in a value of “1101” since this register is really intended to serve only as control register - not a status register. Note that each `cell_sout` value also feeds an extra flip-flop that loads the value only when `bist_dselect_updatedr` is high. The output register formed by these bits does not glitch when data is shifting through the SRCELL shadow register.

Figure B-9. Layout of dselect Register



The data register is also constructed from SRCELL blocks that are combined into a chain. The number of bits in the data register determines the maximum internal bist controller register than can be accessed at any one time using this technique. The MISR is typically the largest register in the design, and consequently often defines the size of the data register.

Figure B-10. Layout of Data Register



The cell_out value from each SRCELL block in the data register is referred to as the “raw_data_reg” value. This signal routes into many internal registers inside the logic BIST controller. The registers in the controller are driven by slow_bist_clock. A synchronization mechanism captures data into the internal controller registers only after the bist_data_updatedr signal is set high. This synchronization mechanism requires five slow_bist_clock cycles to complete the transfer. It is very important that the contents of the “data_reg_raw” remain unchanged during this time.

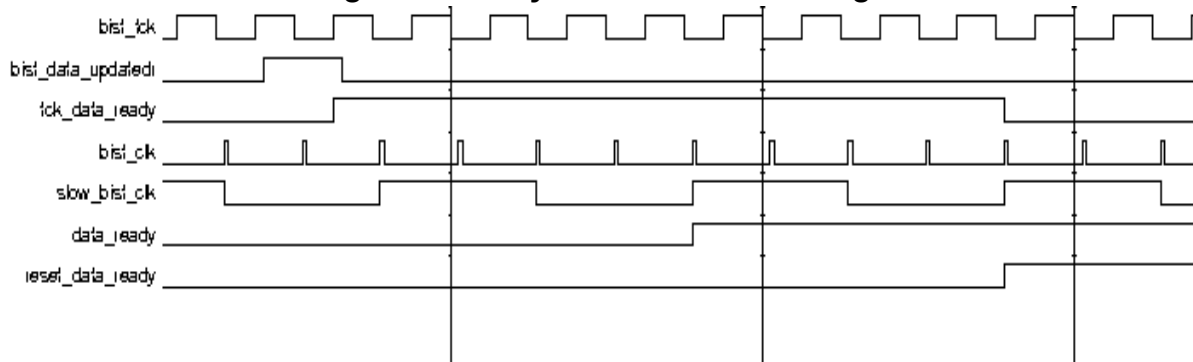
The following diagram shows the synchronization mechanism in action. Notice how the bist_data_updatedr signal is captured into tck_data_ready at the positive edge of tck. This synchronizer is clocked by slow_bist_clk which is running at 1/2 bist_clk rate. The internal “data_ready” signal is driven high after two slow_bist_clk positive edge transitions. The contents of the raw_data_reg register are transferred into the internal registers on all subsequent slow_bist_clk cycles where data_ready is high. It takes two cycles to reset the “tck_data_ready” signal, resulting in the requirement that raw_data_reg remain stable for five slow_bist_clk cycles.

Note



The previous diagrams only pulse bist_clk once for each tck cycle. This lets us use a single template in the WGL file during dselect and data register load operations. The timing for the testbench closely matches the timing in the WGL vectors.

Figure B-11. Synchronization Timing

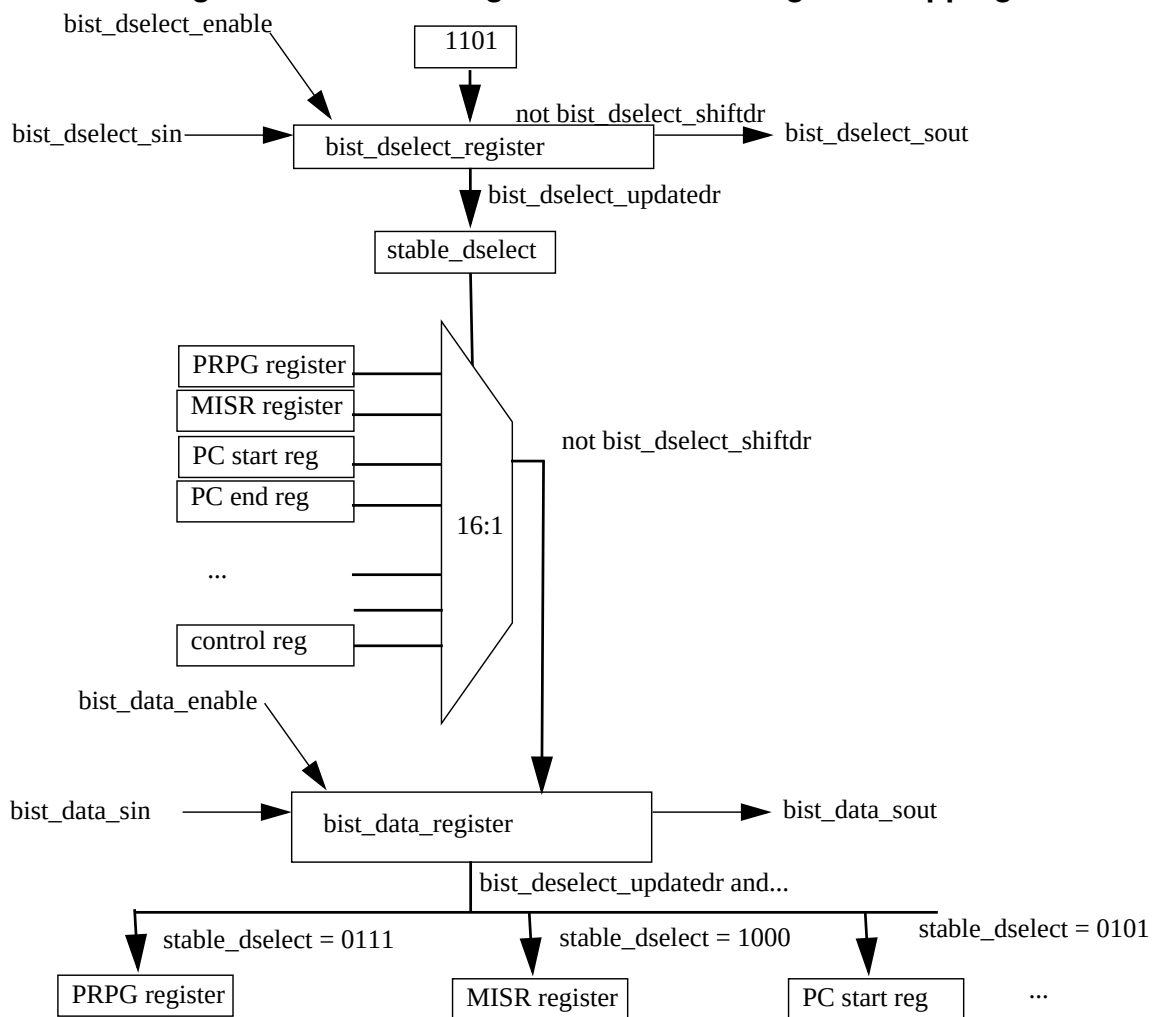


During normal operation, `bist_clk` (and even `slow_bist_clk`) typically run at much higher frequencies than `tck`, so the extra delay is not a serious issue. One should also note that the `dselect` register is independent of the data register, so one can shift a new value into the `dselect` register without altering the value in the “`raw_data_reg`” register as long as the “`bist_dselect_updatedr`” pulse does not occur before the data register has finished loading.

Decoding the Contents of the Data Register

As mentioned earlier, the contents of the data register change based on the current value loaded into the `dselect` register. This mapping is shown in [Figure B-12](#).

Figure B-12. Data_Register to Internal Register Mapping



NOTE: internal registers are driven by **slow_bist_clk**. Unloading values (such as the MISR) must be done only when the BIST controller is in an idle state (end of test). The internal registers are loaded after **bist_data_updatedr** goes high. Synchronization logic is used to avoid glitches.

The decoding of the bits in the dselect register is defined by the following table:

Table B-2. Decoding of dselect Register

deselect value	data register value
0000	Logic BIST de-activated
0001	RUN logic BIST
0010	scan enable control reg.
0011	map control register into data reg.
0100	unused
0101	map pattern count start value into data reg
0110	map pattern count end value into data reg.
0111	map PRPG into data reg.
1000	map MISR into data reg.
1001	map shift speed into data reg.
1010	unused
1011	unused
1100	unused
1101	unused
1110	unused
1111	unused

Using Delay Cycles for Scan Enable Transition

The BIST controller provides the ability to introduce up to 15 cycles of delay between a scan enable transition, and subsequent clock activity. There will be no clock activity in the core during the delay cycles. By default, the hardware is initialized to provide 0 (zero) cycles of delay. This means the scan_enable transition occurs after the negative edge of the last shift clock, and after the negative edge of the capture clock. Use the [Set Scan_enable Switching](#) command to override the default behavior as follows:

set scan_enable switching fast // default behavior - 0 delay cycles

set scan_enable switching slow // 1 delay cycle

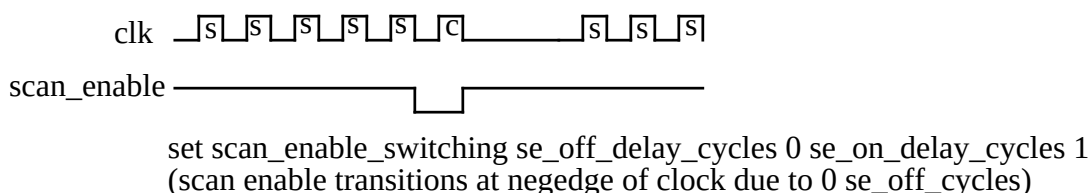
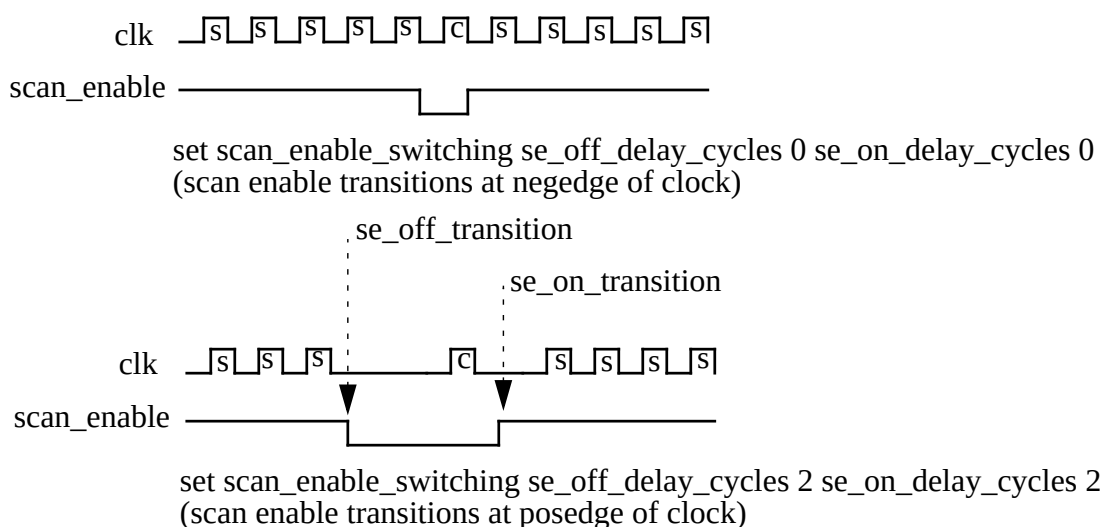
set scan_enable switching cycles 4 // 4 delay cycles after each transition

You should use the [Set Scan_enable Switching](#) command in the LBISTArchitect -bist_controller dofile to set the default value of the internal register that controls this functionality.

You can use the “-delay_cycles” command line option for the LBISTArchitect -tbgen (or -tvgen) to add instructions to the testbench/testvectors to load this register and override the default value. See “[lbistarchitect -tbgen](#)” and “[lbistarchitect -tvgen](#)” in the *LBISTArchitect Reference Manual*.

You activate this register by loading 0010 into the DSELECT register (see [Table B-2](#)). The first four bits of this register (0-3) control the number of delay cycles during the scan enable on transition, and the next four bits control the number of delay cycles during the scan enable off transition. See also “[Layout of the Scan Enable Control Register](#)”.

Figure B-13. Example Scan Enable Transition Switching



NOTE: The scan_enable is “off” during the capture window. The “off” state occurs when the scan_enable signal is low (0).

Layout of Control Register Bits within Data Register

The control register uses the 7 least significant bits of the data register, and can be used after loading the dselect register with 0011 (as shown in the previous table).

The bits in the control register are used as follows:

Table B-3. Control Register Bits

Bit position	Usage
0	single-chain mode (0=off, 1=on) (only works for controllers built with -single_chain option)
1	diagnostics mode (0=off, 1=on) Activating this mode forces the controller to stop after applying the capture window for each pattern. This is used to re-load the MISR and identify all failing patterns in a single pass
2	past flush test (1= past flush test, 0=not past flush test) By default, the controller starts by applying flush test patterns, and stops when the pattern counter matches a hard-coded value (default 32). In order to start the sequence beyond this hard-coded breakpoint, it is necessary to set this bit.
3	unused
6,5,4	capture window size 000 - illegal value 001 - controller waits 1 shift cycle for capture to finish 010 - 2 shift cycles, 011 - 3 shift cycles, 100 - 4 shift cycles 101 - 5 shift cycles, 110 - 6 shift cycles, 111 - 7 shift cycles

Layout of Pattern Counter Start and Stop in Data Register

It is very likely that the data register will contain more bits than are needed to load the pattern counter start and stop values. In this case, the pattern counter is set using the least significant bits of the data register. For example, with a 40 bit data register and a 12 bit pattern counter, bits 11 to 0 of the data register are used to load the pattern counter, and bits 39-12 have no mapping.

Layout of PRPG within Data Register

The PRPG is also typically smaller than the data register. Once again, the PRPG is loaded using the least significant bits of the data register, and the “extra” bits have no mapping.

Layout of MISR within Data Register

The MISR is typically the largest of the internal registers and thus determines the data register size. In this case, the bits of the data register map into the MISR on a 1-1 basis.

Layout of the Scan Enable Control Register

The scan enable control register is an 8-bit register.

[Table B-4](#) defines the scan enable ON and corresponding number of user-specified cycles—register bits 3, 2, 1, and 0.

Table B-4. Layout of the Scan Enable Control Register ON Bit Positions

Bits 3,2,1,0	Number of delay_cycles
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	10
1011	11
1100	12
1101	13
1110	14
1111	15

Table B-5 defines the scan enable OFF and corresponding number of user-specified cycles: register bits 7, 6, 5, and 4.

Table B-5. Layout of the Scan Enable Control Register OFF Bit Positions

Bits 7,6,5,4	Number of delay_cycles
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	10
1011	11
1100	12
1101	13
1110	14
1111	15

Layout of the Shift Speed Register

The shift speed register uses the three least significant bits of the data register. The bits are defined to have the following meaning:

Table B-6. Layout of Shift Speed Register

Bits (2,1,0)	Usage
000	shift at full (bist_clk) rate
001	shift at 1/2 bist_clk rate
010	shift at 1/3 bist_clk rate
011	shift at 1/4 bist_clk rate
100	shift at 1/5 bist_clk rate

Table B-6. Layout of Shift Speed Register

Bits (2,1,0)	Usage
101	shift at 1/6 bist_clk rate
110	shift at 1/7 bist_clk rate
111	shift at 1/8 bist_clk rate

Control Signals to the Core

The BIST controller interacts with the logic in the core design using the general signals listed in [Table B-7](#). Note: The actual names of these signals are defined by the LBISTArchitect configuration files and are set to match the signal names inside the original core design.

Table B-7. Core Interface

Signal Name	Mode	Description
<scan chain serial inputs> core_scan_in1 core_scan_in2 core_scan_in...	Output	The phase shifter outputs use these signals to drive data into the individual scan chains in the core design.
<scan chain serial outputs> core_scan_out1 core_scan_out2 core_scan_out...	Input	Data from the scan chains in the core feed back into the space_compactor (inside the logic BIST controller) through these signals.
<scan enable signals> core_scan_en	Output	These signals control the scan_enable signals inside the core. The logic BIST controller must drive the scan_enables low during the pattern capture cycles.
<MTPI control signals> phase_ctrl1 phase_ctrl2 phase_ctrl...	Output	There is one MTPI control output for each MTPI phase. Each signal, in turn, is set to a '1' by the logic BIST controller to drive a group of test points inside the core.
<clock signals and asynchronous set/resets>	Output	For all designs using multiple clocks; the original clock signals route through the BIST controller to enable selective clock suppression during the capture window.

ATPG Access for the Scan Chains

The BIST controller must be directly connected to the internal scan chains in the core design. This would normally preclude ATPG-style testing using externally defined test patterns. However, some external test patterns are often needed to achieve very high test coverage. For this reason, the original scan chain interface is multiplexed through the BIST controller and made available for ATPG testing.

Table B-8. ATPG interface

Signal Name	Mode	Description
<ATPG scan chain serial inputs> scan_in1 scan_in2 scan_in...	Input	Scan chain inputs to feed into the serial inputs for the core scan chains when logic BIST is not running (bist_run is not set)
<ATPG scan enable signals> scan_en	Input	Scan enable signal to feed into core when logic BIST is not running
<ATPG scan out signals> scan_out1 scan_out2 scan_out...	Output	The scan chain outputs from the core are also connected to output pads to complete the circuit for ATPG access

Asynchronous Logic BIST Test Signals

The following sections list the input and output signals if you configure the Logic BIST controller for an asynchronous test—for a detailed discussion, see [“Setting Up an Asynchronous BIST Test”](#) in the [“Generating the BIST Controller”](#).

If you use this operation and have no requirements for diagnostics or other advanced functionality, then you should do the following:

- Tie the following signals low: bist_tck, bist_shiftdr, bist_dselect_enable, and bist_data_enable, bist_dselect_updatedr, bist_data_updatedr, and bist_diag_clk
- Leave the following signals unconnected at the top level: bist_dselect_sout and bist_data_sout outputs

Parallel Interface Signals

Table B-9 identifies the signals for the parallel interface (-Parallel_interface argument).

Table B-9. Parallel Interface Test Signals

Signal Name	Mode	Description
bist_run_input	Input	The bist_run_input is a single bit input signal for triggering the asynchronous Logic BIST test.
bist_done	Output	The bist_done is a single bit output signal and transitions from low to high at the conclusion of the Logic BIST test.
misr_value	Output	The misr_value is a vector output providing parallel access to the MISR value.

On-Chip Comparator Signals

Table B-10 identifies the signals for the on-chip comparator (-On_chip_comparator argument).

Table B-10. On-Chip Comparator Test Signals

Signal Name	Mode	Description
bist_run_input	Input	The bist_run_input is a single bit input signal for triggering the asynchronous Logic BIST test.
bist_done	Output	The bist_done is a single bit output signal and transitions from low to high at the conclusion of the Logic BIST test.
test_fail	Output	The test_fail is a single bit output signal and goes to low at the conclusion of the Logic BIST test if the internal MISR signature matches the reference signature.

Miscellaneous BIST-Related Signals

These signals are not directly used by the BIST controller, but feed directly into the core.

Table B-11. Miscellaneous BIST signals

Signal Name	Mode	Description
test_en	Input (to core)	The test_en signal activates the MTPI control point logic inside the core. Test points are optional, but can increase fault coverage.
lbist_en	Input (to core)	The lbist_en signal activates the X bounding logic inside the core. This is essential for logic BIST because the propagation of a single 'X' into the MISR invalidates the entire test.

Table B-11. Miscellaneous BIST signals

Signal Name	Mode	Description
test_obs	Inout (to core)	The test_obs signal activates all observe points inserted in the core. Test points are optional, but can increase fault coverage.

Note that test_en, test_obs, and lbist_en must be activated using the external tester before the logic BIST test can function properly, if those signals are present, and not constrained with the Add Pin Constraints command.

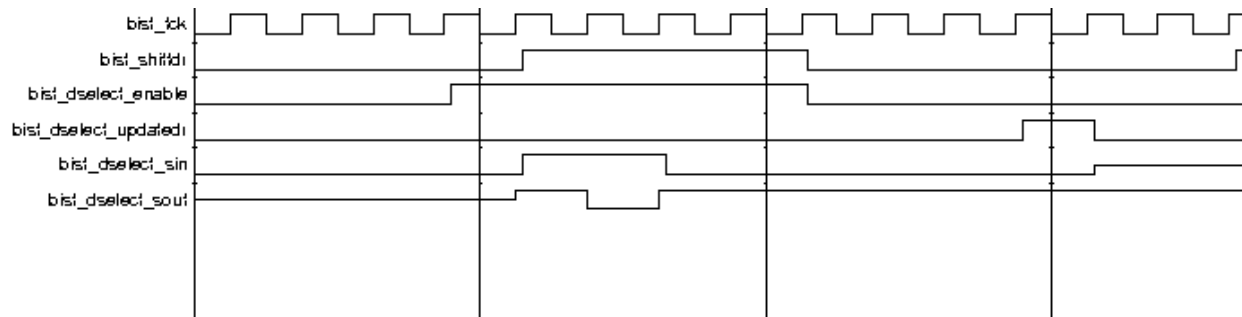
Timing Between ATE and BIST Controller

This section contains diagrams that show waveforms at the interface between the BIST controller and the ATE.

Loading the dselect Register

Operating the Logic BIST Controller consists of two basic operations - loading the dselect register, and the data register. The following diagram shows the timing that is necessary to load a value of (0011) into the dselect register.

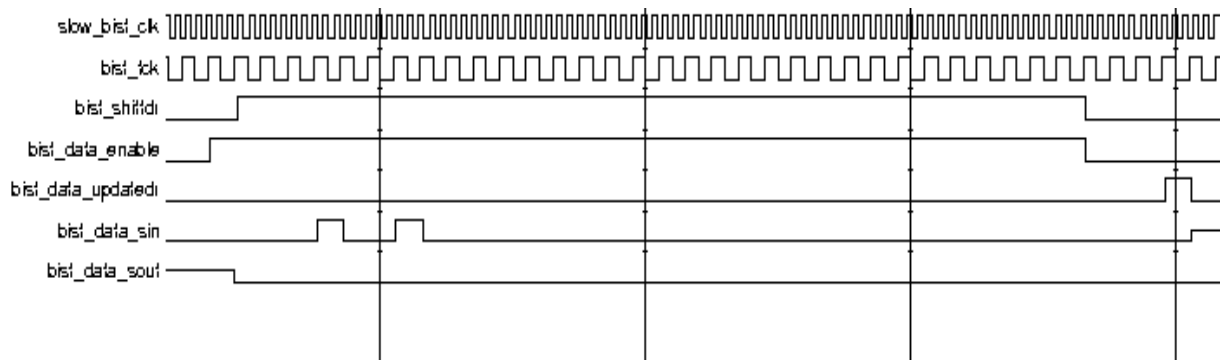
Figure B-14. dselect Register Load



Loading the Data Register

The contents of the data register are mapped into various internal registers based on the current value in the dselect register. This example assumes that the dselect_register was set to (0011), and loads the data register with values that are then driven into the internal control register.

Figure B-15. data register load



Typical Logic BIST Control Sequence

The operation of the Logic BIST controller is entirely defined by sequences of dselect register load and data register load operations.

The following sequence of operations would reset the controller, initialize the internal registers, run the test, and unload the final MISR value.

```
load_dselect with 0011 (select control register)
load data_register with.....00101000 (slow_se is set, capture window size=1)
load dselect with 0101 (select pattern counter start value)
load data register with.....000000000000 (start with pattern 0)
load dselect with 0110 (select pattern counter end value)
load data register with.....000011111111(stop at pattern 255)
load dselect with 0111 (select PRPG register)
load data_register with.....11111111111111111111 (PRPG init value)
load dselect with 1000 (select MISR register)
load data_register with 1.....11111111111111111111 (MISR init value)
load dselect with 0001 (activate Logic BIST test)
```

<wait for test to complete>

unload final MISR value from data register

load dselect with 0000 (turn logic BIST controller off)

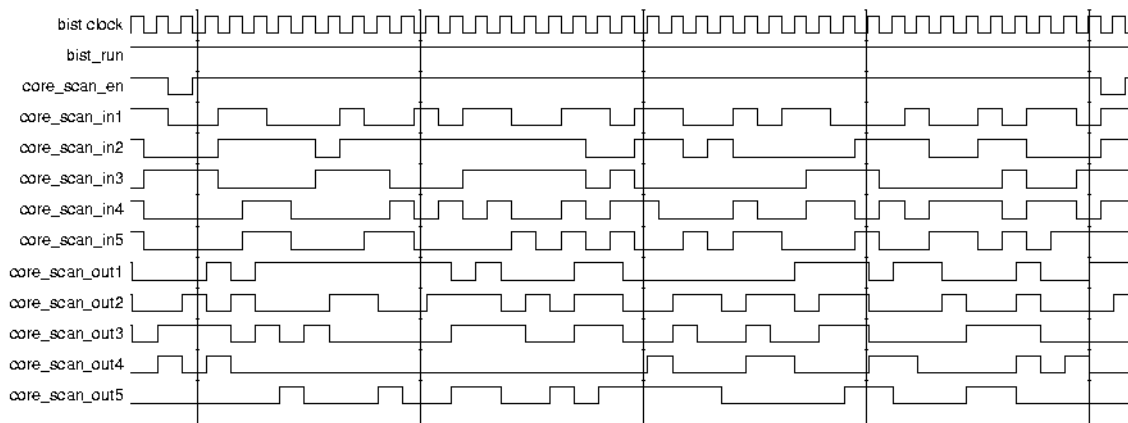
Timing Between BIST Controller and Core

This section contains diagrams that show waveforms at the interface between the BIST controller and the core design.

Loading and Applying a Single Pattern Into the Core

The following diagram illustrates the flow of information between the BIST controller and the core while shifting a single pattern into the internal scan chains.

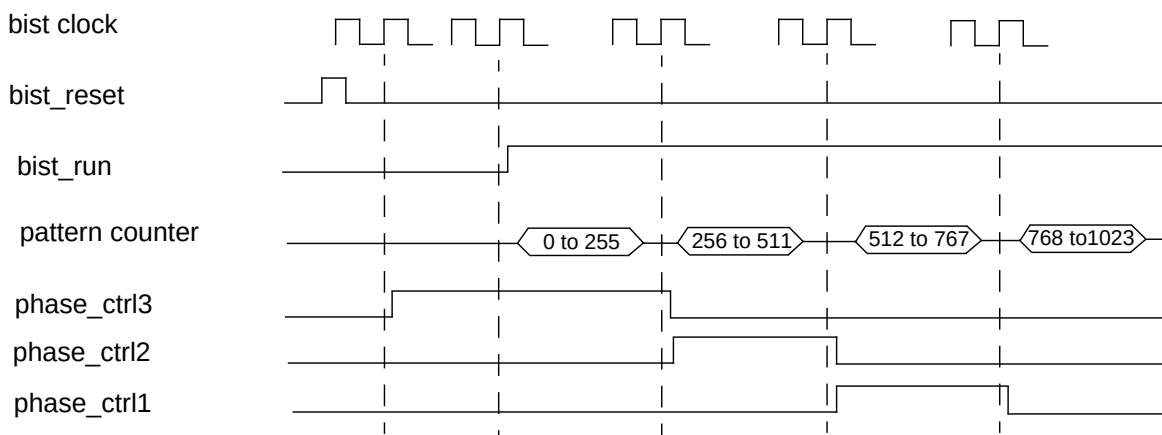
Figure B-16. Scan Chain Pattern Load Timing



Driving the MTPI Signals Into the Core

The BIST controller drives the MTPI signals to enable internal control points in the core design during portions of the BIST test. The basic requirement for MTPI is that the application of patterns be divided into phases, the number of which must be a power of two (typically, 2, 4, or 8). The total number of patterns must also be a power of two, which means the MTPI decode logic can be very simple. The following diagram shows the status of the MTPI control lines given an initial pattern counter value of 1024 and the selection of four MTPI phases.

Figure B-17. Phase Control Signal Timing

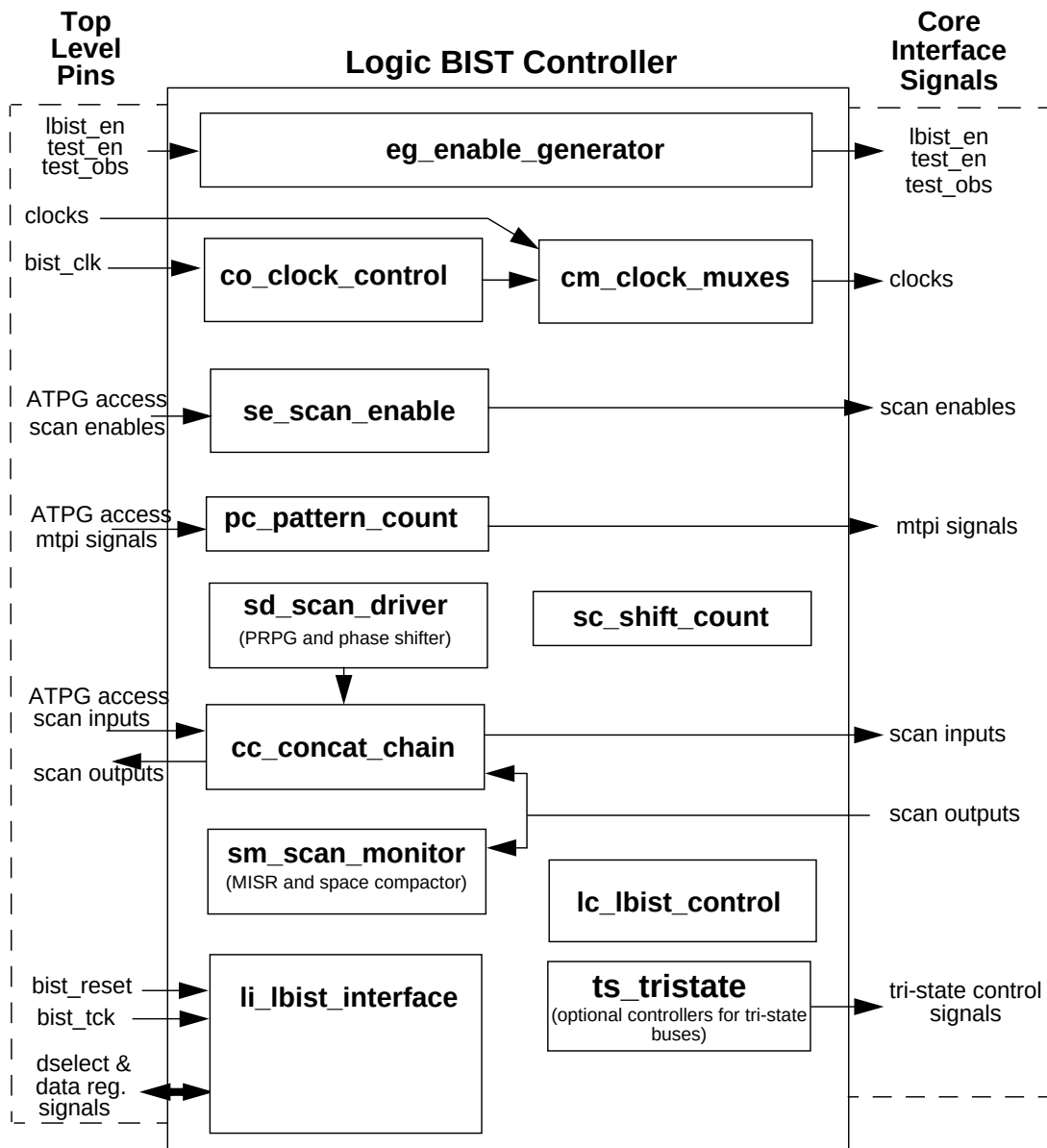


Note: Diagram shows phase_ctrl signals changing immediately after positive clock edge. These signals go through lockup latches so that the output doesn't change until the subsequent negative clock edge.

Brief Description of Sub-blocks

This section provides a more detailed description of the functionality and implementation of each sub-block shown in the Logic BIST Controller diagram of [Figure B-18](#).

Figure B-18. BIST Controller Sub-blocks



For more information on each of the sub-blocks, refer to the following subsections:

Linear Feedback Shift Register (PRPG and MISR)	cm_clock_muxes
sd_scan_driver	eg_enable_generator
sm_scan_monitor	cc_concat_chain
pc_pattern_count	lc_lbist_control
se_scan_enable	li_lbist_interface
sc_shift_count	ts_tristate
co_clock_control	

Linear Feedback Shift Register

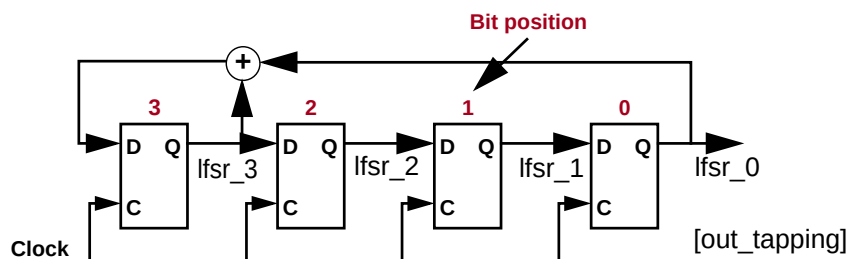
The PRPG and the MISR are both implemented using Linear Feedback Shift Registers (LFSRs). There are two ways to build an LFSR. The classical approach uses a simple shift register where the input to the first cell in the shift register is constructed by combining the appropriate bits inside the shift registers with XOR gates. Note that all the XOR gates in this approach combine to form a single input feeding the first cell of the LFSR. This approach is called out-tapping. Another approach is to insert XOR gates into cells in the middle of the shift register. This approach is called in-tapping, and is useful because the generated hardware involves only two-input XOR gates between register stages and thus can operate at higher clock frequencies. The PRPG shown in the following section uses out-tapping, whereas the MISR uses in-tapping. This is purely arbitrary; you can build either one using either method. You can always create an LFSR using out-tapping which matches the sequence produced by an LFSR using in-tapping. The reverse is also true; each LFSR created using in-tapping has a matching LFSR which uses out-tapping.

sd_scan_driver

The `sd_scan_driver` module combines a LFSR (configured as a PRPG) with a network of XOR gates that provide the scan chain inputs while Logic BIST is running. The PRPG acts as a source of pseudorandom bit streams that have a very distinct phase relationship, stemming from its implementation as an LFSR. Outputs from the PRPG come from consecutive stages in the LFSR. The diagram in [Figure B-19](#) shows the basic structure of a 4-bit LFSR.

Figure B-19. 4-Bit PRPG

Generating Function: $f(x) = x^4 + x^3 + 1$



Note: lfsr_3 through lfsr_0 are PRPG outputs suitable for driving scan chains

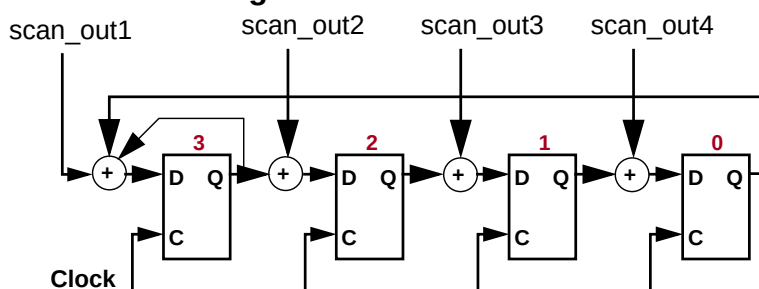
The phase shifter compensates for the inherent aliasing between PRPG outputs (lfsr_3 through lfsr_0 in the preceding figure) and produces bit streams suitable for driving the scan chain inputs in the core. This is done by using XOR gates to merge several PRPG outputs and using the resulting signal to drive the scan chain input.

sm_scan_monitor

The scan monitor combines another LFSR (configured as a MISR) with an XOR gate network driven by the scan chain outputs from the core.

The diagram in [Figure B-20](#) shows the implementation of a MISR capable of processing inputs from 4 scan chains.

Figure B-20. 4-Bit MISR



Generating Function: $f(x) = x^4 + x^3 + 1$ [out-tapping]

The number of inputs to the MISR cannot exceed the number of bits in the LFSR. The space compactor comes into use when there are more scan chains than there are bits in the LFSR. The space compactor simply combines adjoining scan chains using XOR gate structures until the number of space compactor outputs is equal to the number of stages in the MISR.

pc_pattern_count

The pattern count module is used by the control logic (in `lc_lbist_control`) to keep track of how many patterns have been applied to the core. The counter is loaded from the value in the data register whenever `dselect` has been set to the “pattern counter start reg” value, and the `bist_data_reg_updatedr` signal transitions high.

The module also contains an externally loadable value that determines the last pattern that should be run. This register is loaded whenever the `dselect` register is set to the “pattern counter end reg” value, and the `bist_data_reg_updatedr` signal transitions high.

By default, the pattern counter starts at 0, and runs to some hard-coded end value. The pattern counter is also used to select between different capture windows. The least significant bits of the current pattern count value are fed into the clock control block in order to switch between the various hard-coded capture window waveforms. All clocks are driven at `slow_bist_clk` rate when data is shifted through the scan interfaces, but only selective clocks are driven during the capture stage.

se_scan_enable

The `se_scan_enable` module drives the scan enable signals for the core design. The default behavior is to drive the scan enable signals high during the shift cycles, and low during the capture cycle. If the “flush test” count is set to a non-zero value, then the scan enable is held high during the capture cycle for a certain range of patterns. For example, if the controller is built with “set lbist controller -flush_test 32,” then patterns 0 to 31 will be applied with the scan enable signals set to ‘1.’

This is useful for the following reasons. Operating the logic in the normal mode may require the application of many patterns before you can confirm that none of the scan multiplexers are stuck in the capture position. You might have blocks of logic that typically output the same value unless you apply a very unique trigger pattern in a control register. These logic blocks are resistant to detection of this fault type by pseudorandom patterns, and the scan chains that capture the outputs from this type of block will repeatedly scan out the same value.

You might not detect faults in the scan enable muxes for flip-flops near the start of the chain if the “stuck” output value matches the predicted value. The flush test mechanism allows you to quickly test the integrity of the scan chain using a small number of patterns by shifting data through each chain and bypassing the capture cycle. This detects scan enable muxes that are stuck in the capture position. You must apply additional patterns in normal mode to detect scan enable muxes that are stuck in the shift position.

sc_shift_count

The sc_shift_count module keeps track of the number of cycles needed to shift a single pattern into the core. The count is determined by the length of the longest scan chain. The controller uses this counter to keep the scan_enable signal high during the shift phase;

NOTE: The shift counter is also re-used when generating the capture window. It keeps track of how many “slow_bist_clk” cycles are needed to let the logic in the clock control block finish applying each capture window. This number is set by the three “capture window size” bits in the control register.

co_clock_control

The clock control block contains all the logic (except the actual clock gates in cm_clock_muxes) that are driven at the fast (bist_clk) rate. It contains the following elements:

- programmable clock divider - generate slow_bist_clk, and pulse and hold signals for driving shift cycles into the core.

- Synchronizer that “re-times” mode control inputs from state machine (lc_lbist_control) driven relative to slow_bist_clk into bist_clk domain.

- One capture window generator for each clock (and set/reset clock) that is driven into the core.

As previously noted, all logic within this block has been designed with a minimum amount of combinational logic between register stages so that it can operate from a very fast (bist_clk) input signal.

cm_clock_muxes

All the clock signals are routed through the cm_clock_muxes block before feeding into the core. This block inserts muxing logic so that the core clocks can be driven by either a gated version of bist_clk, or an internally generated slow waveform when the Logic BIST controller is active. It also maps a “bist_diag_clk” input into each core clock when the logic BIST test has completed, and (bist_dselect) still contains any value besides “0000” (Logic BIST de-activated).

eg_enable_generator

This module is used to generate test mode control signals for the core. The lbist_enable, test_enable, and test_obs signal are activated whenever the BIST test is running. These signals can also be controlled via primary input pins when the BIST test is not running.

cc_concat_chain

This module muxes the scan chain connections between the external access pins which are provided for ATPG access, and the internal drivers (from sd_scan_driver) and receivers (into sm_scan_monitor). This module is also responsible for merging core scan chains together into larger chains when the “connect iscan chains” option is used. All scan chains can also be merged into a single chain when the “set lbist controller -single_core_chain” option is used.

lc_lbist_control

The module contains the state machine that controls the operation of the Logic BIST test. The bits in the “control register” are also decoded in this module, since most of these bits directly affect the operation of the state machine.

li_lbist_interface

The lbist_interface module contains all the logic that is driven relative to bist_tck. This means the dselect register, and the “raw” data register are implemented in this block.

ts_tristate

Tri-state controllers are used to take control of the enable signals for tri-state drivers inside the core design. Tri-state buses are identified during the BIST-Ready phase. You can configure the tool to automatically modify the netlist and insert muxes, providing external control of the enables for the tri-state drivers. When you use this feature, the logic BIST controller may contain several different ts_tristate modules, each controlling the enables for nets inside the core that have the same number of drivers.

Appendix C

Example Fault Simulation Dofile

Deconstruction of Fault Simulation Dofile

This section provides a line-by-line annotated examination of an example fault simulation dofile—see “[Performing Fault Simulation](#)” for details on using this tool-produced dofile.

During BIST controller generation, LBISTArchitect automatically produces a dofile similar to this example, in this case a dofile for [Core Fault Simulation](#). In contrast, a tool-produced dofile for [Design Fault Simulation](#) is virtually identical to this example, except the design fault simulation dofile contains commands for non-core fault removal, and core-specific inputs and outputs.

Note



When you perform fault simulation, you should use *as is* the tool-produced fault simulation dofile for input into LBISTArchitect.

```
////////////////////////////////////
////////
//      Copyright 2001-2006 Mentor Graphics Corporation
//
//                      All Rights Reserved.
//
//      THIS WORK CONTAINS TRADE SECRET AND PROPRIETARY
//      INFORMATION WHICH IS THE PROPERTY OF MENTOR GRAPHICS
//      CORPORATION OR ITS LICENSORS AND IS SUBJECT TO LICENSE TERMS
//
//
// File Type:      core level fault simulation dofile
// Date Created:   Mon Oct 16 13:00:26 2006
// Tool Version:   LBISTArchitect v8.2006_3.10  Tue Aug 22 21:45:04 GMT 2006
//
////////////////////////////////////
////////
//See Set Design Rule Checks
set drc handling E5 warning
set drc handling C2 warning
set drc handling E9 error
set drc handling E10 error atpg_analysis
set drc handling E11 error atpg_analysis -mode z

//////See Define the Scan Groups
set number shifts 16
add scan groups grp1 cpu_bist_core.testproc
```

```
//See Define the Scan Chains
add scan chains chain1 grp1 scan_in[1] scan_out[1]
add scan chains chain2 grp1 scan_in[2] scan_out[2]
add scan chains chain3 grp1 scan_in[3] scan_out[3]
add scan chains chain4 grp1 scan_in[4] scan_out[4]
add scan chains chain5 grp1 scan_in[5] scan_out[5]
add scan chains chain6 grp1 scan_in[6] scan_out[6]
add scan chains chain7 grp1 scan_in[7] scan_out[7]
add scan chains chain8 grp1 scan_in[8] scan_out[8]
add scan chains chain9 grp1 scan_in[9] scan_out[9]
add scan chains chain10 grp1 scan_in[10] scan_out[10]
add scan chains chain11 grp1 scan_in[11] scan_out[11]
add scan chains chain12 grp1 scan_in[12] scan_out[12]
add scan chains chain13 grp1 scan_in[13] scan_out[13]
add scan chains chain14 grp1 scan_in[14] scan_out[14]
add scan chains chain15 grp1 scan_in[15] scan_out[15]
add scan chains chain16 grp1 scan_in[16] scan_out[16]
add scan chains chain17 grp1 scan_in[17] scan_out[17]
add scan chains chain18 grp1 scan_in[18] scan_out[18]
add scan chains chain19 grp1 scan_in[19] scan_out[19]
add scan chains chain20 grp1 scan_in[20] scan_out[20]
add scan chains chain21 grp1 scan_in[21] scan_out[21]
add scan chains chain22 grp1 scan_in[22] scan_out[22]
add scan chains chain23 grp1 scan_in[23] scan_out[23]
add scan chains chain24 grp1 scan_in[24] scan_out[24]
add scan chains chain25 grp1 scan_in[25] scan_out[25]
add scan chains chain26 grp1 scan_in[26] scan_out[26]
add scan chains chain27 grp1 scan_in[27] scan_out[27]
add scan chains chain28 grp1 scan_in[28] scan_out[28]
add scan chains chain29 grp1 scan_in[29] scan_out[29]
add scan chains chain30 grp1 scan_in[30] scan_out[30]
add scan chains chain31 grp1 scan_in[31] scan_out[31]
add scan chains chain32 grp1 scan_in[32] scan_out[32]

//See Define I/O Pin Constraints
add pin constraint ESFRDI[0] CX
add pin constraint ESFRDI[1] CX
add pin constraint ESFRDI[2] CX
add pin constraint ESFRDI[3] CX
add pin constraint ESFRDI[4] CX
add pin constraint ESFRDI[5] CX
add pin constraint ESFRDI[6] CX
add pin constraint ESFRDI[7] CX
add pin constraint MWAIT CX
add pin constraint PORT0I[0] CX
add pin constraint PORT0I[1] CX
add pin constraint PORT0I[2] CX
add pin constraint PORT0I[3] CX
add pin constraint PORT0I[4] CX
add pin constraint PORT0I[5] CX
add pin constraint PORT0I[6] CX
add pin constraint PORT0I[7] CX
add pin constraint PORT1I[0] CX
add pin constraint PORT1I[1] CX
add pin constraint PORT1I[2] CX
add pin constraint PORT1I[3] CX
add pin constraint PORT1I[4] CX
```

```

add pin constraint PORT1I[5] CX
add pin constraint PORT1I[6] CX
add pin constraint PORT1I[7] CX
add pin constraint PORT2I[0] CX
add pin constraint PORT2I[1] CX
add pin constraint PORT2I[2] CX
add pin constraint PORT2I[3] CX
add pin constraint PORT2I[4] CX
add pin constraint PORT2I[5] CX
add pin constraint PORT2I[6] CX
add pin constraint PORT2I[7] CX
add pin constraint PORT3I[0] CX
add pin constraint PORT3I[1] CX
add pin constraint PORT3I[2] CX
add pin constraint PORT3I[3] CX
add pin constraint PORT3I[4] CX
add pin constraint PORT3I[5] CX
add pin constraint PORT3I[6] CX
add pin constraint PORT3I[7] CX
add pin constraint PROGDI[0] CX
add pin constraint PROGDI[1] CX
add pin constraint PROGDI[2] CX
add pin constraint PROGDI[3] CX
add pin constraint PROGDI[4] CX
add pin constraint PROGDI[5] CX
add pin constraint PROGDI[6] CX
add pin constraint PROGDI[7] CX
add pin constraint SOURCE_DI[0] CX
add pin constraint SOURCE_DI[1] CX
add pin constraint SOURCE_DI[2] CX
add pin constraint SOURCE_DI[3] CX
add pin constraint SOURCE_DI[4] CX
add pin constraint SOURCE_DI[5] CX
add pin constraint SOURCE_DI[6] CX
add pin constraint SOURCE_DI[7] CX
add pin constraint XINTR_SRC[10] CX
add pin constraint XINTR_SRC[11] CX
add pin constraint XINTR_SRC[12] CX
add pin constraint XINTR_SRC[13] CX
add pin constraint XINTR_SRC[5] CX
add pin constraint XINTR_SRC[6] CX
add pin constraint XINTR_SRC[7] CX
add pin constraint XINTR_SRC[8] CX
add pin constraint XINTR_SRC[9] CX
add pin constraint XRAMDI[0] CX
add pin constraint XRAMDI[1] CX
add pin constraint XRAMDI[2] CX
add pin constraint XRAMDI[3] CX
add pin constraint XRAMDI[4] CX
add pin constraint XRAMDI[5] CX
add pin constraint XRAMDI[6] CX
add pin constraint XRAMDI[7] CX
add pin constraint lbist_en c1
add pin constraint test_en c1

//See Define the PRPGs
add lfsrs prpg PRPG 16 FFFF -serial -in
add lfsr taps prpg 11 13 14

```

Example Fault Simulation Dofile

Deconstruction of Fault Simulation Dofile

```
add lfsr conn scan_in[1] prpg 6 9 7
add lfsr conn scan_in[2] prpg 2 14 11
add lfsr conn scan_in[3] prpg 12 0 5
add lfsr conn scan_in[4] prpg 1 8 15
add lfsr conn scan_in[5] prpg 13 3 10
add lfsr conn scan_in[6] prpg 6 14 3
add lfsr conn scan_in[7] prpg 1 8 9
add lfsr conn scan_in[8] prpg 0 15 7
add lfsr conn scan_in[9] prpg 4 2 5
add lfsr conn scan_in[10] prpg 4 13 12
add lfsr conn scan_in[11] prpg 13 0 11
add lfsr conn scan_in[12] prpg 2 11 14
add lfsr conn scan_in[13] prpg 1 5 10
add lfsr conn scan_in[14] prpg 8 15 7
add lfsr conn scan_in[15] prpg 10 9 4
add lfsr conn scan_in[16] prpg 8 6 13
add lfsr conn scan_in[17] prpg 10 11 9
add lfsr conn scan_in[18] prpg 14 7 6
add lfsr conn scan_in[19] prpg 5 15 2
add lfsr conn scan_in[20] prpg 12 0 3
add lfsr conn scan_in[21] prpg 4 12 1
add lfsr conn scan_in[22] prpg 15 9 2
add lfsr conn scan_in[23] prpg 10 1 14
add lfsr conn scan_in[24] prpg 4 11 0
add lfsr conn scan_in[25] prpg 13 5 7
add lfsr conn scan_in[26] prpg 6 8 12
add lfsr conn scan_in[27] prpg 4 14 3
add lfsr conn scan_in[28] prpg 11 9 3
add lfsr conn scan_in[29] prpg 1 10 12
add lfsr conn scan_in[30] prpg 2 8 13
add lfsr conn scan_in[31] prpg 5 15 0
add lfsr conn scan_in[32] prpg 14 1 10

//See Define the MISR
add lfsrs misr MISR 32 FFFFFFFF -serial -in
add lfsr taps misr 10 30 31
add lfsr conn scan_out[1] misr 0
add lfsr conn scan_out[2] misr 1
add lfsr conn scan_out[3] misr 2
add lfsr conn scan_out[4] misr 3
add lfsr conn scan_out[5] misr 4
add lfsr conn scan_out[6] misr 5
add lfsr conn scan_out[7] misr 6
add lfsr conn scan_out[8] misr 7
add lfsr conn scan_out[9] misr 8
add lfsr conn scan_out[10] misr 9
add lfsr conn scan_out[11] misr 10
add lfsr conn scan_out[12] misr 11
add lfsr conn scan_out[13] misr 12
add lfsr conn scan_out[14] misr 13
add lfsr conn scan_out[15] misr 14
add lfsr conn scan_out[16] misr 15
add lfsr conn scan_out[17] misr 16
add lfsr conn scan_out[18] misr 17
add lfsr conn scan_out[19] misr 18
add lfsr conn scan_out[20] misr 19
add lfsr conn scan_out[21] misr 20
add lfsr conn scan_out[22] misr 21
```

```
add lfsr conn scan_out[23] misr 22
add lfsr conn scan_out[24] misr 23
add lfsr conn scan_out[25] misr 24
add lfsr conn scan_out[26] misr 25
add lfsr conn scan_out[27] misr 26
add lfsr conn scan_out[28] misr 27
add lfsr conn scan_out[29] misr 28
add lfsr conn scan_out[30] misr 29
add lfsr conn scan_out[31] misr 30
add lfsr conn scan_out[32] misr 31

//See Add Mask
add output mask -all

//See Define the MTPI Controller
add mtpi controller mtpic1 /phase_cntl1 \
    /phase_cntl2 \
    /phase_cntl3
add mtpi output mtpic1 0 001
add mtpi output mtpic1 15 010
add mtpi output mtpic1 31 100
add mtpi output mtpic1 47 000
add mtpi output mtpic1 64 repeat

//See Define Capture Waveforms
add bist capture cap1 0 8
add bist capture cap2 9 11
add bist capture cap3 12 12
add bist capture cap4 13 13
add bist capture cap5 14 15
set pattern type -seq 1
add mtpi controller mcdc2 scan_en
add mtpi output mcdc2 0 1
add mtpi output mcdc2 32 0

//See Define Clocks
add clocks 0 CLK1
add clocks 0 CLK2
add clocks 0 CLK3
add clocks 1 RESET
add clocks 1 TRESET
set bist chain_test 32

//See Run the Fault Simulation
set sys mod fault
set pattern source bist -store
set pattern classification off
set bist trace -lfsr cpu_bist_lfsr.trace.gz -replace
set fault mode collapsed
add faults -all
set possible credit 0
set random patterns 2048
run
save patterns cpu_bist.pat.gz -scan_test -ascii -replace
save flat model cpu.flat -rep
analyze redundancy
update implication detection
report lfsrs
```

```
report statistics
write faults cpu_bist_topup.faultfile -class ds -class di -noeq -replace
```

Set Design Rule Checks

This part of the dofile sets BIST-specific DRC checks for potential X sources and internal tri-state buses.

```
set drc handling E5 warning
set drc handling C2 warning
set drc handling E9 error
set drc handling E10 error atpg_analysis
set drc handling E11 error atpg_analysis -mode z
```

Define the Scan Groups

This part of the dofile defines the scan groups, and primary inputs and outputs defining the correct scan chains and the scan input and output ports.

```
set number shifts 16
add scan groups grp1 cpu_bist_core.testproc
```

Define the Scan Chains

This part of the dofile defines the scan chains for the scan groups and scan inputs and outputs.

```
add scan chains chain1 grp1 scan_in[1] scan_out[1]
add scan chains chain2 grp1 scan_in[2] scan_out[2]
add scan chains chain3 grp1 scan_in[3] scan_out[3]
add scan chains chain4 grp1 scan_in[4] scan_out[4]
...
add scan chains chain31 grp1 scan_in[31] scan_out[31]
add scan chains chain32 grp1 scan_in[32] scan_out[32]
```

Define I/O Pin Constraints

This part of the dofile constrains specific I/O pins and the BIST control circuitry pins.

```
add pin constraint ESFRDI[0] CX
add pin constraint ESFRDI[1] CX
add pin constraint ESFRDI[2] CX
add pin constraint ESFRDI[3] CX
add pin constraint ESFRDI[4] CX
...
add pin constraint XRAMDI[5] CX
add pin constraint XRAMDI[6] CX
add pin constraint XRAMDI[7] CX
add pin constraint lbist_en c1
add pin constraint test_en c1
```

Define the PRPGs

This part of the dofile defines the PRPG and its connections to the scan pins. Also, there is one Add LFSR Connections command for each of the 32 scan_in pins.

```
add lfsrs prpg PRPG 16 FFFF -serial -in
add lfsr taps prpg 11 13 14
add lfsr conn scan_in[1] prpg 6 9 7
add lfsr conn scan_in[2] prpg 2 14 11
...
add lfsr conn scan_in[32] prpg 14 1 10
```

Define the MISR

This part of the dofile defines the MISR and its connections to the scan pins. Also, there is one Add LFSR Connections command for each of the 32 scan_in pins.

```
add lfsrs misr MISR 32 FFFFFFFF -serial -in
add lfsr taps misr 10 30 31
add lfsr conn scan_out[1] misr 0
add lfsr conn scan_out[2] misr 1
add lfsr conn scan_out[3] misr 2
...
add lfsr conn scan_out[32] misr 31
```

Add Mask

This part of the dofile adds an output mask for each of the outputs.

```
add output mask -all
```

Define the MTPI Controller

This part of the dofile defines the MTPI controller using a combination of MTPI-specific commands.

```
add mtpi controller mtpic1 /phase_cntl1 \
    /phase_cntl2 \
    /phase_cntl3
add mtpi output mtpic1 0 001
add mtpi output mtpic1 15 010
add mtpi output mtpic1 31 100
add mtpi output mtpic1 47 000
add mtpi output mtpic1 64 repeat
```

Define Capture Waveforms

This part of the dofile defines the clock waveforms for the subsequent capture of each pattern. This is done by referring to a named ATPG capture procedure in the test procedure file.

```
add bist capture cap1 0 8
add bist capture cap2 9 11
add bist capture cap3 12 12
add bist capture cap4 13 13
add bist capture cap5 14 15
set pattern type -seq 1
```

Define Clocks

This part of the dofile defines the clocks.

```
add clocks 0 CLK1
add clocks 0 CLK2
add clocks 0 CLK3
add clocks 1 RESET
add clocks 1 TRESET
set bist chain_test 32
```

Run the Fault Simulation

This part of the dofile actually runs the fault simulation.

```
//Changes to the BIST fault simulation mode; flattens the design
set sys mod fault

//Specifies the source of the patterns so you can save them to a file
set pattern source bist -store
set pattern classification off

//Facilitates debug by enabling tracing of the PRPG and MISR values after
//each pattern, and saves them to a file
set bist trace -lfsr cpu_bist_lfsr.trace.gz -replace
set fault mode collapsed

//Adds the faults
add faults -all
set possible credit 0
set random patterns 2048

//Begins the fault simulation
run

//Saves the patterns to an external file
save patterns cpu_bist.pat.gz -scan_test -ascii -replace
save flat model cpu.flat -rep
analyze redundancy

//Reports the PRPG value and MISR signature in the transcript
update implication detection
report lfsrs

//Reports fault coverage statistics in the transcript
report statistics
```



```
//Writes the undetected faults to a file  
write faults cpu_bist_topup.faultfile -class ds -class di -noeq -replace
```


— A —

Asynchronous set and reset, 81

At-speed testing

capture, 89

capture, sequences, 83

transition faults, 82

— B —

BIST circuitry

illustrated, 40

overview, 15

BIST controller

architectural overview, 123

ATPG scan muxes, 152

block diagram, 125

clock controller, 151

clock muxes, 151

clock suppression, 151

control functions, 152

core data path connections, 124

enable functions, 151

external view, 59

flush test counter, 150

generating, 50

generation phase, 16

inputs, 42

interface, 130

internal view, 59

LFSR, 148

MISR, 149

outputs, 42

pattern counter, 150

phase shifter, 148

PLL interface commands, 60

PRPG, 148

scan enables, 150

shift counter, 151

space compactor, 149

sub-blocks, 146

tri-state controllers, 152

BIST verification, 17

BIST-Ready

inputs, 21, 23

multiple invocations, 21

outputs, 21, 23

phase, 16

process flow, 19, 21

Boundary scan, 13

BSDArchitect, 13

driver files for, 17

— C —

Capture

cycle, defined, 126

sequence, at-speed, 83

using PLL clocks, 58

waveforms, at-speed, 55

Clock controller, 151

Clock signals

bist_clk, 72

fast_bist_clk, 74

generating, 71

slow_bist_clk, 74

slow_bist_clock, 74

Clocks

architecture, 72

asynchronous, 81

control architecture, 73

control logic, 72

controlling core, 72

identifying, 28

multiple clock, 81

PLL, 58

scan chain concatenation, 128

scan enable switching, 80

testing, multiple, 86

Connect

internal scan chains, 54

— D —

Design rule checks
 BIST-related, 29
 performing, 22

Diagnostics
 inputs, 106
 outputs, 108
 unload scan chains, 90

— F —

Fault
 coverage, described, 92

Fault simulation
 generation phase, 16
 inputs, 96
 outputs, 96
 process, 95
 sequential, 17
 simulating core faults, 99
 simulating design faults, 98
 types, 92

Faults
 classes, 93
 codes, 93
 coverage, 93
 MTPI technology, 13
 stuck-at, 81
 transition, at-speed, 82

Flush test counter, 150

— G —

Generate
 BIST controller, 50
 clock signals during capture, 80
 slow_bist_clk signal, 74

— H —

Hold mode, 84

— I —

Input bounding
 defined, 31

Insert
 scan circuitry, 22
 test points, 32

Interface signals
 ATPG topup, 141

BIST enables, 142
 core control, 140

Invoke
 LBISTArchitect, 46

— L —

LBISTArchitect
 basic process flow, 13
 defined, 13
 invoking, 46
 tool phases, 15

LFSR *see* Linear Feedback Shift Register

Linear Feedback Shift Register
 defined, 148

— M —

MISR, 149
 defined, 41
 preventing unknown states, 31
 PRPG interface, 89

Modes
 hold, 84
 pulse, 83
 stretch, 85

MTPI *see* Multi-Phase Test Point Insertion

Multi-phase Test Point Insertion, 126
 BIST-Ready phase, 16
 defined, 13, 126
 inserting test points, 32
 signals, 89

Muxes
 ATPG scan, 152
 clock, 151
 controlling X sources, 32
 input bounding, 31

— O —

Output bounding
 defined, 36

— P —

Pattern counter, 126, 150
 defined, 42

Pattern loading
 generating clock signals, 74

Phase shifter, 148
 defined, 41

- PLL
 - interface commands, [60](#)
 - on-chip clocks, [58](#)
- Process flow
 - BIST verification, [17](#)
 - BIST-Ready, [19](#)
 - diagnostics, [104](#)
 - fault simulation, [95](#)
 - for LBISTArchitect, [13](#)
- PRPG *see* Pseudorandom Pattern Generator
- Pseudorandom Pattern Generator, [148](#)
 - defined, [41](#)
 - MISR interface, [89](#)
- Pulse mode, [83](#)
- R —
- Reset signals
 - described, [28](#)
 - testing, [81](#)
- Response processing, [149](#)
- S —
- Scan chains
 - connecting, internal, [54](#)
 - insertion, setting up, [30](#)
 - shifting, [88](#)
 - unloading, [87](#)
- Scan enable signals, [150](#)
 - generating, [80](#)
 - scan_enable, [90](#)
 - timing, [90](#)
- Sequential fault simulation, [17](#)
- Set signals
 - described, [28](#)
 - testing, [81](#)
- Setup
 - asynchronous BIST test, [52](#)
 - BIST controller, [52](#)
 - BIST controller PLL interface, [60](#)
 - scan chains, inserting, [30](#)
- sg_slow_clock_gen, [74](#)
- Shadow registers, [129](#)
- Shift counter, [126](#), [151](#)
 - defined, [42](#)
- Signature compression, [149](#)
- Slow clock generator, [74](#)
- Space compactor, [149](#)
 - defined, [41](#)
- Stretch mode, [85](#)
- Stuck-at testing, [81](#)
- T —
- Test
 - coverage, described, [92](#)
- Test points
 - inserting, [32](#)
 - multiple clock domains, [82](#), [86](#)
- Timing
 - analysis, [88](#)
 - asynchronous signals, diagram, [81](#)
 - BIST activation, diagram, [143](#)
 - BIST activation, diagrams, [144](#)
 - MTPI to core, diagram, [146](#)
 - single pattern to core, diagram, [145](#)
- Tool phases
 - BIST controller generation, [16](#)
 - BIST-ready, [21](#)
 - fault simulation and signature generation, [16](#)
- Transition faults
 - at-speed, [82](#)
- Tri-state controllers, [152](#)
- U —
- Unload
 - scan chains, [87](#)

Third-Party Information

For information about third-party software included with this release of Tessent products, refer to the [*Third-Party Software for Tessent Products*](#).



End-User License Agreement

The latest version of the End-User License Agreement is available on-line at:
www.mentor.com/eula

IMPORTANT INFORMATION

USE OF ALL SOFTWARE IS SUBJECT TO LICENSE RESTRICTIONS. CAREFULLY READ THIS LICENSE AGREEMENT BEFORE USING THE PRODUCTS. USE OF SOFTWARE INDICATES CUSTOMER'S COMPLETE AND UNCONDITIONAL ACCEPTANCE OF THE TERMS AND CONDITIONS SET FORTH IN THIS AGREEMENT. ANY ADDITIONAL OR DIFFERENT PURCHASE ORDER TERMS AND CONDITIONS SHALL NOT APPLY.

END-USER LICENSE AGREEMENT ("Agreement")

This is a legal agreement concerning the use of Software (as defined in Section 2) and hardware (collectively "Products") between the company acquiring the Products ("Customer"), and the Mentor Graphics entity that issued the corresponding quotation or, if no quotation was issued, the applicable local Mentor Graphics entity ("Mentor Graphics"). Except for license agreements related to the subject matter of this license agreement which are physically signed by Customer and an authorized representative of Mentor Graphics, this Agreement and the applicable quotation contain the parties' entire understanding relating to the subject matter and supersede all prior or contemporaneous agreements. If Customer does not agree to these terms and conditions, promptly return or, in the case of Software received electronically, certify destruction of Software and all accompanying items within five days after receipt of Software and receive a full refund of any license fee paid.

1. ORDERS, FEES AND PAYMENT.

- 1.1. To the extent Customer (or if agreed by Mentor Graphics, Customer's appointed third party buying agent) places and Mentor Graphics accepts purchase orders pursuant to this Agreement (each an "Order"), each Order will constitute a contract between Customer and Mentor Graphics, which shall be governed solely and exclusively by the terms and conditions of this Agreement, any applicable addenda and the applicable quotation, whether or not those documents are referenced on the Order. Any additional or conflicting terms and conditions appearing on an Order or presented in any electronic portal or automated order management system, whether or not required to be electronically accepted, will not be effective unless agreed in writing and physically signed by an authorized representative of Customer and Mentor Graphics.
- 1.2. Amounts invoiced will be paid, in the currency specified on the applicable invoice, within 30 days from the date of such invoice. Any past due invoices will be subject to the imposition of interest charges in the amount of one and one-half percent per month or the applicable legal rate currently in effect, whichever is lower. Prices do not include freight, insurance, customs duties, taxes or other similar charges, which Mentor Graphics will state separately in the applicable invoice. Unless timely provided with a valid certificate of exemption or other evidence that items are not taxable, Mentor Graphics will invoice Customer for all applicable taxes including, but not limited to, VAT, GST, sales tax, consumption tax and service tax. Customer will make all payments free and clear of, and without reduction for, any withholding or other taxes; any such taxes imposed on payments by Customer hereunder will be Customer's sole responsibility. If Customer appoints a third party to place purchase orders and/or make payments on Customer's behalf, Customer shall be liable for payment under Orders placed by such third party in the event of default.
- 1.3. All Products are delivered FCA factory (Incoterms 2010), freight prepaid and invoiced to Customer, except Software delivered electronically, which shall be deemed delivered when made available to Customer for download. Mentor Graphics retains a security interest in all Products delivered under this Agreement, to secure payment of the purchase price of such Products, and Customer agrees to sign any documents that Mentor Graphics determines to be necessary or convenient for use in filing or perfecting such security interest. Mentor Graphics' delivery of Software by electronic means is subject to Customer's provision of both a primary and an alternate e-mail address.

2. **GRANT OF LICENSE.** The software installed, downloaded, or otherwise acquired by Customer under this Agreement, including any updates, modifications, revisions, copies, documentation and design data ("Software") are copyrighted, trade secret and confidential information of Mentor Graphics or its licensors, who maintain exclusive title to all Software and retain all rights not expressly granted by this Agreement. Mentor Graphics grants to Customer, subject to payment of applicable license fees, a nontransferable, nonexclusive license to use Software solely: (a) in machine-readable, object-code form (except as provided in Subsection 5.2); (b) for Customer's internal business purposes; (c) for the term of the license; and (d) on the computer hardware and at the site authorized by Mentor Graphics. A site is restricted to a one-half mile (800 meter) radius. Customer may have Software temporarily used by an employee for telecommuting purposes from locations other than a Customer office, such as the employee's residence, an airport or hotel, provided that such employee's primary place of employment is the site where the Software is authorized for use. Mentor Graphics' standard policies and programs, which vary depending on Software, license fees paid or services purchased, apply to the following: (a) relocation of Software; (b) use of Software, which may be limited, for example, to execution of a single session by a single user on the authorized hardware or for a restricted period of time (such limitations may be technically implemented through the use of authorization codes or similar devices); and (c) support services provided, including eligibility to receive telephone support, updates, modifications, and revisions. For the avoidance of doubt, if Customer provides any feedback or requests any change or enhancement to Products, whether in the course of receiving support or consulting services, evaluating Products, performing beta testing or otherwise, any inventions, product improvements, modifications or developments made by Mentor Graphics (at Mentor Graphics' sole discretion) will be the exclusive property of Mentor Graphics.
3. **ESC SOFTWARE.** If Customer purchases a license to use development or prototyping tools of Mentor Graphics' Embedded Software Channel ("ESC"), Mentor Graphics grants to Customer a nontransferable, nonexclusive license to reproduce and distribute executable

files created using ESC compilers, including the ESC run-time libraries distributed with ESC C and C++ compiler Software that are linked into a composite program as an integral part of Customer's compiled computer program, provided that Customer distributes these files only in conjunction with Customer's compiled computer program. Mentor Graphics does NOT grant Customer any right to duplicate, incorporate or embed copies of Mentor Graphics' real-time operating systems or other embedded software products into Customer's products or applications without first signing or otherwise agreeing to a separate agreement with Mentor Graphics for such purpose.

4. BETA CODE.

- 4.1. Portions or all of certain Software may contain code for experimental testing and evaluation (which may be either alpha or beta, collectively "Beta Code"), which may not be used without Mentor Graphics' explicit authorization. Upon Mentor Graphics' authorization, Mentor Graphics grants to Customer a temporary, nontransferable, nonexclusive license for experimental use to test and evaluate the Beta Code without charge for a limited period of time specified by Mentor Graphics. Mentor Graphics may choose, at its sole discretion, not to release Beta Code commercially in any form.
- 4.2. If Mentor Graphics authorizes Customer to use the Beta Code, Customer agrees to evaluate and test the Beta Code under normal conditions as directed by Mentor Graphics. Customer will contact Mentor Graphics periodically during Customer's use of the Beta Code to discuss any malfunctions or suggested improvements. Upon completion of Customer's evaluation and testing, Customer will send to Mentor Graphics a written evaluation of the Beta Code, including its strengths, weaknesses and recommended improvements.
- 4.3. Customer agrees to maintain Beta Code in confidence and shall restrict access to the Beta Code, including the methods and concepts utilized therein, solely to those employees and Customer location(s) authorized by Mentor Graphics to perform beta testing. Customer agrees that any written evaluations and all inventions, product improvements, modifications or developments that Mentor Graphics conceived or made during or subsequent to this Agreement, including those based partly or wholly on Customer's feedback, will be the exclusive property of Mentor Graphics. Mentor Graphics will have exclusive rights, title and interest in all such property. The provisions of this Subsection 4.3 shall survive termination of this Agreement.

5. RESTRICTIONS ON USE.

- 5.1. Customer may copy Software only as reasonably necessary to support the authorized use. Each copy must include all notices and legends embedded in Software and affixed to its medium and container as received from Mentor Graphics. All copies shall remain the property of Mentor Graphics or its licensors. Customer shall maintain a record of the number and primary location of all copies of Software, including copies merged with other software, and shall make those records available to Mentor Graphics upon request. Customer shall not make Products available in any form to any person other than Customer's employees and on-site contractors, excluding Mentor Graphics competitors, whose job performance requires access and who are under obligations of confidentiality. Customer shall take appropriate action to protect the confidentiality of Products and ensure that any person permitted access does not disclose or use Products except as permitted by this Agreement. Customer shall give Mentor Graphics written notice of any unauthorized disclosure or use of the Products as soon as Customer becomes aware of such unauthorized disclosure or use. Except as otherwise permitted for purposes of interoperability as specified by applicable and mandatory local law, Customer shall not reverse-assemble, reverse-compile, reverse-engineer or in any way derive any source code from Software. Log files, data files, rule files and script files generated by or for the Software (collectively "Files"), including without limitation files containing Standard Verification Rule Format ("SVRF") and Tcl Verification Format ("TVF") which are Mentor Graphics' trade secret and proprietary syntaxes for expressing process rules, constitute or include confidential information of Mentor Graphics. Customer may share Files with third parties, excluding Mentor Graphics competitors, provided that the confidentiality of such Files is protected by written agreement at least as well as Customer protects other information of a similar nature or importance, but in any case with at least reasonable care. Customer may use Files containing SVRF or TVF only with Mentor Graphics products. Under no circumstances shall Customer use Products or Files or allow their use for the purpose of developing, enhancing or marketing any product that is in any way competitive with Products, or disclose to any third party the results of, or information pertaining to, any benchmark.
- 5.2. If any Software or portions thereof are provided in source code form, Customer will use the source code only to correct software errors and enhance or modify the Software for the authorized use. Customer shall not disclose or permit disclosure of source code, in whole or in part, including any of its methods or concepts, to anyone except Customer's employees or on-site contractors, excluding Mentor Graphics competitors, with a need to know. Customer shall not copy or compile source code in any manner except to support this authorized use.
- 5.3. Customer may not assign this Agreement or the rights and duties under it, or relocate, sublicense, or otherwise transfer the Products, whether by operation of law or otherwise ("Attempted Transfer"), without Mentor Graphics' prior written consent and payment of Mentor Graphics' then-current applicable relocation and/or transfer fees. Any Attempted Transfer without Mentor Graphics' prior written consent shall be a material breach of this Agreement and may, at Mentor Graphics' option, result in the immediate termination of the Agreement and/or the licenses granted under this Agreement. The terms of this Agreement, including without limitation the licensing and assignment provisions, shall be binding upon Customer's permitted successors in interest and assigns.
- 5.4. The provisions of this Section 5 shall survive the termination of this Agreement.

6. **SUPPORT SERVICES.** To the extent Customer purchases support services, Mentor Graphics will provide Customer with updates and technical support for the Products, at the Customer site(s) for which support is purchased, in accordance with Mentor Graphics' then current End-User Support Terms located at <http://supportnet.mentor.com/supportterms>.

7. LIMITED WARRANTY.

- 7.1. Mentor Graphics warrants that during the warranty period its standard, generally supported Products, when properly installed, will substantially conform to the functional specifications set forth in the applicable user manual. Mentor Graphics does not warrant that Products will meet Customer's requirements or that operation of Products will be uninterrupted or error free. The

warranty period is 90 days starting on the 15th day after delivery or upon installation, whichever first occurs. Customer must notify Mentor Graphics in writing of any nonconformity within the warranty period. For the avoidance of doubt, this warranty applies only to the initial shipment of Software under an Order and does not renew or reset, for example, with the delivery of (a) Software updates or (b) authorization codes or alternate Software under a transaction involving Software re-mix. This warranty shall not be valid if Products have been subject to misuse, unauthorized modification, improper installation or Customer is not in compliance with this Agreement. MENTOR GRAPHICS' ENTIRE LIABILITY AND CUSTOMER'S EXCLUSIVE REMEDY SHALL BE, AT MENTOR GRAPHICS' OPTION, EITHER (A) REFUND OF THE PRICE PAID UPON RETURN OF THE PRODUCTS TO MENTOR GRAPHICS OR (B) MODIFICATION OR REPLACEMENT OF THE PRODUCTS THAT DO NOT MEET THIS LIMITED WARRANTY. MENTOR GRAPHICS MAKES NO WARRANTIES WITH RESPECT TO: (A) SERVICES; (B) PRODUCTS PROVIDED AT NO CHARGE; OR (C) BETA CODE; ALL OF WHICH ARE PROVIDED "AS IS."

7.2. THE WARRANTIES SET FORTH IN THIS SECTION 7 ARE EXCLUSIVE. NEITHER MENTOR GRAPHICS NOR ITS LICENSORS MAKE ANY OTHER WARRANTIES EXPRESS, IMPLIED OR STATUTORY, WITH RESPECT TO PRODUCTS PROVIDED UNDER THIS AGREEMENT. MENTOR GRAPHICS AND ITS LICENSORS SPECIFICALLY DISCLAIM ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NON-INFRINGEMENT OF INTELLECTUAL PROPERTY.

8. **LIMITATION OF LIABILITY.** EXCEPT WHERE THIS EXCLUSION OR RESTRICTION OF LIABILITY WOULD BE VOID OR INEFFECTIVE UNDER APPLICABLE LAW, IN NO EVENT SHALL MENTOR GRAPHICS OR ITS LICENSORS BE LIABLE FOR INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES (INCLUDING LOST PROFITS OR SAVINGS) WHETHER BASED ON CONTRACT, TORT OR ANY OTHER LEGAL THEORY, EVEN IF MENTOR GRAPHICS OR ITS LICENSORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES. IN NO EVENT SHALL MENTOR GRAPHICS' OR ITS LICENSORS' LIABILITY UNDER THIS AGREEMENT EXCEED THE AMOUNT RECEIVED FROM CUSTOMER FOR THE HARDWARE, SOFTWARE LICENSE OR SERVICE GIVING RISE TO THE CLAIM. IN THE CASE WHERE NO AMOUNT WAS PAID, MENTOR GRAPHICS AND ITS LICENSORS SHALL HAVE NO LIABILITY FOR ANY DAMAGES WHATSOEVER. THE PROVISIONS OF THIS SECTION 8 SHALL SURVIVE THE TERMINATION OF THIS AGREEMENT.

9. **HAZARDOUS APPLICATIONS.** CUSTOMER ACKNOWLEDGES IT IS SOLELY RESPONSIBLE FOR TESTING ITS PRODUCTS USED IN APPLICATIONS WHERE THE FAILURE OR INACCURACY OF ITS PRODUCTS MIGHT RESULT IN DEATH OR PERSONAL INJURY ("HAZARDOUS APPLICATIONS"). EXCEPT TO THE EXTENT THIS EXCLUSION OR RESTRICTION OF LIABILITY WOULD BE VOID OR INEFFECTIVE UNDER APPLICABLE LAW, IN NO EVENT SHALL MENTOR GRAPHICS OR ITS LICENSORS BE LIABLE FOR ANY DAMAGES RESULTING FROM OR IN CONNECTION WITH THE USE OF MENTOR GRAPHICS PRODUCTS IN OR FOR HAZARDOUS APPLICATIONS. THE PROVISIONS OF THIS SECTION 9 SHALL SURVIVE THE TERMINATION OF THIS AGREEMENT.

10. **INDEMNIFICATION.** CUSTOMER AGREES TO INDEMNIFY AND HOLD HARMLESS MENTOR GRAPHICS AND ITS LICENSORS FROM ANY CLAIMS, LOSS, COST, DAMAGE, EXPENSE OR LIABILITY, INCLUDING ATTORNEYS' FEES, ARISING OUT OF OR IN CONNECTION WITH THE USE OF MENTOR GRAPHICS PRODUCTS IN OR FOR HAZARDOUS APPLICATIONS. THE PROVISIONS OF THIS SECTION 10 SHALL SURVIVE THE TERMINATION OF THIS AGREEMENT.

11. INFRINGEMENT.

11.1. Mentor Graphics will defend or settle, at its option and expense, any action brought against Customer in the United States, Canada, Japan, or member state of the European Union which alleges that any standard, generally supported Product acquired by Customer hereunder infringes a patent or copyright or misappropriates a trade secret in such jurisdiction. Mentor Graphics will pay costs and damages finally awarded against Customer that are attributable to such action. Customer understands and agrees that as conditions to Mentor Graphics' obligations under this section Customer must: (a) notify Mentor Graphics promptly in writing of the action; (b) provide Mentor Graphics all reasonable information and assistance to settle or defend the action; and (c) grant Mentor Graphics sole authority and control of the defense or settlement of the action.

11.2. If a claim is made under Subsection 11.1 Mentor Graphics may, at its option and expense: (a) replace or modify the Product so that it becomes noninfringing; (b) procure for Customer the right to continue using the Product; or (c) require the return of the Product and refund to Customer any purchase price or license fee paid, less a reasonable allowance for use.

11.3. Mentor Graphics has no liability to Customer if the action is based upon: (a) the combination of Software or hardware with any product not furnished by Mentor Graphics; (b) the modification of the Product other than by Mentor Graphics; (c) the use of other than a current unaltered release of Software; (d) the use of the Product as part of an infringing process; (e) a product that Customer makes, uses, or sells; (f) any Beta Code or Product provided at no charge; (g) any software provided by Mentor Graphics' licensors who do not provide such indemnification to Mentor Graphics' customers; or (h) infringement by Customer that is deemed willful. In the case of (h), Customer shall reimburse Mentor Graphics for its reasonable attorney fees and other costs related to the action.

11.4. THIS SECTION 11 IS SUBJECT TO SECTION 8 ABOVE AND STATES THE ENTIRE LIABILITY OF MENTOR GRAPHICS AND ITS LICENSORS, AND CUSTOMER'S SOLE AND EXCLUSIVE REMEDY, FOR DEFENSE, SETTLEMENT AND DAMAGES, WITH RESPECT TO ANY ALLEGED PATENT OR COPYRIGHT INFRINGEMENT OR TRADE SECRET MISAPPROPRIATION BY ANY PRODUCT PROVIDED UNDER THIS AGREEMENT.

12. TERMINATION AND EFFECT OF TERMINATION.

12.1. If a Software license was provided for limited term use, such license will automatically terminate at the end of the authorized term. Mentor Graphics may terminate this Agreement and/or any license granted under this Agreement immediately upon written notice if Customer: (a) exceeds the scope of the license or otherwise fails to comply with the licensing or confidentiality provisions of this Agreement, or (b) becomes insolvent, files a bankruptcy petition, institutes proceedings for liquidation or winding up or enters into an agreement to assign its assets for the benefit of creditors. For any other material breach of any

provision of this Agreement, Mentor Graphics may terminate this Agreement and/or any license granted under this Agreement upon 30 days written notice if Customer fails to cure the breach within the 30 day notice period. Termination of this Agreement or any license granted hereunder will not affect Customer's obligation to pay for Products shipped or licenses granted prior to the termination, which amounts shall be payable immediately upon the date of termination.

- 12.2. Upon termination of this Agreement, the rights and obligations of the parties shall cease except as expressly set forth in this Agreement. Upon termination, Customer shall ensure that all use of the affected Products ceases, and shall return hardware and either return to Mentor Graphics or destroy Software in Customer's possession, including all copies and documentation, and certify in writing to Mentor Graphics within ten business days of the termination date that Customer no longer possesses any of the affected Products or copies of Software in any form.
13. **EXPORT.** The Products provided hereunder are subject to regulation by local laws and United States ("U.S.") government agencies, which prohibit export, re-export or diversion of certain products, information about the products, and direct or indirect products thereof, to certain countries and certain persons. Customer agrees that it will not export or re-export Products in any manner without first obtaining all necessary approval from appropriate local and U.S. government agencies. If Customer wishes to disclose any information to Mentor Graphics that is subject to any U.S. or other applicable export restrictions, including without limitation the U.S. International Traffic in Arms Regulations (ITAR) or special controls under the Export Administration Regulations (EAR), Customer will notify Mentor Graphics personnel, in advance of each instance of disclosure, that such information is subject to such export restrictions.
14. **U.S. GOVERNMENT LICENSE RIGHTS.** Software was developed entirely at private expense. The parties agree that all Software is commercial computer software within the meaning of the applicable acquisition regulations. Accordingly, pursuant to U.S. FAR 48 CFR 12.212 and DFAR 48 CFR 227.7202, use, duplication and disclosure of the Software by or for the U.S. government or a U.S. government subcontractor is subject solely to the terms and conditions set forth in this Agreement, which shall supersede any conflicting terms or conditions in any government order document, except for provisions which are contrary to applicable mandatory federal laws.
15. **THIRD PARTY BENEFICIARY.** Mentor Graphics Corporation, Mentor Graphics (Ireland) Limited, Microsoft Corporation and other licensors may be third party beneficiaries of this Agreement with the right to enforce the obligations set forth herein.
16. **REVIEW OF LICENSE USAGE.** Customer will monitor the access to and use of Software. With prior written notice and during Customer's normal business hours, Mentor Graphics may engage an internationally recognized accounting firm to review Customer's software monitoring system and records deemed relevant by the internationally recognized accounting firm to confirm Customer's compliance with the terms of this Agreement or U.S. or other local export laws. Such review may include FlexNet (or successor product) report log files that Customer shall capture and provide at Mentor Graphics' request. Customer shall make records available in electronic format and shall fully cooperate with data gathering to support the license review. Mentor Graphics shall bear the expense of any such review unless a material non-compliance is revealed. Mentor Graphics shall treat as confidential information all information gained as a result of any request or review and shall only use or disclose such information as required by law or to enforce its rights under this Agreement. The provisions of this Section 16 shall survive the termination of this Agreement.
17. **CONTROLLING LAW, JURISDICTION AND DISPUTE RESOLUTION.** The owners of certain Mentor Graphics intellectual property licensed under this Agreement are located in Ireland and the U.S. To promote consistency around the world, disputes shall be resolved as follows: excluding conflict of laws rules, this Agreement shall be governed by and construed under the laws of the State of Oregon, U.S., if Customer is located in North or South America, and the laws of Ireland if Customer is located outside of North or South America. All disputes arising out of or in relation to this Agreement shall be submitted to the exclusive jurisdiction of the courts of Portland, Oregon when the laws of Oregon apply, or Dublin, Ireland when the laws of Ireland apply. Notwithstanding the foregoing, all disputes in Asia arising out of or in relation to this Agreement shall be resolved by arbitration in Singapore before a single arbitrator to be appointed by the chairman of the Singapore International Arbitration Centre ("SIAC") to be conducted in the English language, in accordance with the Arbitration Rules of the SIAC in effect at the time of the dispute, which rules are deemed to be incorporated by reference in this section. Nothing in this section shall restrict Mentor Graphics' right to bring an action (including for example a motion for injunctive relief) against Customer in the jurisdiction where Customer's place of business is located. The United Nations Convention on Contracts for the International Sale of Goods does not apply to this Agreement.
18. **SEVERABILITY.** If any provision of this Agreement is held by a court of competent jurisdiction to be void, invalid, unenforceable or illegal, such provision shall be severed from this Agreement and the remaining provisions will remain in full force and effect.
19. **MISCELLANEOUS.** This Agreement contains the parties' entire understanding relating to its subject matter and supersedes all prior or contemporaneous agreements. Some Software may contain code distributed under a third party license agreement that may provide additional rights to Customer. Please see the applicable Software documentation for details. This Agreement may only be modified in writing, signed by an authorized representative of each party. Waiver of terms or excuse of breach must be in writing and shall not constitute subsequent consent, waiver or excuse.